VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A Web-Based Restaurant Management System

By
Trần Quốc Nam - ITITIU21250

A thesis submitted to the School of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
Bachelor of Information Technology

Ho Chi Minh City, Vietnam
Year
2026

A Web-Based Restaurant Management System

APPROVED BY:

_____ ,

Assoc. Prof. Nguyen Van Sinh

THESIS COMMITTEE
(Whichever applies)

ACKNOWLEDGMENTS

First of all, I would like to express my sincere gratitude to ward everyone who supported me throughout the completion of this thesis.

I am thankful to my supervisor, Assoc. Prof. Nguyen Van Sinh, for his invaluable guidance, patience, and constructive feedback throughout this thesis progression. His expertise and encouragement were significantly shaped this work.

I am also grateful to School of Computer Science at International University for providing the necessary opportunities to carry out this thesis.

Finally, I owe my deepest gratitude to my family for their unwavering love, patience, and support throughout my academic journey. This achievement would not have been possible without them.

TABLE OF CONTENTS

ACKNOWLEDGMENTS....3
TABLE OF CONTENTS....4
ABSTRACT....10
CHAPTER 1....11
INTRODUCTION....11
1.1. Background....11
1.2. Problem Statement....11
1.3. Scope and Objectives....12
1.3.1. Scope....12
1.3.2. Objectives....12
1.4. Assumption and Solution....13
1.4.1. Assumption....13
1.4.2. Solution....13
1.5. Structure of thesis....14
CHAPTER 2....15
LITERATURE REVIEW....15
2.1. Overview....15
2.2. KFC web platform [29]....15
2.2.1. Interface design and navigation....15
2.2.2. Order customization and state management....16
2.2.3. Checkout flow and conversion....17
2.3. Haidilao website platform [30]....17
2.3.1. Brand translation and user engagement....18
2.3.2. Menu categorization and Cross-Brand expansion....18
2.3.3. Off-Premise Dining and Delivery Operations....19
2.4. Technologies Analysis....20
2.4.1. Client-Side Architecture and Front-end Frameworks....21
2.4.1.1. Single-Page Applications....21
2.4.1.2. React.js: Component Model and Virtual DOM....21
2.4.1.3. Comparative Analysis: React vs. Vue.js vs. Angular....21
2.4.1.4. Styling Architecture: Tailwind CSS....22
2.4.2. Server-Side Logic and API Implementation....22
2.4.2.1. Requirements for Restaurant Back-end Systems....22
2.4.2.2. FastAPI: Asynchronous Python Framework....23

2.4.2.3. Comparative Analysis: FastAPI vs. Express.js vs. Django.....23
2.4.2.4. RESTful API Design and JWT Authentication.....24
2.4.3. Database Systems and Relational Integrity.....24
2.4.3.1. Data Integrity Requirements in F&B Systems.....24
2.4.3.2. PostgreSQL: Object-Relational Database System.....24
2.4.3.3. Comparative Analysis: PostgreSQL vs. MySQL vs. MongoDB.....25
2.4.3.4. SQLAlchemy: Object-Relational Mapping.....26
2.4.4. Full-Stack Integration and Justification.....26
2.4.4.1. Stack Synergy: React + FastAPI + PostgreSQL.....26
2.4.4.2. Alternative Stack Comparison.....27
2.5. Technique Review.....27
CHAPTER 3.....30
METHODOLOGY.....30
3.1. Overview.....30
3.2. System Overview.....30
3.2.1. Introduction.....30
3.2.2. Overall System Architecture.....30
3.2.3. Layer Architecture.....31
3.2.3.1. Model Layer.....32
3.2.3.2. View Layer.....32
3.2.3.3. Controller Layer.....33
3.2.4. Technology Stack.....34
3.2.5. Technologies justification.....35
3.2.5.1. Front-end Layer.....35
3.2.5.2. Back-end Layer.....36
3.2.5.3. Database Layer.....37
3.3. User requirement analysis.....37
3.3.1. Actor identification and Role definitions.....37
3.3.2. Use case descriptions.....39
3.3.3. Functional Test Cases.....48
3.3.4. Functional Requirements.....50
3.3.4.1. Restaurant Manager Functional Requirements.....50
3.3.4.2. Delivery Personnel Functional Requirements.....51
3.3.4.3. Chef Functional Requirements.....52
3.3.4.4. Waiter Functional Requirements.....52
3.3.4.5. Customer Functional Requirements.....53

3.3.5. Non-Functional Requirements....54
3.3.6. System Data Flow and Interaction....55
3.3.6.1. Sequence flow....55
3.3.6.2. State Diagram....56
3.3.6.3. Data Flow....57
3.4. Communication Mechanisms....58
3.5. Database....59
3.5.1. Database Design....59
3.5.2. Database Relationship....59
3.6. Data Structure....60
3.7. Algorithms and Process Logic....61
3.7.1. FIFO Staff Dispatching Algorithm....61
3.7.2. Finite State Machine (FSM) Order Lifecycle Algorithm....61
3.7.3. Bcrypt Adaptive Key-Derivation Cryptographic Algorithm:....62
3.8. UI/UX Design....62
CHAPTER 4....64
IMPLEMENTATION....64
4.1. Overview....64
4.1.1. Front-end implementation....64
4.1.1.1. Dependencies....64
4.1.1.2. Structure and Organization....65
4.1.1.3. State Management Architecture....65
4.1.1.4. Routing and Navigation Architecture....66
4.1.1.5. Authenticated Data Fetching....67
4.1.2. Back-end implementation....68
4.1.2.1. Dependencies....68
4.1.2.2. Layer Architecture and Modules Organization....69
4.1.2.3. Business Logic....71
4.1.2.3.1. Authentication and Authorization Implementation....71
4.1.2.3.2. FIFO Staff Dispatching Algorithm implementation....72
4.1.2.3.3. Finite State Machine Algorithm implementation....73
4.1.2.3.4. Payment Processing and VietQR Generation....74
4.1.2.3.5. Table and Reservation....74
4.1.2.3.6. Coupon Thresholds....75
4.1.3. Database implementation....75
4.2. Results....76

4.2.1. Authentication and Access Control....76
4.2.2. Customer Ordering and Service Experience Interfaces....77
4.2.3. Staff Workstations and Kitchen Display Systems (KDS)....79
4.2.4. Manager Dashboard....80
4.2.5. Table Reservation and Pre-order Management Interfaces....82
4.2.6. Promotional Discount and Coupon Management Interfaces....83
CHAPTER 5....85
DISCUSSION AND EVALUATION....85
5.1. Discussion....85
5.1.1. "Pay-First" Operational Model....85
5.1.2. Technical Architecture and Maintainability....85
5.2. Comparison....85
5.2.1. Compare with Haidilao....85
5.2.2. Compare with KFC....86
5.3. Limitations....86
5.3.1. Performance....86
5.3.2. Scalability....86
CHAPTER 6....88
CONCLUSION AND FUTURE WORK....88
6.1. Conclusion....88
6.2. Future work....88
6.2.1. Mobile applications....88
6.2.2. AI-driven Analytics....88
6.2.3. Automated Inventory Management....89
REFERENCES....90

List of Figures

Figure 2.2 KFC Homepage[29].....16
Figure 2.3 KFC Order customization [29].....17
Figure 2.4 KFC Checkout flow [29].....18
Figure 2.5 Haidilao homepage [30].....19
Figure 2.6 Haidilao menu category [30].....20
Figure 2.7 Bring Haidilao Home Feature [30].....21
Figure 3.2: Restaurant Management System Architecture.....31
Figure 3.3 Use case diagram.....39
Figure 3.4: Customer Order Placement Sequence.....57
Figure 3.5: State Diagram.....58
Figure 3.6 System Data Flow.....59
Figure 3.7 Database Design.....60
Figure 4.1 front-end dependency declarations.....65
Figure 4.2: Front-end structure.....66
Figure 4.3: Login function and token management.....67
Figure 4.4 Auto-refreshing server state.....68
Figure 4.5 Authentication Interceptor implementation.....69
Figure 4.6 FastAPI initialization, CORS, and router.....69
Figure 4.7 Back-end structure.....70
Figure 4.8 Request/Response DTOs.....71
Figure 4.9 Password hashing service.....72
Figure 4.10 JWT token generation.....72
Figure 4.11 Authorization service implement.....73
Figure 4.12 FIFO Staff Dispatching implementation.....74
Figure 4.13 Finite State Machine Algorithm implementation.....74
Figure 4.14 Payment Processing implementation.....75
Figure 4.15 Database schema automated generation.....77
Figure 4.16 Login and Authentication.....78
Figure 4.17 Menu Page.....78
Figure 4.18 Cart Drawer and Checkout Interface.....79
Figure 4.19 Order Tracking and Review Submission Interface.....79
Figure 4.20 Kitchen Display System.....80
Figure 4.21 Waiter and Delivery Workstation Interface.....81
Figure 4.22 Manager Overview and Metric Analytics Interface.....81
Figure 4.23: Menu Management Interface.....82
Figure 4.24: Table Management Interface.....82
Figure 4.25: Scheduling Interface.....83
Figure 4.26: Customer Table Reservation and Pre-order Interface.....83
Figure 4.27 Reservation Management and Staff Assignment Interface.....84
Figure 4.28: Customer Coupon Redemption.....85
Figure 4.29: Discount Threshold and Coupon Configuration Interface.....85

Index of Tables

Table2.4.4.2 Alternative Stack Comparison 28
Table3.2.3.1 Model Layer 33
Table3.2.3.2 View Layer 33
Table3.2.3.3 Controller Layer 34
Table3.2.1.2 Technology stack summary. 35
Table 3.3.1 Actors Identification and Role Definition 39
Table 3.3.2.1 Use case description for Login 40
Table 3.3.2.2 Use case description for Register customer account 41
Table 3.3.2.3 Use case description for Browse Menu 42
Table 3.3.2.4 Use case description for Manage Cart 43
Table 3.3.2.5 Use case description for Place Dine-in Ordering 44
Table 3.3.2.6 Use case description for Shipping Ordering 46
Table 3.3.2.7 Use case description for Chef Manages Kitchen Orders 47
Table 3.3.2.8 Use case description for Waiter Active Orders 47
Table 3.3.2.9 Use case description for Delivery Man Shipping Orders 48
Table 3.3.2.10 Use case description for Manager Views Dashboard Analytics 50
Table 3.3.3 Functional Test Cases 50
Table 3.3.3.1 Manager Functional Requirements 52
Table 3.3.3.2 Delivery Personnel Functional Requirements 53
Table 3.3.3.3 Chef Functional Requirements 54
Table 3.3.3.4 Waiter Functional Requirements 54
Table 3.3.3.5 Customer Functional Requirements 55
Table 3.3.4 Non-functional requirements 56
Table 3.6.2 Entities Relationship 61
Table 4.1.1.4 Front-end routes 67
Table 4.1.2.1 Back-end dependencies 69
Table 4.1.2.2 Back-end layers for module Order 71
Table 5.2 Features of 3 systems 86
Table 5.2.1 Comparison with Haidilao 86
Table 5.2.2 Comparison with KFC 87

ABSTRACT

This thesis proposes the development of a web-based Restaurant Management System, an online-based platform aimed to combining serving service, kitchen operation, and management into a single ecosystem.

The system is built with a modern and high performance full-stack architecture by tweaking FastAPI for high performance in back-end layer, React for responsive front-end while PostgresSQL acts as a database for entire system. This thesis implement the "Pay-First" operation model with integrated digital payment solution such as VietQR.

Notable features that this system provides are table selection, multi-channel ordering options for in-store and shipping, real-time kitchen display operation, and an administrative dashboard provides business analytic, management tools such as staffs management, menu optimizing, and table adjustment.

Keywords: Restaurant Management System, FastAPI, React, Pay-First Model, Kitchen Display System, Digital Transformation, Single Page Application

CHAPTER 1

INTRODUCTION

1.1. Background

In the modern global economy, digital technology has developed rapidly leads to the fact that how businesses and consumers have transformed entirely in order to adapt with digital era. Among with these advancements, e-wallet and digital payment systems naturally emerged to dominant the current market by offering fast and secure transaction methods while ignoring distance problems. As a result, the growth of ecosystem in digital financial has record a massive increasing in internet connectivity, smartphones, and cashless payment. Apparently, consumers now expect seamless payment experiences, while businesses taking advantage from fast transactions, evolution of financial tracking and reach a wider customer base.

As a matter of course, organizations across multiple sectors have significantly taken steps forward to digital transformation. Companies integrate e-wallet payment gateways into their operations to enhance customer convenience. Academic institutions take advantages of online payment systems for tuition fees, campus services to reduce administrative workload and enhance efficiency. Service industries such as retail, hospitality, transportation, entertainment started to apply cashless and contactless payment options on digital platforms. Governments have also promoted digital payment throughout tax systems, public transportation payments.

This widespread transition proves that digital payments technology is no longer optional but essential for efficient operations and customer satisfaction. As a result, restaurant industries take a step to digital platform in order to expand market reach, reduce manual errors, utilize restaurant management and enhance customer experience.

1.2. Problem Statement

Managing daily operations manually put a significant challenge for traditional restaurants. Tasks such as taking order, managing reservations, processing payments, adjust employee shift are often results in wasting time and prone to human error. Customers may experience long waiting times, incorrect orders ,or limited payments options, while employee suffered from manually handling multiple tasks. Business owner struggle to track operations effectively.

The lack of integration in traditional restaurant management results in fragmented data, making it difficult for monitoring accurately in sales, customer preferences, stock levels and staff productivity. In addition, the absence of real-time information limits the ability in decision making of managers in term of menu, schedules, discounts. As an inevitable result, this creates inefficient operations and reduces the restaurant's ability to respond to customer demand and market trends.

In this regard, the web-based restaurant management software seeks to address these problems through integrating the necessary features on one platform. This will involve incorporating functionalities such as menu viewing online, order placement, handling reservations, making payments via electronic wallets, monitoring stock levels, managing customers' accounts, and reporting sales figures. The system will enable efficient communication between customers, employees, and managers thanks to its user-friendly interface and availability of up-to-date information.

1.3. Scope and Objectives

1.3.1. Scope

The thesis is designed to create a full-fledged web-based management system for restaurants that will facilitate operational and customer service activities within the restaurant business. The proposed system will help the users involved in this process to streamline the work processes and increase efficiency in their work. Such functions will include online menu navigation and ordering, reservation handling, customer account management, order tracking and payments, e-wallet integration, stock and inventory management, financial analytics, notifications, and user authentication via JWT [19] with role-based access control. The UI of the system will be made intuitive enough so that its operation was efficient and user-friendly.

1.3.2. Objectives

The objective of the thesis is to make changes to restaurant activities and provide an integrated system, which would be easier to use from the perspective of both customers and management activities. In order to achieve this goal, several objectives have been set, such as:

- **Digital Order Management**

The introduction of digital order management will help replace traditional order taking activities, thus improving accuracy and efficiency. Clients will be able to view the menu, make custom orders and send their requests via the internet. In addition, the system will allow updating orders on time.

- **Menu and Table Reservations Management System**

This module involves designing of the centralized menu and table reservations system where a manager can modify, delete or add menu items. All the details of the menu items like prices, availability, and descriptions will be displayed here. It will allow customers to make table reservations, and at the same time, the management will manage table arrangements.

- **Customer and Users Management**

In addition to that, a feature designed to manage customer, users, and staff user accounts by performing various actions such as creating, editing, and deleting user accounts of all three types of people. Customers will be able to update their information, preferences, and orders' histories and on the other hand, the management will also manage staff user account details.

- **Payment System**

Another module of our thesis is a system through which we will allow payments using e-wallets and digital payment gateways along with cash payment which will provide more options, easy and safe payment to customers.

- **Notification System**

Features for notifying the users regarding the confirmation of orders, reservations, promotions, and other important notices will be incorporated in the app. They will improve communications and interactions between the users and the restaurant personnel.

- **Sales monitoring**

Sales reports will be created by monitoring completed payments. Helping manager or business owner follow with customers preferences.

- **Authentication and Security**

A reliable system of authentication will be used to verify users' identity and protect the security of the business information. Role-based restrictions will also be applied to secure the information from any unwanted access. Moreover, a convenient login interface will include password reset functionality and other features.

1.4. Assumption and Solution

1.4.1. Assumption

The design and development of the web-based restaurant management system will consider the system system requirements, users behavior, and overall thesis scope. The system is expected to provides both customer and staff members a responsive platform to manage and track order activities effectively, such as menu browsing and order placement. Additionally, prioritize a secure, intuitive and seamless digital experiences for users. The system will stores and manages structured restaurant data within a relational database. To meet these requirements, the platform will leverage a robust, modern technology stack which are React.js framework for building front-end, FastAPI (Python) framework for setting up business logic, security for back-end layer, and PostgreSQL for relational database. For data transfer between client and server side becomes seamlessly and secured user credential, JSON will be utilized by implementation of RESTful APIs and cryptographic protocols for password hashing to standard privacy practices.

1.4.2. Solution

Based on outlined assumptions and objective, the solution for designing the system will be implemented in the following steps:

- **Menu management and data presentation**

- **Front-end** (React.js, Axios, UI Framework)**:** React will be used to build and design a user-friendly interface menu page where customer can browse menu, filter items by categories, prices. Axios will in charge of sending HTTP requests to fetch data in JSON format from back-end layer.
- **Back-end** (FastAPI, PostgreSQL, SQLAlchemy)**:** business logic will be handled by FastAPI. PostgreSQL will store and manage all data related to menu in relational structured data. While SQLAlchemy will serve as Object-Relational Mapper (ORM) that streamlines query operation and ensure accuracy in data transferring to front-end.

- **Order processing and management**

- **Front-end** (React.js, State Management): provides a shopping cart system that allows customer to add items, adjust quantities, and review selected items before process purchasing. After the order placement, React will package the order data and send it to the server in a secure manner. User interface provides order status tracking in real-time.
- **Back-end** (FastAPI, PostgreSQL): FastAPI will validate data in incoming order request and extract item price from PostgreSQL to perform total amount then return payment page to front-end. Upon successful payment will update database to reflect order stage.

- **Authentication and security**

- **Front-end (React.js, Axios):** The application provides an intuitive interface that support user login, registration and profile management. Axios securely transfer user input credential to

back-end by using token bearer. Upon login successfully, user will receive authentication token stored in browser's local storage which will redirect user to suitable role based page.

- **Back-end (FastAPI, Cryptography Libraries):** FastAPI will securely authenticate requests and use Role-Based Access Control to separate regular customers from restaurant administrators. User credentials will be stored securely in PostgreSQL database, with passwords being hashed to avoid unauthorized access to this data. Users who get authenticated will receive JSON Web Tokens (JWT), and there will be middle ware in the FastAPI application that will validate the JWT.

1.5. Structure of thesis

Chapter 1: Introduction

Explaining the background and purpose of the topic and providing an overview of the study. Define problems and clarify objectives of the thesis. The chapter concludes the research by highlighting the scope and solution.

Chapter 2: Literature Review

Reviews relevant studies that related to the topic. Analyzes the history and fundamental concepts in the field while pointing out the weaknesses of the prior research. The chapter also explores the technological of the system proposed.

Chapter 3: Methodology

Discusses the technologies and techniques that have been used for the development of the system. The chapter gives details about the design of the system, the data structure used, and the analytical techniques used.

Chapter 4: Implementation

Provides the procedures followed in developing the system from scratch to its actual implementation.
This chapter outlines the application of the methodology used in Chapter Three.
The chapter presents the outcomes, showing how the system operates in reality.

Chapter 5: Discussion

Evaluates the performance of the system by examining both its strong and weak sides. The chapter also compares the proposed solution with those that exist, giving a critical analysis of the findings obtained.

Chapter 6: Conclusion and Future Work

Provides a synopsis of the major findings of the thesis. Evaluates the success of the system in meeting the aims of the study. The chapter ends with recommendations for further research.

CHAPTER 2

LITERATURE REVIEW

2.1. Overview

The digital transformation of the food and beverage industry has reshaped the way restaurants interact with customers, run their operations, and service their customers. Restaurants used to rely on face-to-face interactions, manual order taking, and neighborhood-based marketing to run their businesses. Due to advanced technology and the internet, along with shifting customer needs, restaurants had to develop sophisticated web-based platforms. This development was accelerated by global events, such as the COVID-19 Pandemic, that required restaurants to shift to customers ordering and paying for food online and offering a contactless delivery service.

The efficacy of restaurant web platforms is evaluated through various academic lenses. The Technology Acceptance Model (TAM) is heavily cited and is combined with established User Experience (UX) principles. In TAM, if a new technology eases the user's interaction and a technology serves the user's goals, then the user is inclined to use that technology. In the context of Food & Beverage, simplifying the serving of an order (menu browsing and adding order modifications) while also reducing the order transaction time by providing real time order status and offering deals through the web platform would be considered a user goal and technology easement. Analyzing previous and existing systems is essential for developing new systems. Thus, the evaluation of existing systems is a strong foundational design steps for new systems and solutions Modeling new solutions starts with evaluation of existing systems. Examining KFC and Haidilao offers opportunities to analyze strategies at opposite ends of the digital spectrum. KFC, with its highvolume, quick service operations, emphasizes speed over everything else, including transactions and up selling. Haidilao, meanwhile, operates in the premium casual dining segment. In this segment, digital operations focus primarily on the advanced management of reservations, customer loyalty, and efforts to create and sustain the exceptional level of service they are famous for, in the digital space. Dissecting the user interfaces, information architectures, and technologies of these systems will show developers the current state of the industry, areas that remain non-served, and provide a basis for building advanced, fully integrated restaurant applications.

This chapter reviews two related but distinct areas: customer-facing restaurant websites and the operational information systems used inside restaurants. The KFC Vietnam and Haidilao Vietnam websites are examined as publicly accessible customer interfaces. Their visible pages can support observations about navigation, menu presentation, customization, reservations, and ordering where these functions are available; they do not, by themselves, establish which backend, point-of-sale (POS), inventory, payment-processing, or kitchen-management technologies the companies use. Accordingly, the website analysis is a usability-oriented review rather than a technical audit of either company's internal systems.

The chapter also considers restaurant management and POS systems through academic literature, then compares the relevant technologies for the proposed River Side Restaurant Management System. This distinction avoids treating consumer websites as equivalent to integrated restaurantmanagement platforms. The Technology Acceptance Model (TAM) is included as a theoretical lens, with its original source cited. TAM proposes that perceived usefulness and perceived ease of use are important determinants of user acceptance; it does not guarantee adoption merely because an interface is easy to use [31].

2.2. KFC web platform [29]

Kentucky Fried Chicken or KFC has one of the most powerful digital platforms among quick service restaurants across the globe. KFC's digital strategy is designed to increase transactions, improve visual engagement, and maximize the average order value through strategic cross-selling. Analyzing the KFC website unveils a highly optimized user experience journey to reduce friction from the landing page to the final page checkout.

2.2.1. Interface design and navigation

The KFC website uses large product images and a red/white color scheme to establish an appealing visual hierarchy. Navigation is typically a sticky header or prominent side navigation where customers can find the most important categories of the site like Combos, Sharing, or Sides. KFC's design minimizes the need for customers to click to access features. This design choice is in compliance with UX heuristics to reduce the time and number of clicks it takes to access features.

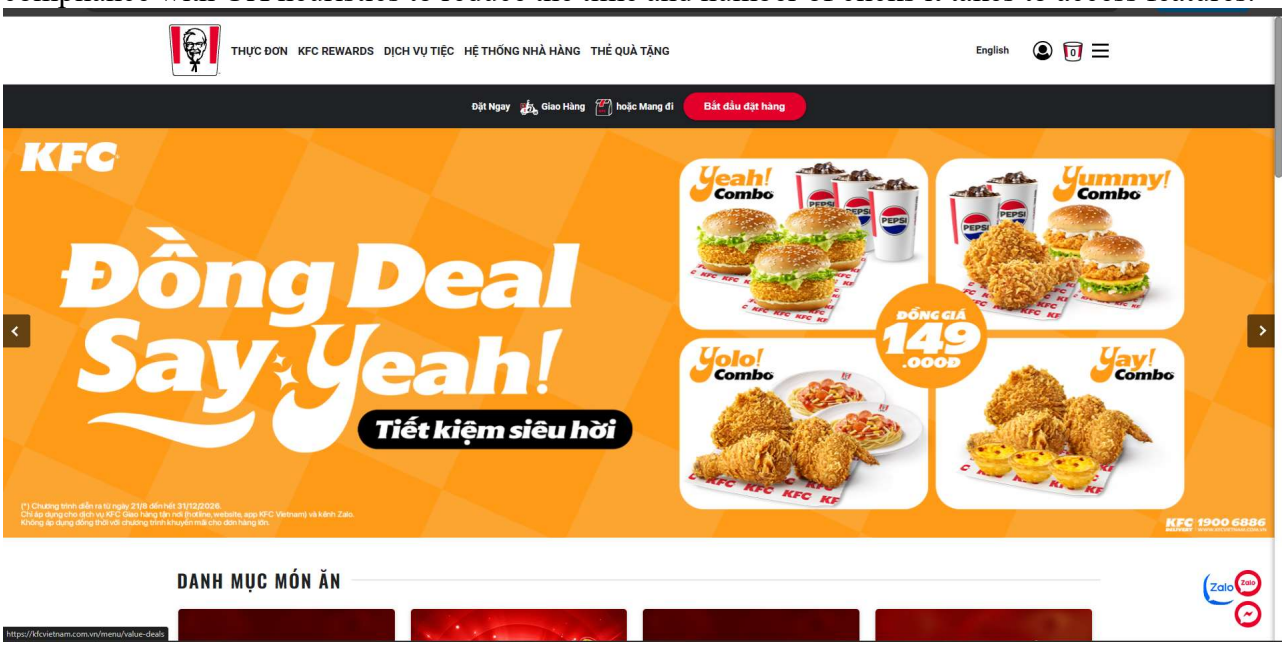


Figure 2.2 KFC Homepage[29]

From the standard point of usability, the platform often makes use of dynamic loading for displaying menus as it does not prompt the user to refresh the page completely. This technique helps in engaging users and minimizing their waiting duration. However, assessments of QR platforms may point out problems of visual disorder resulting from repeated advertising messages and pushy up selling tactics, although such critiques may be temporarily confusing for consumers.

2.2.2. Order customization and state management

A valuable feature of the KFC platform is its product customization interface. In many situations, the user is obliged to change the standard meal set, choose the drink size or add extra items. The interface is working through the pop-up windows or dedicated customization pages which adapt the pricing dynamically according to the user's selections.

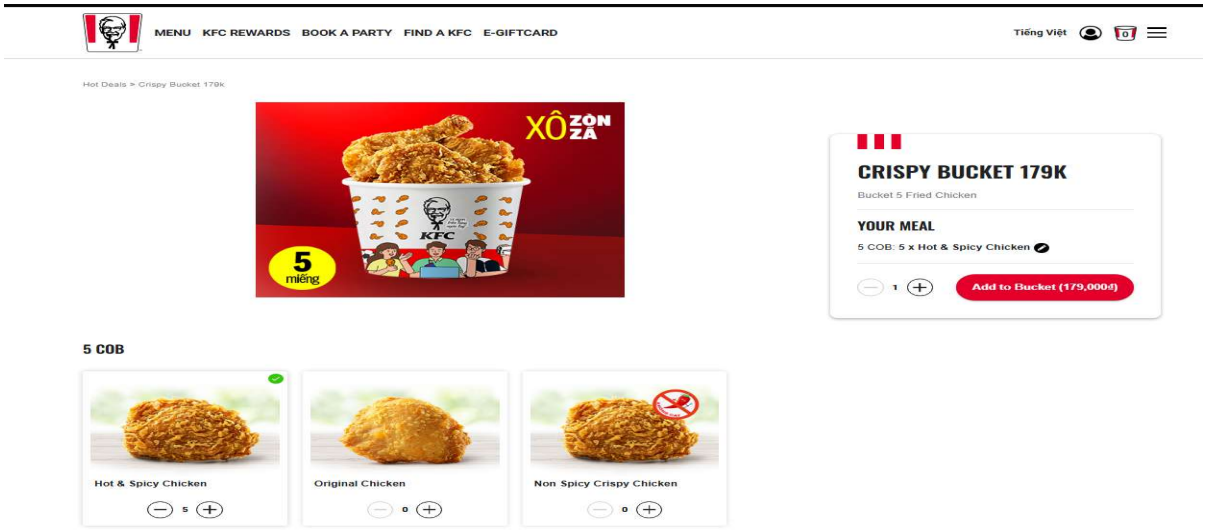


Figure 2.3 KFC Order customization [29]

This feature requires complicated state management on the front-end to make sure that the cart corresponds to the user's real-time decisions. The cart is usually displayed in a form of a sliding drawer or a floating icon, which gives the user the opportunity to see the total price and the items selected at any time. Keeping the cart/screen always accessible, the service decreases the cart abandonment rates, which is a key success factor for e-commerce.

2.2.3. Checkout flow and conversion

The design of the checkout process on the KFC platform facilitates conversions effectively and quickly. The customers can opt for guest checkout or login as registered users. The system has the advantage of using past experience, such as the previously used delivery addresses or payment methods for registered customers to achieve seamless transactions.

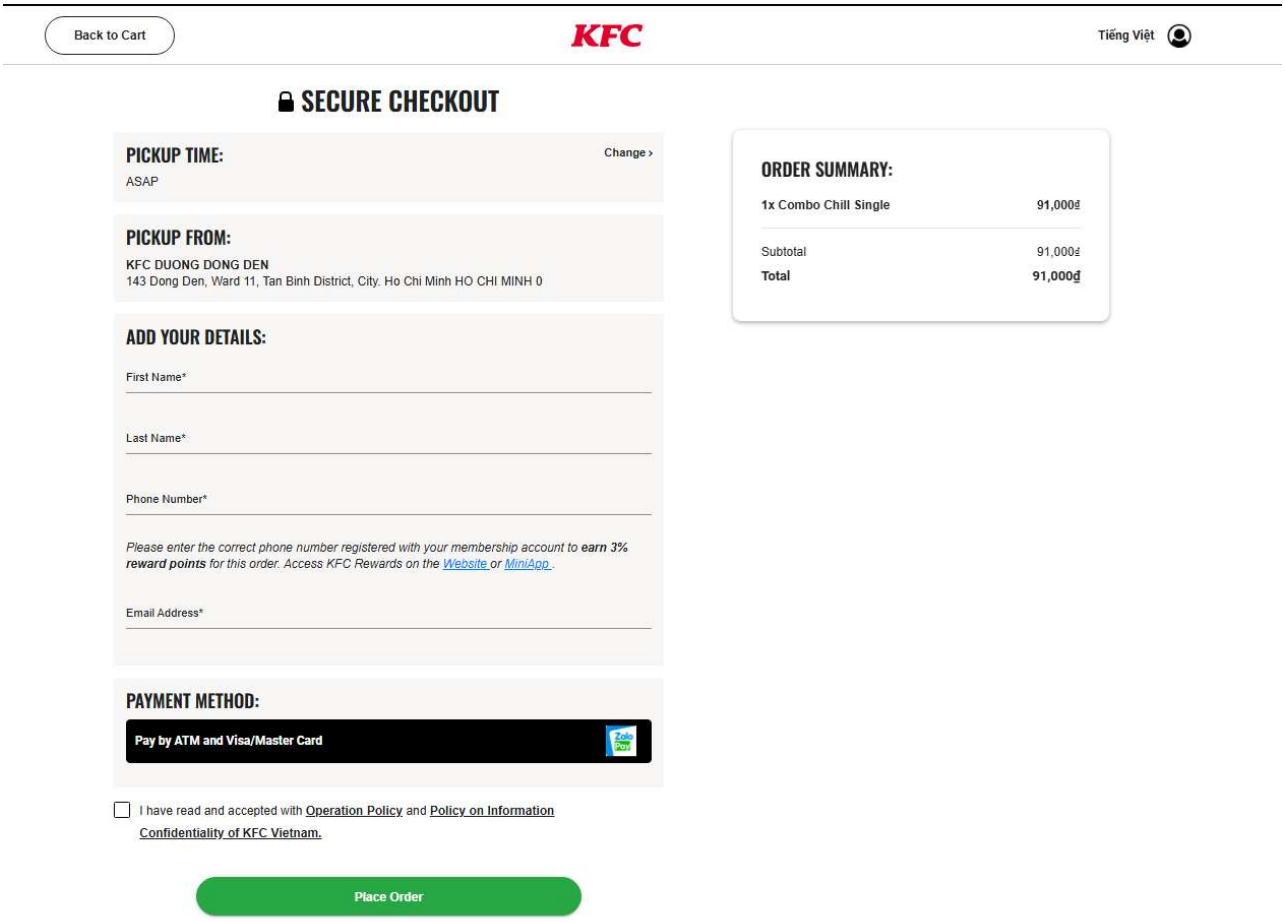


Figure 2.4 KFC Checkout flow [29]

The presence of numerous options for payments (credit cards, e-wallets, or cash on delivery) guarantees accessibility. From the academic view, simplified flow is in line with conversion rate optimization, which stipulates that reducing the number of mandatory fields in forms and ensuring the transparency of pricing (including tax and delivery fees in the final price) directly influence the success of the transaction.

2.3. Haidilao website platform [30]

Haidilao takes a totally different outlook from that of fast-food chains in regard to their online presence. Haidilao has become one of the world leaders due to its passionate customer service. Their digital platforms serve the goal of merging the principle of the digital convenience with that of premium hospitality. Among the most essential distinguishing elements of the platform are reservation management, customer loyalty system implementation, and pre-dining engagement which the brand puts ahead of its speed of transactions.

2.3.1. Brand translation and user engagement

The interface of Haidilao is much more simplistic and cleaner than that of industry players in the fast-food business. Its design is centered around friendly atmosphere and the idea of dining. Instead of making customers order now, the service offers them join the queue or book a table with just a few clicks.

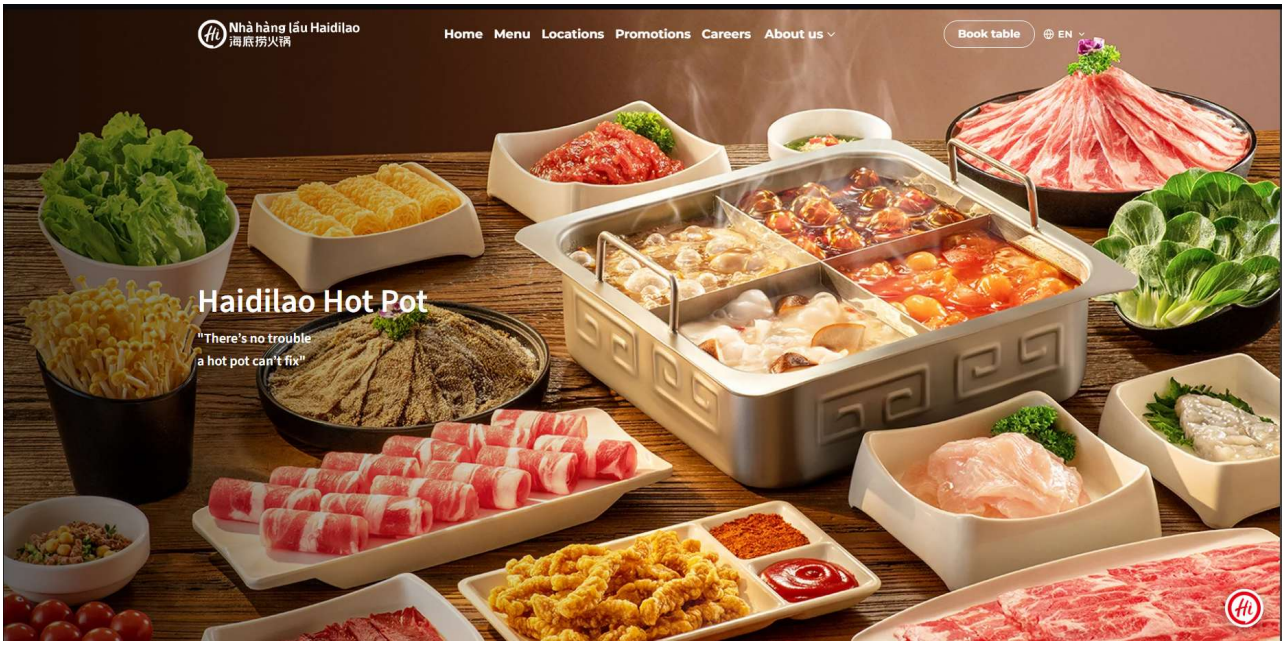


Figure 2.5 Haidilao homepage [30]

Such structural choice serves the point that the company is a service-oriented business. By enabling the customers to join the queue and minimizing the unhappy experience of waiting which is pivotal in restaurants, the system requires synchronizing data in real-time.

2.3.2. Menu categorization and Cross-Brand expansion

Unlike providing a working menu on wheels for instant consumption, the Haidilao system views its offering as an all-inclusive digital menu. Practically, the interface sorts items into different categories: Soup Bases, Main Dishes, Dipping Sauces, Appetizers, and Beverages. Thus, the consumers get familiar with the complexity of hotpot dining before coming to the restaurant.

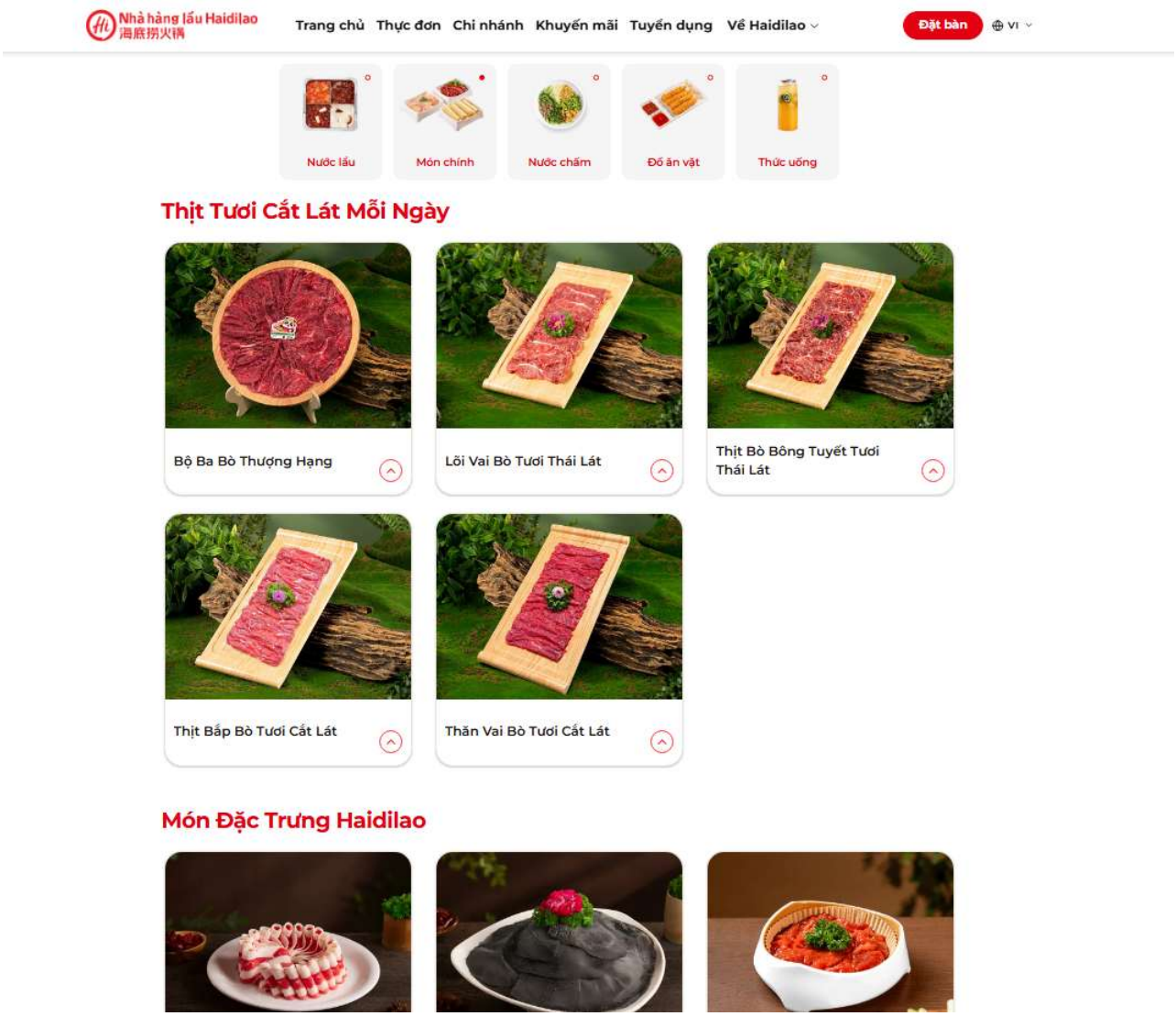


Figure 2.6 Haidilao menu category [30]

To counteract this, the platform has developed an information architecture that is highly methodical. The user interface divides the various items into stages within the meal, arranging them into five themes: Soup Bases, Main Courses, Dipping Sauces, Appetizers, and Drinks. Effectively, this approach mimics the way the dining experience takes place over time thus ensuring that the consumers understand the food preparation steps.

By presenting a large variety of food products in the way that has been mentioned above, the platform is in compliance with the principles of usability. For each category, the platform uses appropriate images and typography. From the academic view, the platform is also in line with the UX principle of progressive disclosure, thus ensuring that it provides only necessary information when the user opens the relevant category.

2.3.3. Off-Premise Dining and Delivery Operations

While bookings are the foremost part of the user journey, a large part of the company's efforts goes into bringing its "Bring Haidilao Home" service to its customers. Delivering hotpot has its own unique challenges, different from regular fast food delivery; hotpot usually includes the transportation of raw materials, special broths, and even special cooking equipment.

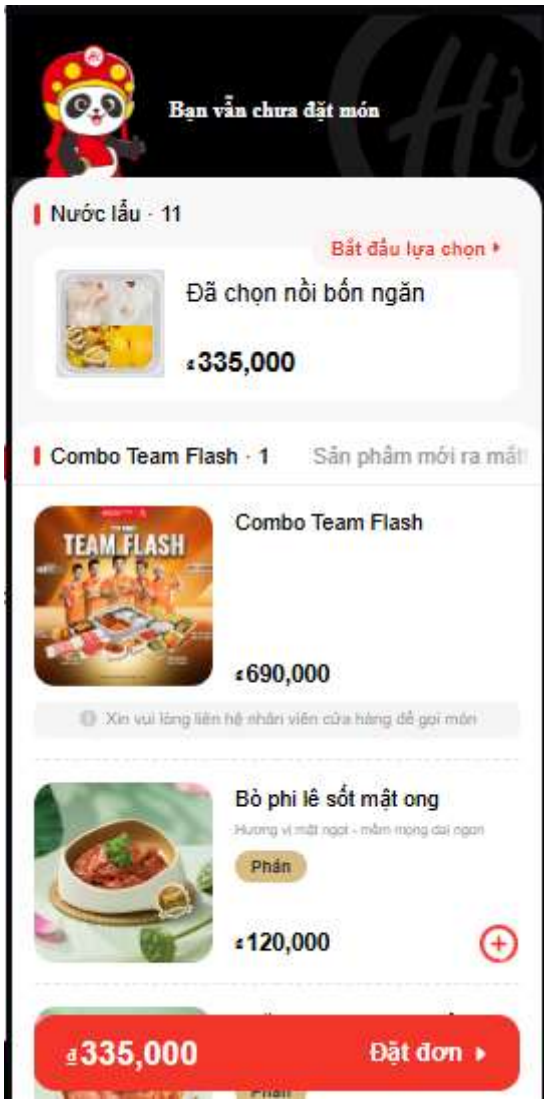


Figure 2.7 Bring Haidilao Home Feature [30]

The delivery interface has to cover these specific needs of the hotpot delivery business. The users can make orders of large meals combining various items for home meals. The technology that backs this system has to calculate many things we can't see in regular takeout available on the market.

2.4. Technologies Analysis

The creation of a web platform for a modern restaurant entails careful choice of technologies throughout the software stack. The main engineering goals of the Food & Beverage (F&B) management system are as follows: fast interactive performance from the client side, transaction protection and concurrence from the server side, and relational database guarantees from the data side. This section is dedicated to the architecture's technologies, giving examples of their comparison to their main alternatives and justifying them according to the requirements of the restaurant management system, which include real-time updates of Kitchen Display System(KDS), multi-role access, and fast order processing during peak hours while ensuring all the transactions are properly executed.

2.4.1. Client-Side Architecture and Front-end Frameworks

2.4.1.1. Single-Page Applications

The user interface layer belonging to web applications has undergone a dramatic change over the past ten years. In the past web applications were based on multi-page HTML documents which had to be reloaded every time a user performed any action. However, the majority of applications are nowadays created using Single-Page Applications (SPAs) principles. The exchange of information with the server in SPAs has become much easier as well. Only one HTML document is loaded as a main application shell, which is then modified by the application scripts with every user action. Any changes to the Document Object Model (DOM) [1] require modification of its source code, which allows for partial updates of a page content without reloading assets. This way of using the application makes it possible for the user to work with the application just like with the desktop application, since there is no need to wait for the entire HTML document to load again. This is important, particularly in the case of kitchens of restaurants and service counters where personnel interact with the application for a significant part of their working hours.

2.4.1.2. React.js: Component Model and Virtual DOM

The system's front-end aspect has been achieved using React.js, a free library for JavaScript involved in creating graphical interfaces initially created and supported by Meta (Facebook)[2]. React implements a declarative, component-based approach to build UIs, which are broken down into smaller, self-contained, and reusable components. Each component manages its rendering logic, state, and life cycle. For example, a single OrderCard component can be used dozens of times on different pages (Kitchen Display boards, waiter dashboards, manager's order tables), exposing different data to each via React's props mechanism.

However, React's main feature is the virtual DOM, which is a virtual tree structure of the HTML document, which serves as a basis for calculating the most efficient way of updating the browser's real DOM. Instead of updating or changing real DOM elements (which is an expensive operation in terms of CPU and memory consumption), React batches all changes on the virtual DOM and later applies them to the real one once the application state has changed. For example, when a chef marks an order as "Ready", React will update the corresponding OrderCard component in the application state, compare the new Virtual DOM tree with the previous version (calculating the difference in O(n) time complexity via heuristic line number comparison, rather than performing a full $O(n^3)$ tree diff [3]), and apply the changes to the browser's DOM so that the Kitchen Display board updates. In the context of a restaurant, this becomes particularly important since dozens of OrderCards may be updated at once during peak hours without causing UI freezes or delays.

Finally, React 19 features the new React Compiler [4], a tool which automatically analyzes component trees and eliminates redundant renders by memorizing the results of component computations. This allows the manager's dashboard to render its charts and tables, form elements, and modals without additional re-renders caused by changes in parent components.

2.4.1.3. Comparative Analysis: React vs. Vue.js vs. Angular

Three frameworks are dominant in the modern SPA world: React, Vue.js., and Angular. A comparison is needed to justify the choice of React for this system.

- **Vue.js**.[5] (maintained by Evan You and the open-source community)

  Provides an advanced technique with simple template language and reactive model of user interface illustrated by Java Script Proxy [5]. Its Options API offers structured and easy-to-learn component model while new Composition API introduces flexibility similar to that

found in React. Vue is great for fast prototyping and small development teams. At the same time, the Vue ecosystem for enterprise applications is not very rich in terms of server-state management tools, the ability to render components in parallel, or code analysis. The popular TanStack Query library for server-state caching and synchronization, which this system heavily relies on, was originally designed as React Query – and while it supports multiple frameworks, it has the most features and community support for React [6].

- **Angular** [7] (maintained by Google)

  Is a comprehensive framework that tries to create everything available by default including routing, forms, HTTP-client, etc. It utilizes strict methodology (TypeScript) and architecture MVC (Model-View-Controller). There is less boilerplate code and tooling support, but such a framework can be less convenient for rapid development and simple applications.

- **Justification for selecting React**

  This thesis needs (a) to build a component architecture that would reuse UI logic for 5 different roles (Customer, Chef, Waiter, Delevery personnel , Manager) [1], (b) render KDS boards with real-time information about the orders, (c) leverage TanStack React Query for server-state synchronization and caching [6], and (d) have access to a mature ecosystem of external libraries (e.g., Recharts for data visualization, Lucide for icons, React Router for routing). The React framework is the only one that meets all four criteria while providing the best performance footprint.

2.4.1.4. Styling Architecture: Tailwind CSS

The visual styling of the application is implemented using **Tailwind CSS** (version 4), a utility-first CSS framework that provides low-level, composable utility classes applied directly in component markup [8]. Unlike traditional CSS methodologies (BEM, SMACSS) or component-scoped approaches (CSS Modules, Styled Components), Tailwind eliminates the need for custom class naming conventions and separate stylesheet files. Each utility class maps to a single CSS property-value pair (e.g., p-4 produces padding: 1rem), enabling developers to construct responsive, consistent layouts without context-switching between markup and stylesheet files.

Tailwind CSS v4 introduces a Rust-based compilation engine, **Oxide**, which delivers build performance improvements of up to 5x compared to version 3, with full CSS builds completing in under 5 milliseconds for projects of moderate scale [9]. For this restaurant system, which must render consistently across desktop management terminals, tablet-mounted KDS units, and mobile customer devices, Tailwind's responsive break point utilities (sm:, md:, lg:, xl:) enable granular, device-adaptive layouts within a single component definition, significantly reducing the maintenance burden of multi-device support.

2.4.2. Server-Side Logic and API Implementation

2.4.2.1. Requirements for Restaurant Back-end Systems

The server tier of a restaurant management platform must satisfy several concurrent engineering constraints: (a) processing high-throughput API requests during peak dining hours (lunch and dinner services), where dozens of simultaneous order submissions, payment confirmations, and status updates converge; (b) enforcing complex, multi-step business logic such as order state machines, automated staff dispatching algorithms, and coupon validation workflows; (c) maintaining strict authentication and role-based authorization across four actor categories (Customer, Chef, Waiter, Manager); and (d) exposing a well-documented, type-safe API contract for the SPA front-end.

2.4.2.2. FastAPI: Asynchronous Python Framework

The back-end of this system is implemented using FastAPI – a modern framework for building RESTful APIs [10]. It is based on Starlette (ASGI), providing HTTP server capabilities and WebSocket support, and Pydantic, a library for declarative data classes using Python type annotations [11].

The framework's first distinguishing feature is native support for Python's async/await keywords through its ASGI compatibility. Unlike the traditional WSGI (Web Server Gateway Interface) specification, which defines a synchronization API for communication between web servers and Python web applications, ASGI (Asynchronous Server Gateway Interface) [12] standard defines how a server can communicate with asynchronous applications. When a WSGI application receives an HTTP request, it creates a thread or process for it and waits until the response ends. However, an ASGI server can process thousands of simultaneous connections on a single CPU core by designating specific threads for I/O (input/output) operations, such as querying a database or reading a file. In a restaurant example, if a hundred clients make requests to the server at once, some of them will ask for the menu, some will send payment confirmations, and some will check their order status: in this case, a synchronous server will have to process these requests one after another, while an asynchronous server can switch to another request after the I/O command (e.g., query to the database) and thus increase performance.

The second noteworthy feature is that FastAPI provides auto-generated API documentation. This functionality is based on Python type hints and Pydantic's schema inference, and it allows developers to view their API's request/response schemas at the /docs and /redoc endpoints [10]. For the front-end developers, this means a reduced amount of necessary API-related documentation code and a living example of all available API requests.

Finally, a request validation mechanism is implemented in FastAPI that utilizes Pydantic's validation rules to check the correctness of data sent in the request body. This validation is powered by pydantic-core, a Rust implementation of Pydantic's core [13] that is used in FastAPI. According to the official benchmarks, Pydantic v2 is up to 17x faster than v1 in practical tests on validating deeply nested objects [13]. Thus, when a client sends an HTTP request to the server, its body will be validated as per the DTO classes much faster, which is crucial under high load.

2.4.2.3. Comparative Analysis: FastAPI vs. Express.js vs. Django

- **Express.js (Node.js ecosystem)[14]:**

  The advantages of Node.js event loop and non-blocking I/O approach are prominent as it has many JavaScript developers available. It does not come with the in-built input validation, auto-generation of API documentation, or dependency injection. For each one of these features, external libraries must be used (Joi/Zod, Swagger-jsdoc, etc.). JavaScript's dynamic typing is also a disadvantage since it does not guarantee compile-time safety as seen with Python type annotations as explained by FastAPI/Pydantic.

- **Django (Python ecosystem)[15]:**

  A fully-fledged framework that comes bundled with features for building applications and follows Model-View-Template architecture. Provides integrated ORM, admin interface, authentication system, and form handling. Its WSGI-based synchronous architecture limits concurrency capacity at scale, even with Django 4.1's limited async view support [16] ; ORM layer still requires synchronous database access (sync_to_async wrappers needed). Incorporates irrelevant components such as template rendering and session management, which are unnecessary for a headless API-only application.

- **Justification for selecting FastAPI:**

  The restaurant system will require (a) asynchronous request processing for order/payment/status activities, (b) Pydantic-based DTO validation for rigid input requirements across 14 API routing modules, (c) the included OpenAPI documentation for front-end collaboration, and (d) cryptography and JWT support for secure authentication purposes. FastAPI is the only framework that satisfies all four requests while delivering a better capacity than Django and type-safe than Express.js.

2.4.2.4. RESTful API Design and JWT Authentication

The React single-page application communicates with the FastAPI server through a REST-oriented API [18]. Each resource has a URI, and clients access it with HTTP methods such as GET, POST, PUT/PATCH, or DELETE. This arrangement keeps frontend and backend responsibilities separate, but a change to the API contract can still require updates to both applications.

The system authenticates users via JSON web tokens (JWT) [19]. These allow secure authentication by signing the session data and passing it along. They're represented in compact format suitable for being passed along via an Authorization header. Each contains several "claims" which encode things about the user and their current session that would normally need to be fetched from a database – they are encoded but not encrypted so sensitive info shouldn't be included.

When signing the token using HS256, this means we'll use the HMAC algorithm with SHA-256 and share our secret key. It's important to note that while HS256 will authenticate the contents and ensure nothing has changed since creation, it doesn't provide actual encryption nor hide the contents. This also means that when the server validates a request with a jwt, it needs to verify both that the signature checks out, as well as some subset of the claims – at minimum expiration and issuer/audience if those fields were included.

Passwords are stored using bcrypt [20], a password-hashing function designed to make offline guessing more costly. Bcrypt is not an encryption method: password hashes are not decrypted during login. Instead, the submitted password is verified against the stored hash using the algorithm's verification process. The work factor should be selected and tested for the deployment environment and updated as hardware capabilities change.

2.4.3. Database Systems and Relational Integrity

2.4.3.1. Data Integrity Requirements in F&B Systems

The information system's persistent data layer has to ensure that every transaction is processed correctly. In case of an error that occurs anywhere during the processing of the transaction, the whole operation should be rolled back. Taking a food and beverage system as an example, when a customer places an order, one has to perform several transactions: update the records of the order, order items, payments, tables, and staff. Therefore, the database should have features that allow reliable concurrency control and ACID compliance.

2.4.3.2. PostgreSQL: Object-Relational Database System

PostgreSQL 16 [21],[22] Relational Database Management System was chosen as a part of the proposal system. RDBMS has features like relational constraints, transaction, indexes, Full Text Search (FTS), and JSON / JSONB data types support.

PostgreSQL provides ACID features (Atomicity, Consistency, Isolation, and Durability):

- ACID property represents the guaranteed behaviors while performing transactions on DB systems. These properties' actual behaviors depend on DB configurations and used Isolation levels. Atomicity: When processing the whole transaction, either all the changes will be committed or all will be rolled back, depending on whether the transaction succeeds or fails.
- Consistency: The transactions should maintain the database's predefined rules which may include Primary key, Foreign Key, Unique, Check & Application Enforced Constraint. However, there is no assurance from DB side about fulfilling all business rules except representing them with relevant constraints or using logic inside the transactions.
- Isolation : PostgreSQL uses multi-version concurrency control (MVCC) to coordinate concurrent transactions. Its default isolation level is Read Committed, where each statement sees data committed before that statement began. PostgreSQL also supports Repeatable Read and Serializable isolation. The required level should be chosen according to the consistency needs of each operation; no isolation level should be described as universally stronger with out specifying the comparison and behavior being discussed [22].
- Durability : once a transaction is committed successfully, PostgreSQL's write-ahead logging (WAL) and recovery mechanisms help preserve committed changes against crashes, subject to the configured durability and storage environment.

PostgreSQL ENUM types can restrict a column to a predefined set of labels, which can be useful for stable categories such as order or payment status. They should be used only where the set of values is appropriately controlled; frequently changing categories may be easier to manage in a reference table.

2.4.3.3. Comparative Analysis: PostgreSQL vs. MySQL vs. MongoDB

MySQL [24] is a relational database management system whose InnoDB storage engine supports transactions and foreign-key constraints. MySQL 8.0 supports window functions and common table expressions, including recursive CTEs; therefore, these features must not be listed as unsupported. MySQL does not provide PostgreSQL's JSONB data type, although it supports a native JSON type and offers indexing options for selected JSON expressions and values. The databases differ in implementation details and feature sets, so the comparison should focus on the requirements of this project rather than claim that MySQL lacks modern SQL functionality [24].

The default transaction isolation levels differ: PostgreSQL defaults to Read Committed, whereas InnoDB in MySQL defaults to Repeatable Read. These defaults do not justify claiming that PostgreSQL provides a higher isolation level by default. PostgreSQL Read Committed gives each statement a snapshot as of that statement's start, while PostgreSQL Repeatable Read provides a transaction-level snapshot. MySQL/InnoDB Repeatable Read has its own consistent-read and locking-read behavior. Isolation semantics and concurrency outcomes should be compared precisely, and application transactions should be designed to prevent conflicts such as double-booking a table or processing an order twice [22], [24].

MongoDB [25] is a document-oriented database that supports multi-document ACID

transactions. Multi-document transactions have been supported since MongoDB 4.0 for replica sets, with support for sharded clusters added in MongoDB 4.2. Thus, it is incorrect to state that MongoDB lacks multi-document ACID transactions or cannot maintain consistency for multi-document operations [26]. MongoDB's document model may suit data that is naturally represented as nested documents, whilerelational databases can be a natural fit for a system with numerous entities, foreign-key relationships, joins, and referential-integrity requirements.

For this project, PostgreSQL is selected because the data model contains interrelated entities such as customers, staff, reservations, tables, menu items, orders, order items, and payments. Relational constraints and SQL transactions support the required relationships and multi-entity operations. This is a project-specific design justification, not a claim that MySQL or MongoDB are incapable of transactional processing. The final schema and relationship counts must match the implemented database and ER diagram. For the current thesis draft, use 16 tables and 22 relationships if those are the verified final counts.

2.4.3.4. Restaurant POS and Management systems

A restaurant point-of-sale (POS) system records sales transactions and commonly supports order entry, bill calculation, payment recording, and sales reporting. Depending on the product and deployment, it may integrate with menu management, inventory, staff workflows, customer records, and a kitchen display system (KDS). A full restaurant management system (RMS) may cover a broader set of back-office and front-of-house functions than a POS alone; these terms should not be treated as interchangeable in every context.

Research on restaurant technology has considered both POS use and the adoption of broader restaurant management systems. Mandabach et al. examined restaurant managers' perceptions of POS technology, including its effects on service transactions and work routines [33]. A more recent study of restaurant management-system adoption in Vietnam applies TAM alongside the Theory of Planned Behavior and Innovation Diffusion Theory, emphasizing that adoption is shaped by multiple user and organizational factors rather than interface usability alone [34]. These studies complement the website review by addressing staff-facing operations and technology adoption.

For the proposed system, the operational requirements include role-based access for customers and staff, order and order-item management, payment records, table status, reservation handling, and coordination of order status with kitchen and service workflows. These requirements are evaluated at the system-design level; they should not be attributed to KFC or Haidilao unless reliable evidence documents that the companies' internal POS or RMS implementations use the same features or technologies.

2.4.4. Full-Stack Integration and Justification

2.4.4.1. Stack Synergy: React + FastAPI + PostgreSQL

The combination of the technologies employed (React front end, FastAPI back end, PostgreSQL database) is an optimal choice that provides reliability and performance required for the system under consideration.

Concurrency support: the combination of React's Virtual DOM batching, FastAPI's asynchronicity and PostgreSQL's MVCC provide the system with a full flow of the processing performed concurrently. For example, if the status of some kitchen was changed by the chef and the updated state of that kitchen needs to be transferred to the client (using Virtual DOM conversion), the server (using patch method) and the database (instant update using MVCC). Concurrent processing of other

participants involved in the process, be it waiters, customers or managers, will not interfere with this flow.

Type-safe data processing: there are pydantic DTO schemas on the edge of the API that enforce DDD rules of data structure not only for the API input parameters, but for the database model as well. As a consequence, all the errors that typically occur in the runtime environment which lacks reasonable schemas are not present in the stack (express.js + MongoDB is an example of such system).

2.4.4.2. Alternative Stack Comparison

Criterion	React + FastAPI + PostgreSQL	Vue.js + Express.js + MongoDB	Angular + Django + MySQL
Concurrency model	Async ASGI event loop	Node.js event loop	Synchronous WSGI workers
Input validation	Pydantic (automatic, typed)	Manual (Joi/Zod)	Django Forms (template-oriented)
API documentation	Auto-generated OpenAPI	Manual Swagger setup	Django REST Swagger plugin
Transaction integrity	Native ACID + MVCC	Limited multi-document ACID	ACID (weaker isolation defaults)
Relational modeling	23 FK relationships native	Application-level refs	Native but fewer SQL features
Bundle size (client)	~42 KB gzipped	~33 KB gzipped	~143 KB gzipped
Real-time KDS suitability	Virtual DOM batching	Proxy reactivity	Zone.js change detection

Table 2.4.4.2 Alternative Stack Comparison

The comparison table should avoid categorical claims such as “PostgreSQL has stronger default isolation than MySQL,” “MongoDB has limited multi-document ACID,” or “React is the only framework that meets the criteria.” MySQL 8.0 supports window functions and recursive CTEs, MongoDB supports multi-document transactions, and React, Vue, Angular, FastAPI, Express, Django, PostgreSQL, MySQL, and MongoDB can all be used to build production systems with different design trade-offs.

A defensible comparison should state the criteria that matter to this project: alignment with the existing codebase and developer skills, relational modeling and referential integrity, transaction behavior, validation and API documentation, ecosystem support, deployment requirements, maintainability, and testing results. Performance and bundle-size figures should be included only when measured under a stated methodology or supported by a reliable, directly relevant source. The selection of React, FastAPI, and PostgreSQL is therefore justified by fit to the project’s requirements, not by claiming that alternative stacks are categorically incapable or inherently inferior.

2.5. Technique Review

The literature relevant to this project falls into three areas: customer-facing restaurant digital services, staff-facing restaurant POS/management systems, and general software-engineering techniques. The KFC and Haidilao website review provides examples of visible customer journeys, but it does not replace research on operational systems. Research on POS systems examines how transaction technology affects service workflows and staff routines [33], while restaurant management-system adoption research investigates user perceptions and organizational influences [34].

TAM was introduced by Davis to explain technology acceptance through perceived usefulness and perceived ease of use [31]. In this project, these constructs can guide the discussion of whether customers and staff perceive the proposed system as useful and easy to operate. However, merely mentioning TAM does not constitute an empirical application of the model. If the thesis does not collect user data or test TAM hypotheses, it should be presented as a conceptual framework informing usability considerations rather than as a validated evaluation of user acceptance.

The remaining technique review discusses SPA architecture, component-based UI rendering, utility-first CSS, server-state management, asynchronous APIs, stateless authentication, relational transactions, and ORM-based database access. These are design concepts used to explain the proposed implementation; claims about their performance or security should be limited to what is supported by documentation, testing, or cited research.

- **Single-Page Application (SPA) Architecture**

This design uses the Single Page Application principle, according to which the browser loads a single HTML file and dynamic updates its content based on user activities without submitting server requests for a new page [1]. Thus, repeated loading of the same resources is avoided, which is of utmost importance in the case of operational interfaces, including kitchen display boards as well as real-time order tracking dashboards that require constant visual exchanges with satisfied low latency.

- **Component-Based UI Rendering**

ReactJS is a declarative JavaScript library utilized to build the front end. It breaks down user interfaces into independent and reusable component trees [2]. React's virtual DOM reconciliation algorithm finds out whether any real changes are required in the DOM tree; therefore, rendering efficiency remains optimal even under a heavy flow of data typical to active order management processes. Recent enhancements in React's pipeline of compiling and concurrent rendering boost the efficiency of re-rendering complex interactivity scenes.

- **Utility-First CSS Approach**

Tailwind CSS is a utility-first CSS framework that provides basic styles to components at a markup level in order to manage visual styling on various UI components [8]. The release of version 4.0 of the framework introduced a Rust-based compiler exclusively generating the CSS codes appearing in the codebase, making it possible to generate the most minimal production stylesheets [9]. In the process, visual consistency and responsiveness are secured through adherence to various screen sizes, ranging from small mobile devices to wide displays, while getting rid of any conflicts related to specificity as it takes place with traditional component-oriented CSS designs.

- **Declarative Server-State Management.**

Such processes as fetching data in an asynchronous way, caching client information, background data, and tracking the lifecycle of data changes take place using TanStack React Query [6]. This

refers to the fact that the operations related to coordinating HTTP requests are fully wrapped into declarative hooks (useQuery, useMutation), which let the cache be automatically invalidated and the information about out-of-date data be current without any need to check it manually.

- **Asynchronous Server Framework and API Design**

The backend application layer utilizes FastAPI, which is a new Python web framework based on the Asynchronous Server Gateway Interface (ASGI) specification and Python's native async/await coroutine model in order to process requests without blocking. Validation of requests and responses is performed using Pydantic schemas that impose strict types before they reach the API [11]. The API itself follows the REST architectural style proposed by Fielding [18], organizing resources within a uniform URI hierarchy and using standard HTTP methods (GET, POST, etc.) to access resources.

- **Stateless Authentication and Credential Security**

The process of user verification is carried out in compliance with the JSON Web Token (JWT) standard described in RFC 7519 [19]. Tokens include claims about the user's role (sub, role) and can be used without server-side session storage. Password storage is based on the bcrypt algorithm developed by Provos and Mazières [20].

- **Relational Database and ACID Compliance**

The persistence aspect relies on PostgreSQL system. The latter is an open-source object-relational database management system that is famous for the extensiveness of capabilities, standards-compliance, and the provision of strong transaction guarantees [21]. Multi-Version Concurrency Control (MVCC) employed by PostgreSQL [23] provides for concurrent access for simultaneous reading and writing to the shared tables. In this way, customers may read the menu items while the kitchen staff may alter the status of the orders. The use of the full-fledged ACID transaction semantics guarantees the atomicity of operations performed in several tables, such as the creation of an order, adjustment of the inventory, or settlement of payment.

- **Object-Relational Mapping (ORM) and Injection Prevention**

The database is accessed through the SQLAlchemy framework which is an advanced toolkit for ORM. It translates Python classes into relational schemas and creates the SQL commands that exclusively use the parameterized approach [27]. It solves the problem of SQL injection defined by the OWASP Foundation [28] due to the elimination of string concatenation from the SQL command building process, thus achieving improvement of transaction management and maintenance of relations between various models.

CHAPTER 3

METHODOLOGY

3.1. Overview

This chapter is expected to provide an overall view of the implementation steps to develop a comprehensive restaurant operation system. It begins with giving a brief introduction to thesis's technology stack involved in development stages, along with the justification for each selected technology and how the system architecture follows the MVC (Model-View-Controller) pattern. Analyzes the user requirements to identify actors and define roles with their own set of permission and functionalities within the system. Functional requirements are organized into ten distinct modules covering authentication, menu management, shopping cart, table management, order processing, delivery, payment, staff management, scheduling, Manager dashboard analytics. Non-functional requirements addressing security, performance, data integrity, and maintainability are specified. Furthermore, presents use case diagrams with use case description for each scenario, the core business workflow for order life cycle, the data entity model summarizing database entities and their relationships, API endpoint directory, and the front-end page to route mapping for entire application.

3.2. System Overview

3.2.1. Introduction

The Restaurant Management System is a complete and advanced website-oriented application that is designed to simplify and automate the restaurant day-to-day processes. The system offers different functionalities for five types of clients: Customer, Waiter, Chef, Delivery personnel, and Administrator to enable the different client roles to manage the whole flow from selecting the food and making an order to cooking and delivering the food to the client.

3.2.2. Overall System Architecture

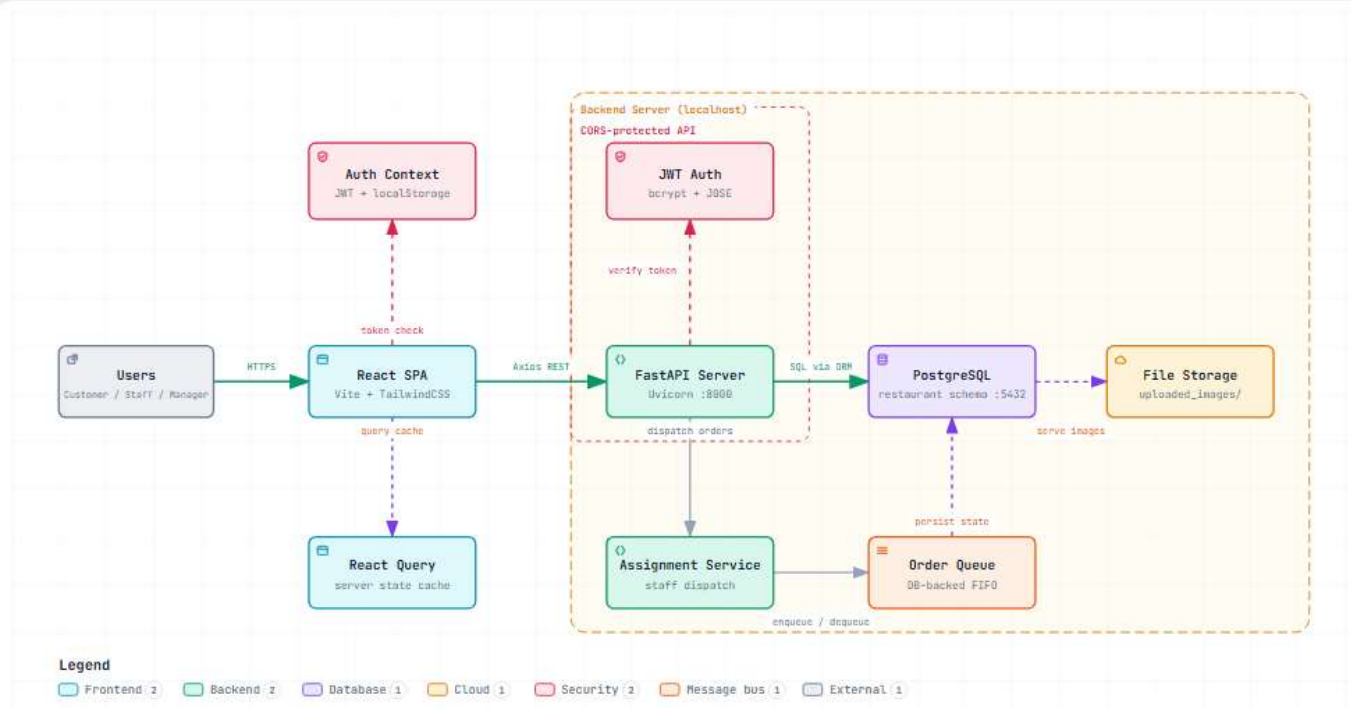


Figure 3.2: Restaurant Management System Architecture

The Restaurant Management System architecture uses a multi-layered system that allows development and operations to be separate across different areas, like the front-end, back-end, security, and data storage. In the presentation layer, Customers, Staff, and Managers interact with the restaurant through a React SPA, which has been built using Vite and TailwindCSS, with HTTPS being used for communications with the back-end server. To optimize the process, a React Query server state cache has been employed to store the data that is accessed many times by the user locally. In addition to optimization, user authentication is also accomplished utilizing the Auth Context, which enables JWT tokens to be stored in the local storage along with user roles.

On the server side, FastAPI server using Uvicorn features a REST API with a CORS-enabled authentication to carry out business operations and respond to incoming requests from the front-end using Axios. A separate authentication module is applied to provide security using the JWT tokens that are verified via bcrypt for password protection and JOSE (HS256) for signing the tokens before authorizing a valid request to a secured endpoint to make sure that all requests are verified before they make their way through the business operations process. In addition, the back-end is equipped with the Assignment Service responsible for managing order dispatching to any public staff member automatically and cooperating with the order queue, which operates in FIFO, to make sure that all orders are processed correctly.

Data storage is performed by a PostgreSQL database that is being accessed by SQLAlchemy as an ORM layer, which provides management of the restaurant's major schema data, including users, menu items, orders, and staff assignments. Apart from structured data being stored, the system also comes with a separate File Storage component that deals with uploaded images and static files, which helps to retrieve images and static files separately from the main database, therefore minimizing the load on the main data layer. These two components together represent a single request life cycle: i.e., after user action was performed in a React SPA, it travels over HTTPS to the FastAPI server, where the request is authenticated, is acted upon according to business rules, and finally is stored in PostgreSQL or goes through the Order Queue, and then a response message with all the required static files is returned to the client.

The process of establishing an automated restaurant management and food delivery system requires an organized requirements engineering procedure for collecting various operation workflows. Modern food service environments function based on coordinated communications among front-of-house staff, cooking staff, delivery logistics staff, and managers. In order to achieve reliability of operations, integrity of data, and high level of service, the system must represent real restaurant practice in the form of specific functional requirements.

This chapter indicates the main specifications required for creating the system. Thus, by studying the roles and responsibilities of each party involved in the process of getting food (customers, waiters, chefs, delivery personnel, and managers), the software architecture establishes appropriate role-based access levels to the system, as well as appropriate information flows and state transitions of the system in real time.

3.2.3. Layer Architecture

The development of Back-end system follows the MVC (Model-View-Controller) architecture pattern. This is implemented as a layered architecture with clear separation of concerns:

- As each layer will have a well-defined responsibility. The database logic will reside within the repository, while the business logic, like password hashing, order processing, etc., will stay in services layer. At the same time, the HTTP issues will be managed through controller.

Consequently, by keeping these layers separate, it becomes easier to solve any undesired issue.

- Test ability: as each layer can be tested independently.
- Maintainability: while introducing any new entity/feature, one will need to create corresponding classes/files within each layer.

3.2.3.1. Model Layer

Models represent the essential data structures and definitions of business entities of the system which were developed with the help of SQLAlchemy ORM (Object-Relational Mapping) being mapped to PostgreSQL tables. They constitute domain entities of the application including: orders, employees, clients, tables, payments and reflect the data validation and relational constraints incorporated in the domain entities. With the help of models, persistent data definitions are being integrated by the system and allow distinction between the logic of the database schema and logic of the application providing consistency in data and flawless object-relational mapping in the business services. In this thesis, it is separated into three sub-layers:

Sub-layer	Directory	Responsibility
ORM Models	models/	Define database table structures as Python classes by using SQLAlchemy's declarative base.
DTO Sachemas	schemas/	Define Pydantic models for request validation(Create/ Update/PUT/DELETE) and response serialization (Response DTOs)
Repositories	repository/	Executes all direct database queries by using SQLAlchemy's session-based query API. Each repository provides CRUD operations and specialized queries(e.g find_by_status, find_by_customer_id,...)

Table 3.2.3.1 Model Layer

3.2.3.2. View Layer

Views are the website presentation layer, where users interact with and is built as a dynamic Single Page Application (SPA) by using React and Vite. This layer provides different user interfaces based on the user role (customers, waiters, chef, delivery man, and managers), allowing them to perform their activities and receive updates on the current operation. By separating this presentation layer from back-end application logic, the application can operate in real-time. The application is able to successfully communicate with its back-end by using RESTful JSON API.

Component	Directory	Responsibility
Pages	pages/Manager/, pages/staff/, pages/customer/, pages/common/	Full-page UI components, each corresponding to a specific

		route and user role
Components	components/	Reusable UI elements — MapLayout.jsx (interactive restaurant floor plan), layout/ (ManagerLayout, StaffLayout, CustomerLayout for navigation structure)
Context	context/	Global state management — AuthContext.jsx (JWT token, user role, login/logout), CartContext.jsx (shopping cart with localStorage persistence)
API Client	api/axios.js	Centralized Axios instance with BE server URL and a request interceptor that automatically attaches the JWT token from localStorage to every outgoing request's Authorization: Bearer header

Table 3.2.3.2: View Layer

3.2.3.3. Controller Layer

Controllers act as the orchestration tier and are implemented using FastAPI REST endpoint routers. This tier intercepts incoming HTTP requests from the Views, validates request DTO payloads, verify JWT authentication claims, and invokes the appropriate services such as order queue and staff assignment. By using controllers, it abstracts HTTP transaction mechanics from core business services, thus ensuring strict separation of concerns and enforcement of centralized access control before carrying out data operations.

Controller	**Prefix**	**Key Responsibilities**
auth.py	/auth	Login endpoint — delegates credential verification to AuthService
order.py	/orders	CRUD + status-based filtering + shipping order endpoints
payment.py	/payments	CRUD + status/method update (PATCH) + query by order
customer.py	/customers	CRUD + registration + search
staff.py	/staff	CRUD + registration + search +Role update
menu_item.py	/menu-items	CRUD + filter by type

table.py	/tables	CRUD + availability filtering + status update
order_item.py	/order-items	CRUD for individual line items within an order
staff_schedule.py	/staff-schedules	CRUD + multi-criteria filtering (by day, shift, staff)

Table 3.2.3.3 Controller Layer

3.2.4. Technology Stack

Layer		Libraries	Role
Front-end	Framework	ReactJS	UI component library
	Build Tool	Vite	Development server & bundler
	Styling	TailwindCSS	Utility-first CSS framework
	Routing	React Router DOM	Client-side navigation
	State & Data	TanStack React Query	Server state management & caching
	Axios	Axios	API communication with interceptors
Back-end	FastAPI (Python)	FastAPI (Python)	REST API framework
	ASGI Server	Uvicorn	High-performance async server
	ORM	SQLAlchemy	Object-Relational Mapping
	Data Validation	Pydantic	Request/response DTO schemas
	DB Driver	Psycopg2	PostgreSQL adapter for Python
	Authentication	python-jose (JWT)	JSON Web Token generation/verification
	Password Security	bcrypt	Password hashing

Database	PostgreSQL		Relational database (RDBMS)

Table 3.2.1.2 Technology stack summary

3.2.5. Technologies justification

3.2.5.1. Front-end Layer

- ReactJS (UI Component Library):

As evaluated in Section 2.4.1.2 regarding component Model and virtual DOM reconciliation, ReactJS is leveraged to design the component-based client-sided application. In the presented solution, it is used to implement the necessary UI parts, namely, the kitchen display Kanban, table selection layout, and customer's shopping cart, which utilize the virtual DOM rendering mechanism. This allows for frequent UI updates and renders without waiting for the whole page to refresh.

- Vite (Development Server & Bundler):

Vite is used as a development server and module bundler. Specifically, this project uses Vite's built-in HMR (Hot Module Replacement), which is beneficial for rapid multi-page application development. Additionally, Vite is used to bundle the production code with Rollup, which reduces the assets' sizes on the client side, thus decreasing the mobile and dining room tablet's UI loading times.

- Tailwind CSS (Utility-First CSS Framework):

Consistent with the design and layout principles reviewed in Section 2.4.1.4, the application makes use of the Tailwind CSS utility classes. They are used to construct the responsive behavior of the UI components in accordance with the different user roles. For example, specific statuses' colors help identify the orders' states in the kitchen's display. Additionally, the responsive UI utilizes the same design tokens to render elements on mobile phones and wide screens.

- React Router DOM (Client-Side Navigation):

React Router DOM is used to implement the routing mechanism. It is done by creating the separate routing instances for each of the application's pages (i.e., /, /staff, /manager), which are then merged together by the application's App component. In this way, the application's designers and developers achieve the isolation between the customers', staff, and manager's sections.

- Tanstack React Query (Server State Management & Caching):

TanStack React Query is used to implement the client-side server state management and caching. In the present solution, it is utilized to subscribe to the specific API endpoints, which allows for keeping the customer's order and table booking data in the client-side cache. This way, the application fulfills the requirements to update the kitchen display in real-time and reduce the server's requests.

- Axios (API Communication with Interceptors):

Axios is used to implement the REST API communication on the client side. In the current project, Axios interceptors are used to fetch the JSON web token from browser storage and add it to the request headers. In this way, the Axios instance, which is created for the Customer and Staff pages, fulfills the error handling and authentication requirements.

3.2.5.2. Back-end Layer

- FastAPI (REST API Framework):

In line with the comparative analysis of the server-side framework in Section 2.4.2.2, FastAPI was selected as the main tool for implementing the core async REST API. The framework's architecture comprises fourteen separated router modules, each representing a set of endpoints and following the Controller–Service–Repository pattern. The choice of the framework was justified by the project's needs in terms of high-concurrency request handling, which translates into the server's ability to process dining, payments, and staff status mutation requests concurrently.

- Uvicorn (ASGI Server):

In line with the set of benchmarks for the asynchronous tools in Section 2.4.2.2, Uvicorn was chosen as the ASGI server for hosting the FastAPI app. The tool was considered optimal for running the production-grade API due to its capacity to concurrently handle thousands of event loop iterations per second. Uvicorn was deployed to serve the application by utilizing the ASGI specification and Python's asynchronous features, including socket polling, to ensure the request handling during lunch and dinner time.

- SQLAlchemy (Object-Relational Mapping):

SQLAlchemy was selected as the ORM framework for model creation and database interaction. The library is used to establish the business logic and rules for all seventeen models of the domain layer. In particular, the framework is used to declare persistent objects and their attributes, track the transactions using scoped sessions (SessionLocal), and generate query validation rules for table joins, foreign keys, and column constraints. The tool enforces the ACID compliance during database operations, including order placements, reservations, and payments.

- Pydantic (Request/Response DTO Schemas):

As seen in Section 2.4.2.2, the project uses the contract-first approach to API design with the emphasis on request/response validation. Pydantic was selected as the validation library for its ability to parse the DTO classes and enforce the input and response data types and structures. The tool is used to implement the request/response deserialization and validation before the business logic is executed. In particular, the deserialization step is applied to the dining and checkout requests to support the request/response dto objects.

- Psycopg2 (PostgreSQL Database Adapter):

The application utilizes the PostgreSQL database with the set of features for the high-concurrency write-ahead log operations. Psycopg2 was selected as the database adapter to provide the low-level driver for PostgreSQL, which would support the connection pooling and the full binary protocol for psycopg2.adapters and psycopg2.sql. The tool enables the efficient use of the database with the concurrent SELECT and UPDATE statements, including the UPDATE with FOR UPDATE clause.

- python-jose (JSON Web Token Generation & Verification):

To build the stateless authentication model detailed in Section 2.4.2.4, the JWT tokens are used to enable the stateless authentication between the user's clients and the server. The python-jose library was chosen for its implementation of the HS256 symmetric encryption and key derivation functions to sign and encode the JWTs. The library was integrated into the FastAPI app to install the two endpoints: one to receive the user's credential and issue the JWT and another to validate the presented

JWT, from which the user's id and role will be extracted to establish the access rights to the protected API routes.

- bcrypt (Password Security):

The project uses bcrypt library to provide the one-way encryption of the users' passwords for storing them securely in the database. As provided in Section 2.4.2.4, the authentication service requires the hashing of the user's credentials to ensure confidentiality and resist the disclosure of the stored data in case of tampering. Using the bcrypt library, the cost factor is set to 12 rounds to ensure the secure hash generation prior to storing the customers' and employees' passwords in the database.

3.2.5.3. Database Layer

PostgreSQL (Relational Database Management System): As justified through the relational integrity and concurrency analysis in Section 2.4.2.3, the application employs the PostgreSQL database to ensure the ACID-compliant transactions and the MVCC isolation levels. The database is used to store the seventeen entities described by the domain layer models in a separated restaurant schema. The database provides the built-in locking mechanisms (SELECT … FOR UPDATE) to ensure the atomicity and isolation of the concurrent transactions, including dining, payment, and reservations.

3.3. User requirement analysis

User requirement analysis is the initial stage of the software development process. Its main goal is to relate general ideas and expectations to specific software elements. A food business enterprise implies day-to-day operations where multiple stakeholders work towards the same goal but with different means and objectives. In terms of F&B, the main actors are customers who seek satisfactory experience and additional staff, including waiters, cooks, couriers, and accountants. Each of these users operates on a separate interface with unique data and instructions. Therefore, it is necessary to make a user requirement analysis to ensure validity and remove potential roadblocks in data transmission and delivery.

This study considers entries of stakeholders in the analyzed restaurant, as well as the peculiarities of their operation. In addition, a few major actors in the system are identified. The functional and non-functional requirements of the system are determined based on the principles of their work and the data that they must process. The key components include the table service, which uses the application's interface and places orders, the high-priority display for the cook, and the courier's ability to track the order until its delivery. These and many other details are described further in this chapter. The gathered requirements are then broken down into functional components, use cases, and non-functional requirements. The collected data will be used as the basis for subsequent stages of development, including the creation of the database and the interface.

3.3.1. Actor identification and Role definitions

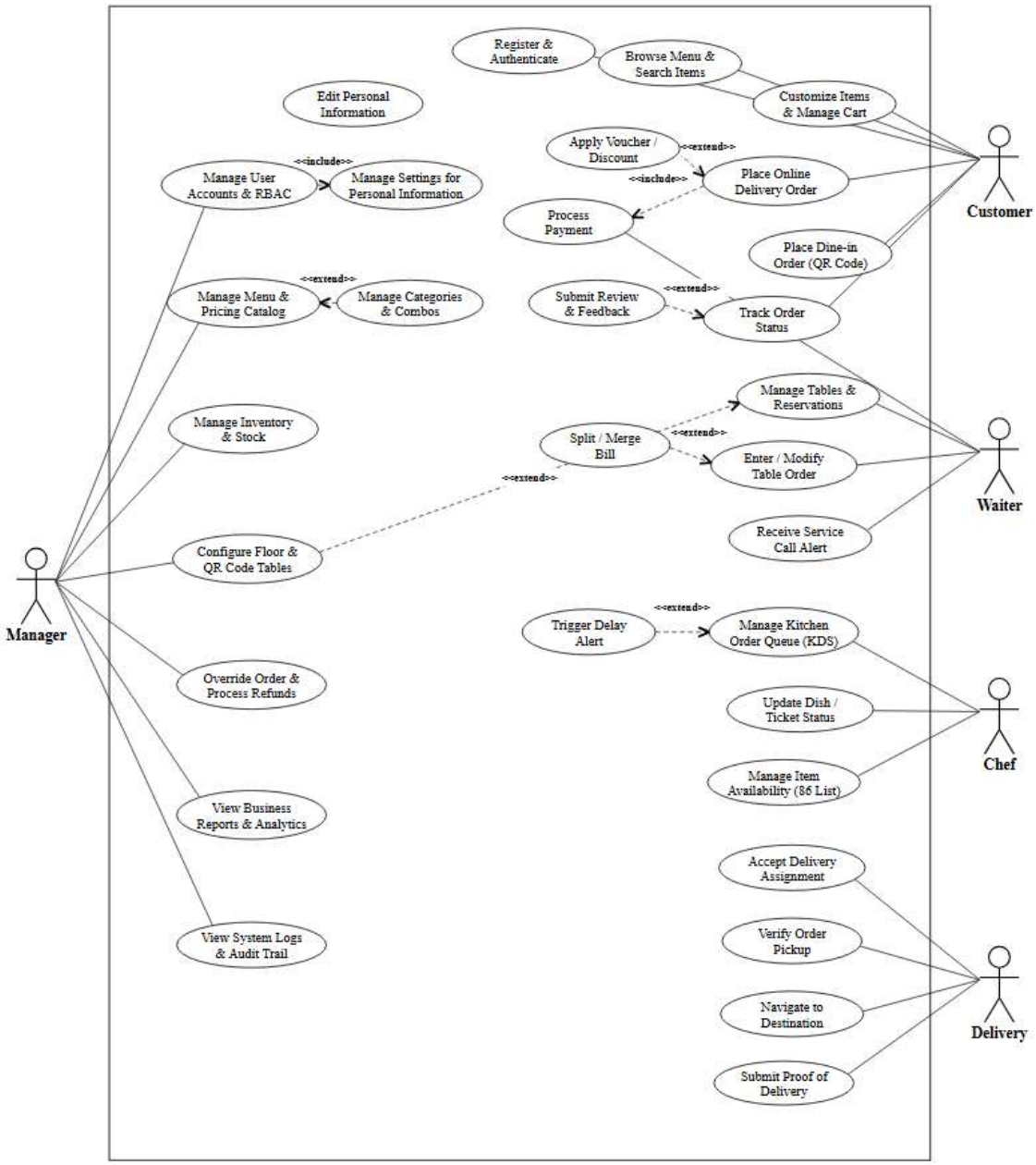


Figure 3.3 Use case diagram

Actors	**System Role(s)**	**Description**
Customer	customer	End-users who browse menus, place orders, make payments,
Waiter	waiter	Staff who select tables, create orders on behalf of walk-in customers and serve food
Chef	chef	Kitchen staff who view incoming orders and update

		cooking/readiness status
Delivery Man	delivery man	Staff who handle shipping orders — pick up ready orders, mark them delivering/delivered
Manager	Manager	Full system access — manages staff, menus, tables, schedules, payments, customers,

Table 3.3.1 Actors identification and Role definition

3.3.2. Use case descriptions

- **UC-1: Login**

The Login use case involves all the actors listed above in the description. The user is requested to log in by entering the correct username and password, after which a POST request is sent to the server asking to /auth/login. The server finds the user by the given username and then compares the given password with the bcrypt hash stored in the database. The comparison is performed using constant time algorithm to prevent possible attacks. If the credentials are correct, a JWT is issued with user data and the role they play in the application. The client stores the received token in the browser so that subsequent requests are authorized. Then the client router parses the token, reads the role from it, and performs a redirect to the main page of the selected user. If the login was not successful, an error is returned, and a message is displayed on the screen indicating the problem. In this case, the user's username and password will be saved in the browser so that they can be used again.

Field	**Description**
Name	UC-1 Login
Actors	Customer, Waiter, Chef, Delivery Man, Manager
Precondition	The user has a registered account in the system (customer or staff). The user is not currently logged in.
Post condition	The user is authenticated, a JWT token is stored in localStorage, and the user is redirected to the appropriate dashboard based on their role.
Main Scenario	1.The user navigates to the Login page. 2. The user enters their username and password. 3. The user clicks the "Sign In" button. 4. The system sends a POST request to /auth/login with the credentials. 5. The back-end verifies the credentials against both the Staff and Customer tables. 6. Upon successful verification, the back-end generates a signed JWT access token

	containing the user's ID and role, returning it to the client. 7.The front-end stores the JWT in localStorage to authorize subsequent requests via the Authorization: Bearer <token> header, alongside non-sensitive session metadata (user ID, role). 8.The front-end router inspects the token's role claim to guide the user to their initial view (Manager → /manager/dashboard, Waiter/Chef → staff/dashboard, Delivery → /staff/delivery, Customer → /menu), while all subsequent protected resources remain guarded by backend token validation.
Alternative Scenario	**A1 — Invalid credentials:** At step 5, if the username does not exist or the password does not match, the back-end returns a 401 error. The front-end displays "Invalid credentials" and the user remains on the Login page. **A2 — Empty fields:** At step 3, if either field is empty, the browser's required-field validation prevents form submission.

Table 3.3.2.1 Use case description for Login

- **UC-2 Register Customer Account**

The Register Customer Account use case enables the target customer to register on the restaurant management system. The requester accesses the application form on the restaurant's website and fills in their personal information, including the full name, username, email address, phone number, and password. Upon clicking the submit button, the request is checked if the input data is in the correct format. The application will accept a valid email and a password with at least a minimum number of characters. The request then moves to the customer provision endpoint.

The controller checks if the submitted email and username are already in the system. It routes the request to the customer repository, which searches the database for the availability of the new customer's details. After the successful provision, the controller forwards the request to the security module. The process involves creating a salted password using bcrypt with an iteration count of 12. The new customer's plain text password will then be inserted into the database with the customer's personal information and an indicator of an active account. Finally, the server will respond with a success message, which is redirected to the login page to authenticate the new customer.

Field	**Description**
Name	UC-2 Register Customer Account
Actors	Customer
Precondition	The user does not have an existing account.
Post condition	A new customer account is created with the role customer. The password is hashed and stored securely.

Main Scenario	1. The user navigates to the Registration page via the "Create one" link on the Login page. 2. The user fills in the registration form: username, password, full name, email, and phone number. 3. The user submits the form. 4. The system validates that the username and email are unique. 5. The back-end securely hashes the plain-text password using bcrypt (12 salt rounds), immediately discarding the raw password from memory. 6. The system persists the new Customer record with the salted password hash and an active status flag into the database. 7. The server returns a 201 Created response, and the front-end redirects the user to the Login page with a success message.
Alternative Scenario	**A1 — Duplicate username:** At step 4, if the username already exists, the system returns an error "Username already exists" and the form remains open. **A2 — Duplicate email:** At step 4, if the email already exists, the system returns an error "Email already exists

Table 3.3.2.2 Use case description for Register customer account

- **UC-3 Browse Menu**

This Browse Menu application feature allows both unregistered users and registered clients to see and learn about the food and drinks offered in the restaurant, including prices and nutritional values. Once the menu webpage is opened, the app will issue an HTTP GET request asynchronously using TanStack React Query to get the current menu from /menu-items and use client-side caching to lessen the server load during the future access of the menu. The system displays the food and drink items divided into several groups (appetizers, mains, desserts, beverages, etc.) as well as represented as individual dishes shown in the card components with high-quality images of dishes and basic information about them – name, description, price per unit, and overall customer star rating. Real-time availability indicators are displayed to inform users whether the dish is sold out or has "86'd" status imposed by the kitchen staff thereby making ordering impossible. Users can filter the dishes according to categories, search for they desired items using keywords, or click on the dish that interests them and view the extra information about possible allergens and how the dish is made.

Field	**Description**
Name	UC-3 Browse Menu
Actors	Customer
Precondition	The user has access to the application (login is optional for menu browsing).

Post condition	The user can view all available menu items with their details.
Main Scenario	1. The user navigates to the Menu page (/menu). 2. The system fetches all menu items from GET /menu-items/. 3. The system displays menu items in a grid/list with item name, type, price, description, and image. 4. The user can filter items by type (food/drink). 5. The user can click on an item to view details or add it to the cart.
Alternative Scenario	**A1 — No items available:** If no menu items are returned, the system displays an "No items available" message. **A2 — Item unavailable:** Items with is_available = false are either hidden or displayed with an "Unavailable" badge and cannot be added to the cart.

Table 3.3.2.3 Use case description for Browse Menu

- **UC-4 Manage Cart**

The Manage Cart use case allows shoppers to assemble, check, and amend their dish selections before proceeding with an order. When a visitor clicks on the “add-to-cart” button for a menu item, the procedure interacts with the CartContext, either saving the item as a separate line-item or updating the record. The client views all items in a designated cart view or slide-over drawer to see what has been chosen, dynamically increase or decrease the item quantities, or to remove items completely. In this way, all changes mean that the cart automatically re-calculates item amounts, provides discounts, and generates a summary of the gross amount to be paid. All updates of cart status are replicated in local browser storage.

Field	**Description**
Name	UC-4 Manage Cart
Actors	Customer
Precondition	The user is on the Menu page or Cart page.
Post condition	The cart reflects the user's selected items with correct quantities, notes, and calculated totals. Cart data is persisted in local Storage.
Main Scenario	1. The user selects a menu item and specifies a quantity and optional notes on the Menu page. 2. The user clicks "Add to Cart". 3. The system adds the item to the cart (or merges quantities if the same item with the same notes already exists).

	4. The cart icon in the navigation bar updates to show the current item count. 5. The user navigates to the Cart page (/cart) to review items. 6. The user can increase/decrease quantities or remove items. 7. The system displays the real-time cart total.
Alternative Scenario	**A1 — Clear cart:** The user clicks "Clear Cart" to remove all items at once. **A2 — Browser closed:** If the user closes the browser and reopens it, the cart is restored from localStorage

Table 3.3.2.4 Use case description for Manage Cart

- **UC-5 Place Dine-in Order**

The Place Dine-in Order use case allows for the placement and kitchen processing of dine-in orders made by either customers or waitstaff. Whenever a dine-in order is placed by an on-site customer, the diner is linked to a particular table either by scanning a unique QR code at the correct table, obtaining a table_id from the QR code, or choosing a table directly from a floor map. Once the user has added the items to their cart and added optional notes about the dishes (e.g. dietary restrictions, spice levels), the order is passed on to the backend. The system then executes an independent database transaction which includes confirming the table is free, creating a new order header with the service type on-premise and status Pending, adding the required information to order_items, and changing the status of the table to occupied. Once all these operations are performed, the created order will be transferred to the kitchen operations pipeline and the table will receive a confirmation screen.

Field	**Description**
Name	UC-5 Place Dine-in Order
Actors	Customer
Precondition	The customer is logged in, has items in the cart, and at least one table is available.
Post condition	A new order is created with status paid. The selected table is marked as occupied. Payment record is created with status completed. Cart is cleared.
Main Scenario	1. The customer navigates to the Table Selection page (/tables). 2. The system fetches all tables from GET /tables/ and displays available (unoccupied) tables. 3. The customer selects an available table. 4. The customer selects a payment method (Cash or QR). 5. The customer confirms the order.

	6. The customer submits the order, and the client sends an atomic checkout request via POST /orders/ (or POST /orders/checkout) containing the order parameters, cart items, and chosen payment method. 7. Within a single database transaction, the system: a) Validates table availability and sets is_occupied = true. b) Creates the order record with order_type = dine_in. c) Persists all order line items in bulk. d) Applies any eligible discounts/coupons and records the initial payment record. (If any sub-operation fails, the entire transaction is rolled back, preventing orphaned records). 8.The system commits the transaction and returns the confirmed order and payment summary. 9.The cart is cleared and the customer is redirected to the Orders/Payment page.
Alternative Scenario	**A1 — No available tables:** At step 2, if all tables are occupied, no tables are selectable and the customer must wait. **A2 — QR payment:** At step 4, if QR is selected, the system redirects to the QR Payment page displaying a QR code before confirming.

Table 3.3.2.5 Use case description for Place Dine-in Ordering

- **UC-6 Place Shipping Order**

The Place Shipping Order use case allows remote customers to place orders from outside the premises for takeout or delivery. When the customer finishes choosing dishes and adding them to the cart, they go to the checkout window and select the type of delivery and the shipping details, including the address, phone number, and special instructions for delivery. The customer now needs to choose a payment option, either through bank transfer using dynamic QR code creation or by making a cash payment upon delivery. The order is placed, and an HTTP POST request is sent to the /orders endpoint, where the system verifies stock quantities and checks if the address input complies with the operational radius regulations, and registers the order header as "Shipping/Delivery" and assigned a status of "Pending" or "Paid," as per payment confirmation. The completed system then provides the order tracking number which can be used to trace the stages of order completion.

Field	Description
Name	UC-6 Place Shipping Order
Actors	Customer
Precondition	The customer is logged in and has items in the cart.
Post condition	A new shipping order is created with status paid. No table is assigned. Payment record is created. Cart is cleared.
	1. The customer navigates to the Shipping page (/shipping). 2. The customer enters a delivery address. 3. The customer selects a payment method (Cash or QR). 4. The customer confirms the order. 5. Within a single database transaction, the system: a) Creates the order record with order_type = Shipping and shipping address b) Persists all order line items in bulk. c) Applies any eligible discounts/coupons and records the initial payment record. (If any sub-operation fails, the entire transaction is rolled back, preventing orphaned records). 6.The system commits the transaction and returns the confirmed order and payment summary. 7.The cart is cleared and the customer is redirected to the Orders/Payment page.
Alternative Scenario	**A1 — Empty address:** At step 4, if the address field is empty, the system displays a validation error and prevents submission. **A2 — QR payment:** If QR is selected, the customer is shown a QR code payment screen before order finalization.

Table 3.3.2.6 Use case description for Place Shipping Ordering

- **UC-7 Chef Manages Kitchen Orders**

The Chef Manages Kitchen Orders use case describes the chef's activities inside the kitchen, where he uses the Kitchen Display System to track the progress of the tickets. Using the KDS, the chef can see the orders retrieved from the /orders endpoint in the "Pending" and "Cooking" statuses. The display shows the order tickets with their sequence numbers, time of creation, quantity, special

modifiers, and indicates the type of dining (dine-in or takeaway). The tickets have to be moved by the chef from one status to another, where the “Ready” status triggers the action for waiters to take the orders to the customers or for the couriers to pick them up. In addition, the tickets can be manually closed by the chef to indicate that a particular dish is no longer available.

Field	**Description**
Name	UC-7 Chef Manages Kitchen Orders
Actors	Chef
Precondition	The chef is logged in with role chef. There are orders with status paid or preparing.
Post condition	Order statuses are updated to reflect kitchen progress (preparing → ready).
Main Scenario	1. The chef views the Kitchen Display (Chef Dashboard). 2. The system fetches all dine-in orders with status paid or preparing. 3. Each order card displays the order items with actual item names, quantities, and special instructions. 4. For paid orders, the chef clicks "Start Cooking" to update status to preparing. 5. For preparing orders, the chef clicks "Mark Ready" to update status to ready. 6. The dashboard auto-refreshes to show new incoming orders.
Alternative Scenario	**A1 — No orders:** If no orders are pending or preparing, the dashboard shows an "All caught up!" message.

Table 3.3.2.7 Use case description for Chef Manages Kitchen Orders

- **UC-8 Waiter Monitors and Manages Active Orders**

The Waiter Monitors and Manages Active Orders use case allows the waiter to monitor and manage the active orders as well as the floor. The waiter logs in to their mobile tablet device and accesses the active order screen. This screen provides him with the list of the tables, active tickets, stage of meals, and special offers requested. When the KDS indicates the order is “Ready”, the waiter is alerted and goes to the kitchen pass to collect the meals and delivers them to the customer. The waiter then updates the status of the ticket to “Served”. The waiter also uses the use case to manage the tables. They can add a new beverage or side to the ticket, void, print, or bill the tickets. Waiters can also end a guest’s meal by presenting a check or a digital payment screen after the food has been delivered.

Field	**Description**
Name	UC-8 Waiter Monitors and Manages Active Orders
Actors	Waiter
Precondition	The waiter is logged in. There are active orders

	in the system.
Post condition	Order statuses are updated according to the workflow. Tables are freed when orders are completed.
Main Scenario	1. The waiter navigates to Active Orders (/staff/orders). 2. The system fetches all orders and displays them sorted by newest first. 3. For orders with status ready, the waiter clicks "Mark Serving" to update status to serving. 4. For orders with status serving, the waiter clicks "Mark Served" to update status to served. 5. For orders with status served, the waiter clicks "Complete" to update status to completed. 6. The system frees the associated table (is_occupied = false).
Alternative Scenario	**A1 — No active orders:** If there are no active orders, the page displays an empty state message.

Table 3.3.2.8 Use case description for Waiter Monitors and Manages Active Orders

- **UC-9 Delivery Man Ships Orders**

The Delivery Man Ships Orders use case covers the entire outbound logistics and execution process of off-premises customer deliveries. After getting confirmation from a manager and the kitchen, delivery drivers get access to a mobile-friendly dashboard with the orders that the drivers should make delivery of. The drivers get important data for delivery purposes such as customer contact numbers, the location where the order is to be delivered, and navigational maps/directions that include other necessary delivery information. After taking the pack from the restaurant, the driver changes the status of the order from “Ready” to “Out for Delivery”, and at that moment, the customer gets notified of the ongoing delivery process. When the driver arrives at the location where the order is to be delivered, he/she marks the transaction as successful by ensuring the payment has been received and handing over the order, thus completing the entire order lifecycle and bringing his/her status back to ready for potential delivery.

Field	**Description**
Name	UC-9 Delivery Man Ships Orders
Actors	Delivery Man
Precondition	The delivery man is logged in with role delivery_man. There are shipping orders with status ready or delivering.
Post condition	Shipping order statuses are updated to reflect delivery progress.
Main Scenario	1. The delivery man views the Delivery Dashboard (/staff/delivery). 2. The system fetches all active shipping orders

	from GET /orders/shipping/active. 3. Each order card displays the customer's shipping address, order items, and total amount. 4. For ready orders, the delivery man clicks "Pick Up" to update status to delivering. 5. For delivering orders, the delivery man clicks "Mark Delivered" to update status to delivered.
Alternative Scenario	**A1 — No deliveries:** If no shipping orders are active, the dashboard shows an empty state. **A2 — Order canceled:** If an order is canceled while the delivery man is en route, the order disappears from the dashboard.

Table 3.3.2.9 Use case description for Delivery Man Shipping Orders

- **UC-10 Manager Views Dashboard Analytics**

The Manager Perspectives Dashboard Analytics approach allows restaurant authorities to obtain consolidated information on operations, reports about finances, and performance metrics across the system. When entering the establishment's management interface, the system enables data analytics through a service layer creating key performance indicators (KPIs) based on time periods chosen by the user. The manager analyzes the high-level summary metrics, such as total sales income, number of completed orders, average check and percentage of occupied tables. Below the summary layer, graphs illustrate weekly sales dynamics, peak time orders, and dishes. Additionally, the dashboard calculates customer satisfaction averages based on dish reviews and Nobel requests from VIPs and allows authorities to make effective decision about staff timetables, promotional discounts, and list of orders to purchase.

Field	Description
Name	UC-10 Manager Views Dashboard Analytics
Actors	Manager
Precondition	The Manager is logged in with role Manager.
Post condition	The Manager can view real-time business metrics and statistics.
Main Scenario	1. The Manager navigates to the Dashboard (/Manager/dashboard). 2. The system fetches aggregated data from multiple endpoints (orders, payments, tables, staff). 3. The dashboard displays: today's total revenue from completed payments, total order count with breakdown by status, number of currently occupied tables, and total staff count. 4. The Manager uses this information to monitor restaurant operations in real time.
Alternative Scenario	**A1 — No data:** If there are no orders or payments for the day, the dashboard displays zero values for revenue and order counts.

Table 3.3.2.10 Use case description for Manager Views Dashboard Analytics

3.3.3. Functional Test Cases

Test ID	Use case	Actors	Action	Result
FT-UC1-01	UC-1 Login	Registered active customer	Submit valid credentials	Login succeeds; customer identity and token/session returned
FT-UC1-02	UC-1 Login	Registered customer	Submit incorrect password	Rejected; no token/session issued
FT-UC1-03	UC-1 Login	Unknown account	Submit credentials	Rejected without disclosing account existence
FT-UC1-04	UC-1 Login	Any user	Submit missing/malformed credentials	Validation error; no session issued
FT-UC1-05	UC-1 Login	Invalid or expired token	Call protected endpoint	401 Unauthorized
FT-UC2-01	UC-2 Register Customer Account	New email; valid required fields	Submit registration	Customer account created with customer role
FT-UC2-02	UC-2 Register Customer Account	Existing email	Submit registration	Rejected; existing account unchanged
FT-UC2-03	UC-2 Register	Invalid email or missing	Submit registration	Validation error; no

	Customer Account	field		account created
FT-UC2-04	UC-2 Register Customer Account	Payload requests manager/chef role	Submit registration	Privileged role ignored or rejected; no escalation
FT-UC3-01	UC-3 Browse Menu	Anonymous visitor; available items exist	Request menu	Available items and required display fields returned
FT-UC3-02	UC-3 Browse Menu	Menu contains unavailable item	Request menu	Item omitted or marked unavailable, per product rules
FT-UC3-03	UC-3 Browse Menu	Empty menu or service failure	Request menu	Controlled empty state/error; application remains usable
FT-UC4-01	UC-4 Manage Cart	Customer; available items	Add, update quantity, remove	Cart updates correctly; subtotal recalculated
FT-UC4-02	UC-4 Manage Cart	Customer; item in cart	Set zero, negative, or invalid quantity	Rejected; cart remains valid
FT-UC4-03	UC-4 Manage Cart	Cart contains unavailable/deleted item	Refresh cart or checkout	Stale item flagged/rejected; stale price cannot be ordered
FT-UC5-01	UC-5 Place Dine-in Order	Customer; valid cart and table details	Submit dine-in order	Correct type, items, total, table, and initial status persisted
FT-UC5-02	UC-5 Place Dine-in Order	Customer; empty cart or missing table	Submit dine-in order	Rejected; no order created
FT-UC5-03	UC-5 Place Dine-in Order	Customer token	Call manager analytics endpoint	403 Forbidden; analytics data absent
FT-UC6-01	UC-6 Place Shipping Order	Customer; valid cart and address	Submit shipping order	Shipping order and address persist with correct items/total
FT-UC6-02	UC-6 Place Shipping Order	Customer; invalid/missing address	Submit shipping order	Rejected; no order created
FT-UC6-03	UC-6 Place Shipping Order	Customer token	Call delivery-only shipment endpoint	403 Forbidden; no order mutation
FT-UC7-01	UC-7 Chef Manages Kitchen Orders	Chef; eligible kitchen orders exist	Request kitchen queue	Only eligible kitchen orders returned
FT-UC7-02	UC-7 Chef Manages Kitchen Orders	Chef; eligible order	Apply allowed kitchen status transition	Update succeeds and persists
FT-UC7-03	UC-7 Chef Manages Kitchen Orders	Chef; invalid transition	Update kitchen status	Rejected; stored status unchanged
FT-UC7-04	UC-7 Chef Manages Kitchen Orders	Customer token	Call chef queue/update endpoint	403 Forbidden; no disclosure or mutation
FT-UC8-01	UC-8 Waiter Monitors and Manages Active Orders	Waiter; active orders exist	Request active-order list	Current active orders and service details returned
FT-UC8-02	UC-8 Waiter Monitors and Manages Active	Waiter; active order	Apply allowed waiter action	Update succeeds and persists

	Orders			
FT-UC8-03	UC-8 Waiter Monitors and Manages Active Orders	Waiter; terminal order	Attempt disallowed update	Rejected; terminal order unchanged
FT-UC8-04	UC-8 Waiter Monitors and Manages Active Orders	Customer token	Call waiter order endpoint	403 Forbidden; no disclosure or mutation
FT-UC9-01	UC-9 Delivery Man Ships Orders	Delivery user; eligible shipment	Request shipping queue	Eligible/assigned shipments returned per rules
FT-UC9-02	UC-9 Delivery Man Ships Orders	Delivery user; eligible shipment	Mark shipped/delivered	Permitted status update persists
FT-UC9-03	UC-9 Delivery Man Ships Orders	Delivery user; unassigned/ineligible order	Attempt shipment update	Rejected; order state unchanged
FT-UC9-04	UC-9 Delivery Man Ships Orders	Customer token	Call delivery list/update endpoint	403 Forbidden; no data or mutation
FT-UC10-01	UC-10 Manager Views Dashboard Analytics	Manager; known fixture data	Request metrics for a period	Metrics match fixture data and selected period
FT-UC10-02	UC-10 Manager Views Dashboard Analytics	Manager; invalid date/filter	Request analytics	Validation response or documented empty result
FT-UC10-03	UC-10 Manager Views Dashboard Analytics	Customer token	Call manager analytics endpoint	403 Forbidden; analytics payload absent
FT-UC10-04	UC-10 Manager Views Dashboard Analytics	Missing/invalid/expired token	Call manager analytics endpoint	401 Unauthorized

Table 3.3.3 Functional Test Cases

3.3.4. Functional Requirements

3.3.4.1. Restaurant Manager Functional Requirements

The whole restaurant ecosystem is completely controlled by managers through the help of an integrated administration dashboard. In catalog and asset management, managers perform operations that involve creation, reading, updating, and deletion (CRUD) techniques involving menu categories, dish specifications, dynamic pricing plans, and multimedia holdings, in conjunction with systematic inventory threshold controls that help to keep track of inventory depletion and replenishment. When it comes to organization, managers handle workforce governance by granting staff access privileges and implementing role-based access control (RBAC) systems. As for financial operations, the administrative arm generates cash flow data and represents sales trends using charting libraries, as well as causing the governance of discounts and coupons and the auditing of taxes and payments. Finally, management is responsible for the setup of global system parameters, including operating hours, maximum dining duration, table capacity limits and automatic queue dispatch thresholds.

Requirement Identifier	Requirement Name	Detailed Description
FR-MGR-01	Role-Based Access Control (RBAC)	Provisions, edits, and revokes employee accounts, defining precise permission scopes for waiters, kitchen staff, delivery drivers, and shift leads.

FR-MGR-02	Menu & Catalog Engineering	Manages category hierarchies, creates new menu items, sets pricing rules, updates descriptive metadata, and configures scheduled promotion bundles.
FR-MGR-03	Table & Zone Configuration	Adjusts table seating capacities, occupancy
FR-MGR-04	Business Intelligence & Analytics	Generates aggregate analytical reports detailing gross revenue, popular menu items, peak ordering hours, average fulfillment latency, and driver efficiency metrics.
FR-MGR-05	System Auditing & Operational Logs	Tracks user activity logs, transaction histories, price modifications

Table 3.3.3.1: Manager Functional Requirements

3.3.4.2. Delivery Personnel Functional Requirements

Delivery agents manage the outward fulfillment logistics for the off-premise orders by means of a specific mobile-compatible courier platform. After the order has been confirmed and the food has been prepared, the agents receive notifications concerning the dispatch based on FIFO dispatcher priority in a queue called OrderQueue. The platform allows to get transit information that consists of the contact information of the client and the delivery addresses, geographic coordinates of the waypoints and some details for transit (for example, contactless delivery). Once the food has been received by the agent, he fills in the status in real time by changing it from “Out for delivery” to “Delivered”. In order to make the workflow more accountable the interface uses different methods of verifying proof of delivery by providing the system with customer signature or informing the cooperation that the order has been delivered.

Requirement Identifier	Requirement Name	Detailed Description
FR-DEL-01	Driver Onboarding & Availability Status	Toggle availability status (Online / Ready for Dispatch / Offline).
FR-DEL-02	Order Pickup Verification	Receives incoming delivery dispatch assignments, validates package pickup from the kitchen
FR-DEL-03	Dispatch & Order Acceptance	View item details, drop-off location
FR-DEL-04	Proof of Delivery Submission	Customer confirmation

Table 3.3.3.2: Delivery Personnel Functional Requirements

3.3.4.3. Chef Functional Requirements

Kitchen staff (cooks and executive chefs) engage with a responsive Kitchen Display System (KDS) in order to minimize errors in the preparation process and simplify the flow of food production orders. Incoming food orders are processed based on the time of order, type of service (being eat-in, delivery, etc.), and the difficulty of the preparation procedure, employing a visual Kanban-type status tool. Line cooks manage the life span of orders at the station level, moving orders across specific stages of preparation ("Pending" -> "Preparing" -> "Ready"). The KDS screen shows ticket and item-specific remarks, warnings, and table numbers, working together with the system of timers that shows when the orders are ready to be served.

Requirement Identifier	Requirement Name	Detailed Description
FR-CHF-01	KDS Station Login	Grants kitchen staff access to specific preparation stations (e.g., Grill, Pastry, Beverage, Line Assembly) based on assigned work shifts.
FR-CHF-02	Real-Time Ticket Queue	Displays incoming order tickets chronologically with visual color codes highlighting wait times, urgency levels, and order fulfillment channels (Dine-in vs. Delivery).
FR-CHF-03	Item & Ticket Progression	Updates the execution life cycle of individual dishes and overall order tickets across stages (Queued, Preparing, Ready).

Table 3.3.3.3: Chef Functional Requirements

3.3.4.4. Waiter Functional Requirements

Egged on by their waiters, the front-of-house dining floor operations are executed in a systematic manner, ensuring the smooth turnover of tables in a way that facilitates customer experience by helping them through the intermediate process of dining within the premises. The dining staff uses the portable tablets as a means of receiving and displaying the interactive floor chart layout that shows the status of all the tables by the help of various indicators such as, but not limited to, whether the table is reserved or ready to be used by the guest or is being cleaned. The duties performed by the waiters include the process of assigning tables, entering dine-in orders into the system with the help of swift order collection, adding information related to the dish preparation (e.g., cooking temperature) and ensuring delivery of the order to the KDS system through HTTP and REST protocols. Moreover, the waiters monitor the progress of the entire life cycle of the table as well as assist the guests with their calls in case of necessity, correct any errors if needed and manage bill elimination processes.

Requirement Identifier	Requirement Name	Detailed Description
FR-WTR-01	Shifts	Allow waiter to view working shifts
FR-WTR-02	Interactive Floor & Table Map	Displays a real-time visual map of the dining floor indicating table occupancy statuses (Vacant, Occupied, Reserved, Bill Requested, Needs Cleaning).
FR-WTR-03	Table Assignment & Reservation	Allows staff to seat incoming guests, link customer parties to table identifiers, and check upcoming reservation schedules.
FR-WTR-07	Order Status Monitoring	Alerting the waiter when finished items are ready for food running and table delivery.

Table 3.3.3.4: Waiter Functional Requirements

3.3.4.5. Customer Functional Requirements

Restaurants bring customers into their ecosystem through an omnichannel approach that includes both off-premise remote ordering and digital dining on-premise. When ordering remotely, the customer uses a unified web solution to view their food provided in attractive and high-quality imagery through categorized digital menus. They can filter menus according to their dietary needs, have their selected items stored in their shopping basket until final checkout that could be made through different payment methods (QR code, e-wallet, bank transfer, etc.). In contrast, when dining

inside the restaurant, customers interact with the restaurant's system by using their mobile phones to scan the dynamic QR code issued specifically for their table. The scanned URL connects the order to a specific table (table_id) and loads an online ordering system for customers to submit their dinner orders directly to the kitchen without involving waiters. At the same time, customers are enabled to track the status of their orders ("Confirmed"-> "Pending" -> "Ready"->"Serving/Delivering") and finally receive a bill at the end of the dining experience.

Requirement Identifier	Requirement Name	Detailed Description
FR-CUS-01	Account Registration & Authentication	Allows customers to register, authenticate via email/phone credentials or OAuth providers, securely recover passwords, and maintain persistent session states.
FR-CUS-02	Profile & Coupons	Enables users to update personal profiles, and view available coupon
FR-CUS-03	Menu Browsing & Search	Provides a searchable, categorized digital menu featuring item descriptions, pricing
FR-CUS-04	Cart Management	Enables adding, modifying quantities, removing items, and reviewing pricing breakdowns (including subtotals, applied promotions, service fees, and taxes).
FR-CUS-05	Checkout & Multi-Channel Payment	Facilitates secure payment transactions via integrated gateways (credit/debit cards, e-wallets, bank transfers) or cash-on-delivery (COD).
FR-CUS-06	Real-Time Order Tracking	Displays live order status milestones (Pending, Confirmed, In Preparation, Ready for Pickup, Out for Delivery, Completed) with map tracking.
FR-CUS-07	Feedback & Review Submission	Enables customers to submit numerical ratings and qualitative reviews with photo attachments for ordered dishes, delivery service, and overall experience.
FR-CUS-08	Order History & Reordering	Maintains a chronological archive of completed and canceled orders, allowing users to view digital invoices and reorder past baskets with a single action.

Table 3.3.3.5: Customer Functional Requirements

3.3.5. Non-Functional Requirements

ID	Requirement	Category
Non-FR-01	Uses JWT-based authentication with HS256 algorithm for stateless session management	Security
Non-FR-02	All passwords will be hashed using bcrypt before storage	Security
Non-FR-03	The back-end API uses CORS middleware to restrict access and allow for known front-end origins	Security
Non-FR-04	The system uses PostgreSQL with a dedicated restaurant schema for all database tables	Reliability
Non-FR-05	The system enforces referential integrity with foreign keys and cascading deletes across all related entities	Data Integrity

Table 3.3.4 Non-functional requirements

3.3.6. System Data Flow and Interaction

3.3.6.1. Sequence flow

The sequence diagram captures the exact sequencing of interactions happening during the use case: ordering by the customer. There are three parts in the execution of this sequence: Authentication, Order Placement, and Staff Dispatch. Each of these contains different components: Customer, React SPA, FastAPI, JWT Auth, Assignment Service, and PostgreSQL.

During the first stage of the operation, Authentication, the customers enter their credentials in React SPA, which results in sending the POST request to the /auth/login on the FastAPI server. The FastAPI server will ask JWT Auth service to verify the credentials. The JWT Auth will confirm the password and send back the signed JWT access token containing the user's role.

During the Order Placement process, the React SPA makes a POST request to /orders after the customer adds products to their cart. The FastAPI performs the task of storing the order as well as the various items associated with the order in PostgreSQL and transmitting the order_id back along the call chain.

After the order gets created, a similar POST request to /payments is sent to document the payment process. This ensures that the order payment process remains consistent throughout the operation.

In the Staff Dispatch phase, the system autonomously tries to search for available staff for fulfilling the order by calling the function try_assign_chef(). The function fetches an available chef record from PostgreSQL and changes the status of the chef to "preparing" if a match is found. The FastAPI subsequently verifies the assignment to React SPA after which the order of the customer is marked as successfully placed and confirmed. This shows that the order placement is not an atomic process but rather a coordinated process that requires various authenticated, validated and stored steps to be performed.

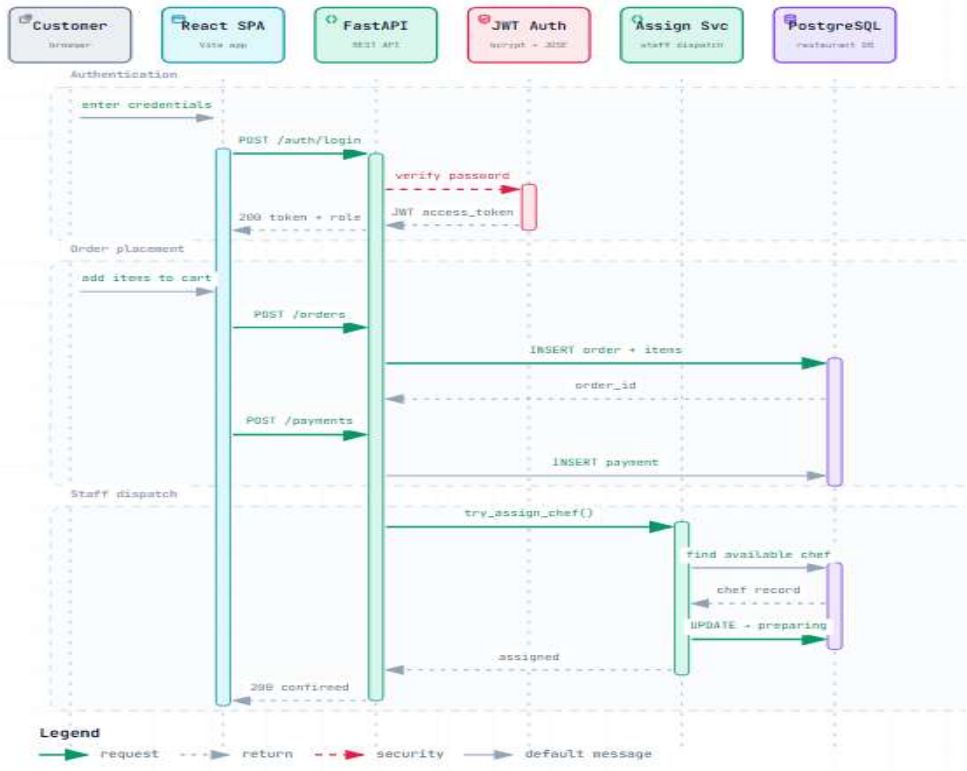


Figure 3.4: Customer Order Placement Sequence

3.3.6.2. State Diagram

The working process carried out by the Restaurant Management System represents the entire order processing cycle, as well as the links with every participant in the system. The process starts from the moment the client logs in or registers through the interface. After the login is successful, the client can use the digital catalog, choose the dishes they want, and place the order in the shopping cart using the React Context API.

When the order is made, the system takes two paths of order fulfillment. The first one is when the client dines at the restaurant and chooses the table to place the order at. The order is then saved in the system with the status marked as “pending”. After that the order gets from the Status of pending to preparing from the Chef after the order appears on the Kitchen Display System. After the order comes to the phase of ready, the waiter is informed about it and goes to the kitchen to serve the order. In the delivery situation, a member of the delivery team accepts the task once he has received a new order. The delivery person is responsible for checking the details of the order before bringing it to the end customer, after which the status of the order changes to delivered.

The manager has an overall dashboard at all times to see ongoing operations, track the status of the order, and understand the revenue and the status of employees' actions.

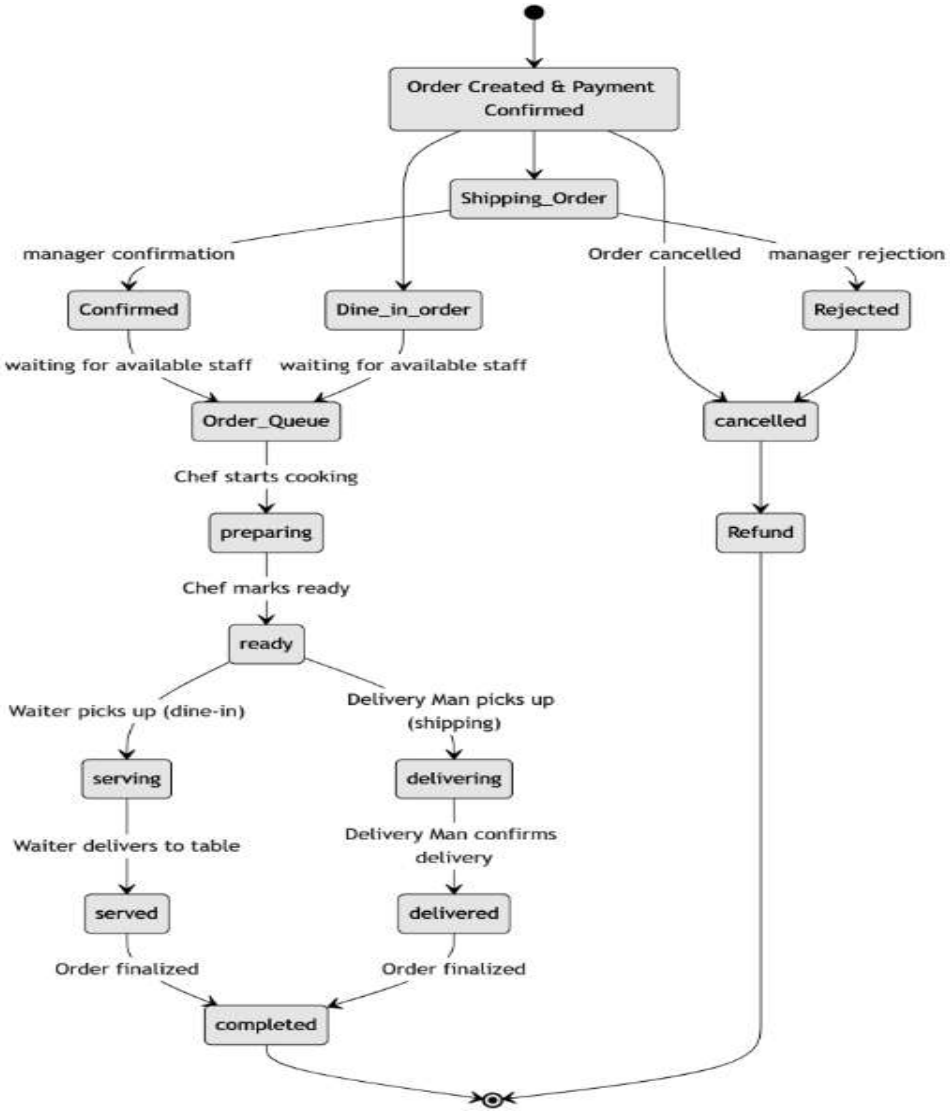


Figure 3.5: State Diagram

3.3.6.3. Data Flow

The layout of the system's information flow is broken down into five distinct stages: Input, API Layer, Services, Storage, and Output, in which each stage is recognized by its own logical boundary that has to be overcome by data on its way to the final destination. The Input stage is where three different user interfaces are realized: Customer UI, Staff UI, and Manager UI. All of them are implemented in React SPA and facilitate the flow of the primary data into the system, including order and cart data, operations done by staff, and administrative commands. The data is then sent to the API layer, where FastAPI Router serves as the only entry point to all requests and starts the process of authentication by transferring the traffic to the JWT Guard to check the token before the protected operation can start. Once all requests have been checked for accuracy, they will be channeled to the appropriate unit in the Services layer. Here, all requests related to orders are managed by Order Service, while requests related to payments are handled by the Payment Service, and assignment actions of the personnel are made by the Assignment Service, which interacts with the Order Queue to ensure that orders are sent out on a first-in-first-out basis. The processed information is then moved to the Storage section, where the Orders Database/Bases and Staff Database/Bases store records of all transactions and data on the availability of the personnel respectively. Finally, in the last stage,

all information in the Orders Database/Bases is gathered in a report called Revenue Reports, which periodically appears in order to comply with the requirements of business analytics.

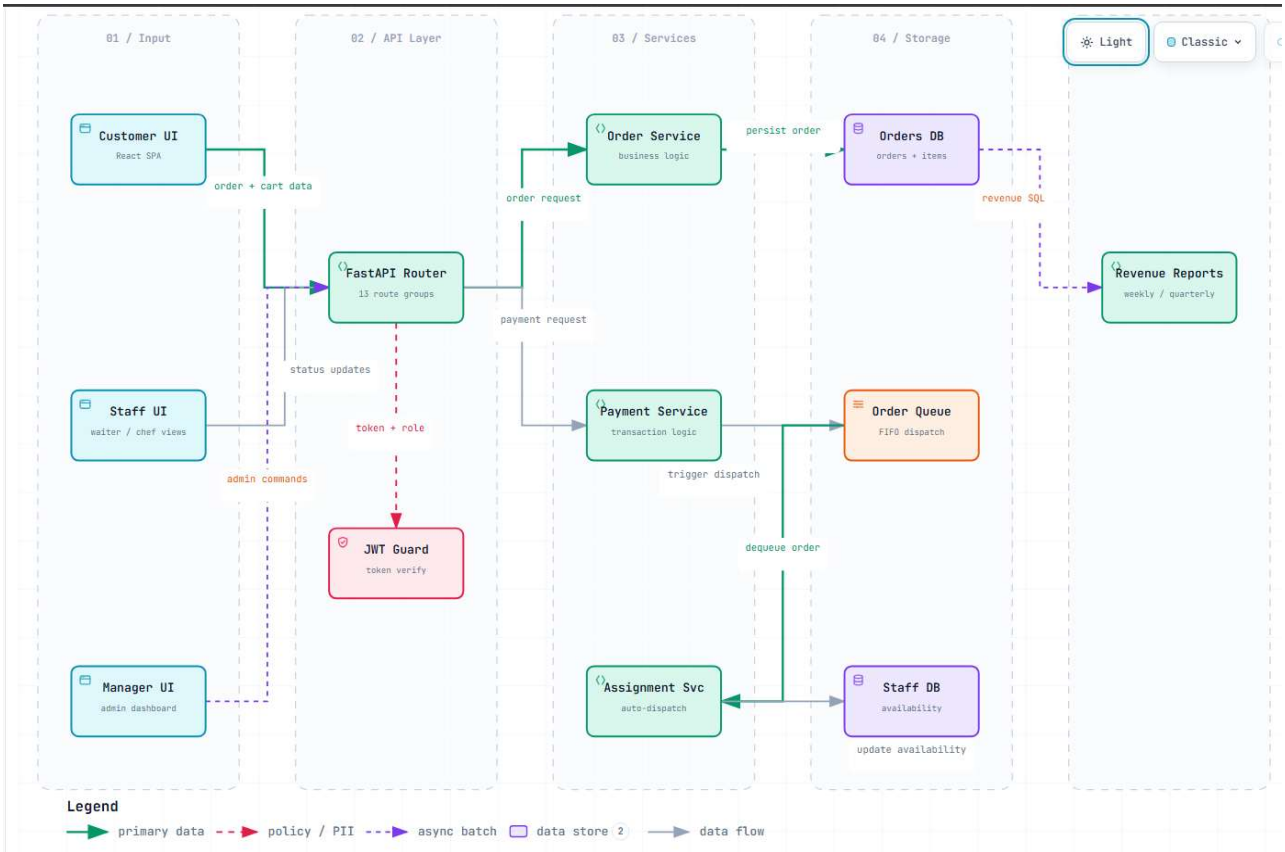


Figure 3.6 System Data Flow

3.4. Communication Mechanisms

- **CORS (Cross-Origin Resource Sharing)**

FastAPI configures CORS middle ware in main.py to explicitly allow requests from front-end origins since front-end runs a different port from back-end and by default browser blocks cross-origin request. This ensure that on launching product only configured ports are allow to communicate with back-end server, which enhance security for database.

- **Axios JWT interceptors**

The client-side has an Axios object that is configured within the file called axios.js. The Axios object comes with a configuration that includes a request interceptor. Whenever any request is being made, the JWT token is automatically added to the request's Authorization header using the bearer format. There is no need for each of the pages to manually add the authorization token to the header.

- **Pydantic DTO Validation**

Every incoming request body is automatically validated by FastAPI against a Pydantic DTO schema before the controller logic executes. If the incoming JSON fails validation such as missing field or incorrect data type, FastAPI will immediately returns an error code 422 response with detailed error messages and the request never reach service or repository layers.

3.5. Database

3.5.1. Database Design

Database design is defined as the relational persistence scheme implemented on the PostgreSQL environment where the restaurant schema is built around. Highly normalized tables are established in the form of 16 persistent tables with 22 relationships, which are linked together with the help of primary/foreign key constraints and dynamic indexes in order to maintain integrity of data during transactions which include high level of concurrency of order processing. Database Design guarantees ACID compliance for all the critical transactional events that may take place and at the same time it permits execution of analytical queries.

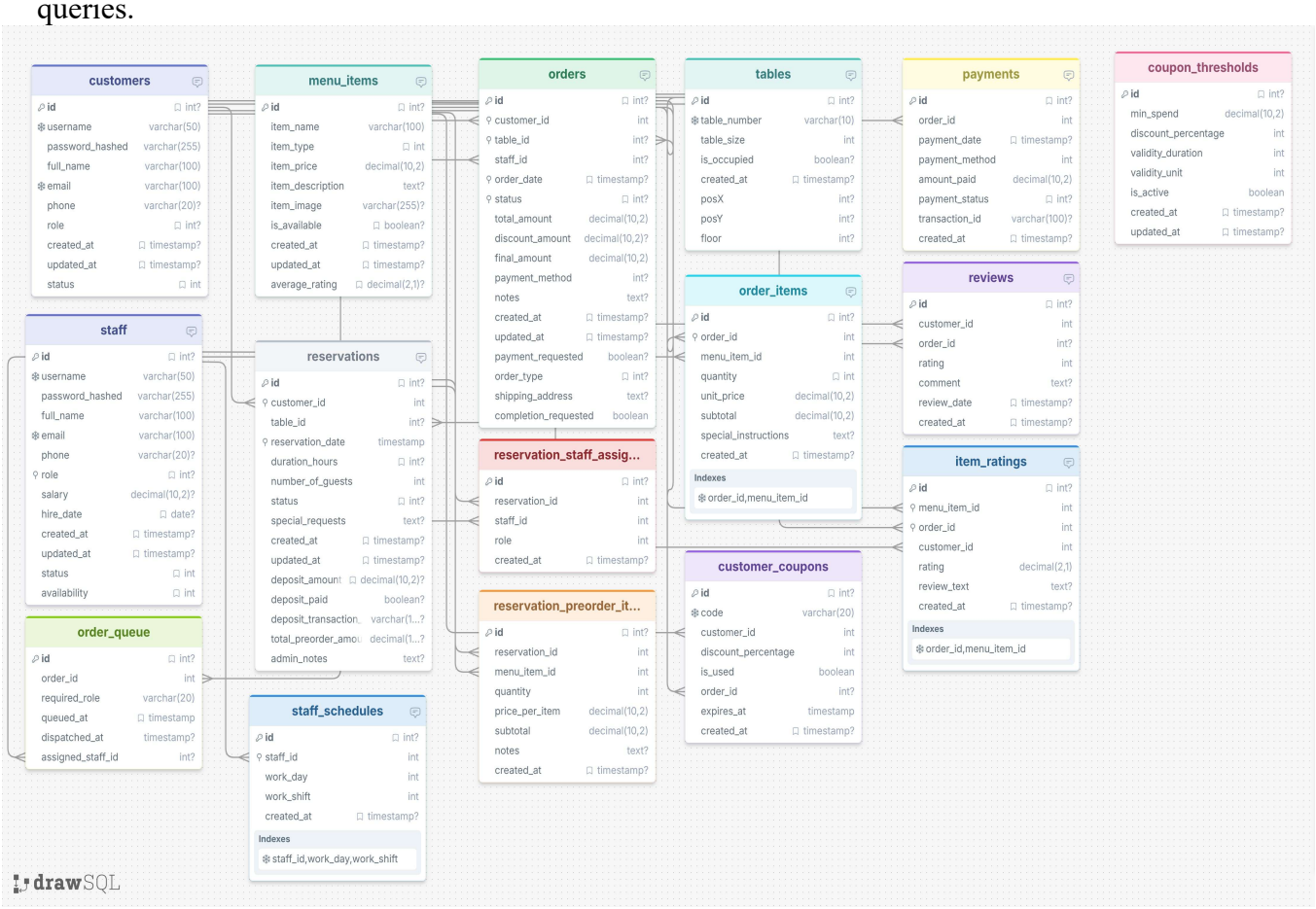


Figure 3.7 Database Design

3.5.2. Database Relationship

In the context of relational database design, there is a terminology referred to as "Relationship," which refers to how two or more tables are connected logically through primary and foreign keys. Instead of gathering data in just one huge table, relational databases split information into different entities, where each entity represents either a real-world object or a concept. This is the basis of the theory of relational databases and differentiates good normalized designs from flat data structures. Depending on how often records of one table correspond to records of another table, relationships are classified into three categories: one-to-one relationships, one-to-many relationships, and many-to-many relationships.

In terms of the database design for this thesis, various relationships should be taken into account, since the Restaurant Management System consists of a number of related entities, and its information

has to be coherent, lightweight, and clear. Specifically, it would not be possible to meaningfully exist without relating to the customer placing the order, the staff member performing it, and the various menu items being ordered. Consequently, it is important to build relationships among the tables that would save from having to either duplicate the same information in multiple tables or come up with ways of storing it without making it unreliable for updates and queries. Define the foreign keys of all of the relationships properly since referential integrity requires having a record of one table (ex. payment) relate to an existing record in the second table (ex. that of the order). Moreover, relationships allow for easy and precise data extraction using SQL joins, permitting the software to reproduce the whole information about events such as details of the order, payment status, and team assigned by combining data from several interconnected databases instead of using only one overloaded structure, In this way, relationships are not only a technological convenience but a basic condition for achieving data precision, applying complex business logic (e.g. team dispatching, order life cycle control), and ensuring trustworthy scaling.

Table	Related Table	Relationship
customers	orders	One-to-Many (1:N)
customers	reservations	One-to-Many (1:N)
customers	reviews	One-to-Many (1:N)
customers	vip_requests	One-to-Many (1:N)
customers	item_ratings	One-to-Many (1:N)
customers	customer_coupons	One-to-Many (1:N)
staff	orders	One-to-Many (1:N)
staff	staff_schedules	One-to-Many (1:N)
staff	order_queue	One-to-Many (1:N)
staff	reservation_staff_assignments	One-to-Many (1:N)
tables	orders	One-to-Many (1:N)
tables	reservations	One-to-Many (1:N)
orders	order_items	One-to-Many (1:N)
orders	payments	One-to-Many (1:N)
orders	order_queue	One-to-One (1:1)
orders	reviews	One-to-Many (1:N)
orders	item_ratings	One-to-Many (1:N)
orders	customer_coupons	One-to-Many (1:N)
menu_items	order_items	One-to-Many (1:N)
menu_items	item_ratings	One-to-Many (1:N)
menu_items	reservation_preorder_items	One-to-Many (1:N)
reservations	reservation_staff_assignments	One-to-Many (1:N)
reservations	reservation_preorder_items	One-to-Many (1:N)

Table 3.6.2: Entities Relationship

3.6. Data Structure

In this thesis, fundamental data structures such as Lists, Queues, and Hash Tables are implemented to optimize data handling across all layers from the front-end to back-end, and database.

- **List (Order items, Menu Catalogs)**

List is class of collections that help represent menu collections, order items (OrderItem) and roles permission in list respectively. They are stored as dynamic arrays (in JavaScript in client-side and Python in the server-side) in order to retrieve them and insert new items into them in efficient manner at O(1).
At the back-end, SQLAlchemy is utilized to establish relationship between Order and OrderItem, where order items are put together as an array of objects allowing batch validation, price summation, and automatic relational insertion.

- **Queue (Order assignment & Staff Dispatching)**

The Queue is used when the kitchen preparation and staff dispatch face demand higher than the capacity available (e.g. all chefs or delivery drivers are busy). Moreover, it helps prevent the conflict between processes and guarantee the fairness of operations by processing orders in the ascending order_Id (first-in-first-out).
Instead of a volatile in-memory queue, the queue's data is stored in the persistent database (OrderQueue model including order_id, required_role, created_at timestamp). The orders yet to be assigned are saved as waiting and the Assignment Service subsequently dequeues the first entry in the order queue whenever the staff with appropriate qualification is available.

- **Hash tables**

The key-value mappings are used for the encoding/decoding of JWT payload, preserving the state of parameters and enabling the O(1) look-up during the session authorization as well.
The utility create_access_token serializes claims (subject, role, expiry date) to the hash map before signing it using cryptographic methods to ensure the immediate O(1) permission check in any secure route.

3.7. Algorithms and Process Logic

3.7.1. FIFO Staff Dispatching Algorithm

- Mechanism: Upon completion of the task by the personnel of the organization, the order_queue is scanned automatically by the system. The algorithm quickly checks the list of remaining orders in the order of receipt of requests and finds out if personnel members are available for jobs (chefs, waiters, delivery people). It assigns the freed staff to the next available order updating staff status to "busy" and removing the corresponding entry from the queue.
- Complexity:
- Time Complexity: O(N) through indexed database look ups find_available_by_role, where N is number of pending order
- Space Complexity: O(N) auxiliary space required to hold pending queue entries.
- Effect on system: Eliminates the necessity to coordinate staff activities manually, prevents delays in processing some orders due to the starvation of orders, and speeds up the cooking process during rush hours.

3.7.2. Finite State Machine (FSM) Order Lifecycle Algorithm

- Mechanism: The advancement of any particular order is governed by a certain set of deterministic transitions (pending → paid → preparing → ready → served/delivered → completed). Each transition is first checked to verify whether its requirements related to pre-conditions are met (for example, a payment confirms that a kitchen order can be assigned (or

that delivery can be made and the order marked complete)) and then starts the resulting side effects (for instance, makes the staff available or frees the table).

- Complexity: $\frac{O(1)Time}{O(1)Space}$ for each transition.
- Effect on system: Guarantees the integrity of data by eliminating the possibility of skipping necessary states.

3.7.3. Bcrypt Adaptive Key-Derivation Cryptographic Algorithm:

- Mechanism: Password security relies on the use of bcrypt salted hashing, with a parameter that can be set for one of its components (cost factor=12).
- Complexity: $O(2^C \cdot n)$ where C is configurable cost factor and n is input password length
- Effect on system: Creates artificial delay in verification, preventing any brute force or rainbow-table attacks on passwords.

3.8. UI/UX Design

- **Layout Architecture**

The user interface has an advanced, flexible design that applies advanced web development techniques and technologies, including CSS Grid, Flexbox, and Tailwind CSS.
The interface of the Customer Portal is built on the principle of visual navigation due to a grid of dish cards organized by category. They have an image preview and Add to Cart buttons. Besides, there is a sliding drawer bar that facilitates checkout.
On the Staff and Kitchen Workstation (KDS) level, kanban-style boards are devised with an organizational structure depending on the order status – Preparing, Ready, and Delivering. The kitchen staff, as well as waiters, can update the order status through user-friendly cards
In the case of the Manager Administration Dashboard, sticky sidebar menus are created to interact with data tables and summary cards showing revenue, active orders, or the status of staff, accompanied by modal forms meant for editing menu items and operating the inventory.

- **Color Palette and Visual Hierarchy**

The graphical facets of the platform incorporate a tailored contemporary color scheme that abides by the 60-30-10 design principles while fulfilling the requirements of the WCAG 2.1 accessibility guidelines.
The participants' background and surfaces take up about 60 % of the visual elaboration while employing the colors of deep slate and neutral shades in dark mode and bright neutral color in light mode, specifically aimed at minimizing the strain on the eyes when serving in restaurants' areas of high contrast where the workers are using screens.

Around 30% of the UI is represented by structural and card details. A very delicate glass morphism effect is utilized in the design of cards providing soft and plain semi-transparent card borders, ensuring that the users perceive visual hierarchy without needing strong borders and drops.

The remaining 10% is reserved for semantic accent and state colors, which communicate system status at a glance:

- **Emerald Green:** indicates success states, available tables or staff, and completed transactions.

- **Amber / Orange:** represents pending actions, orders currently being prepared, and kitchen warnings.
- **Cyan / Sky Blue:** highlights primary interactive call-to-action elements, active tabs, and customer navigation cues.
- **Rose Red**: flags cancellations, payment errors, and unavailable staff.

CHAPTER 4

IMPLEMENTATION

4.1. Overview

The implementation of the Restaurant Management System involving in the front-end client application, back-end API server, and database layer. Each section will contains the description about the technologies used, the architectural patterns, and provides representative code from the actual code base

4.1.1. Front-end implementation

4.1.1.1. Dependencies

The client-side app is designed as a Single-Page Application (SPA) with the use of React for initial rendering of components, put together and run via Vite that allows for fast Hot Module Replacement (HMR) during development. Routing on the client side is done through React Router DOM where navigation is being done in a smooth way without loading any new pages in a browser. Asynchronous server-side state operations like fetching data, caching, background refetching and mutation management are being performed through TanStack React Query v5 which works on bringing HTTP life cycle into submission to client components. Communication with back-end API is done through the use of Axios with the help of a centralized interceptor that provides automatic inclusion of JWT Bearer tokens from local storage to the Authorization header attached to every outgoing request. Finally, Tailwind CSS is utilized for styling of the application providing a utility first approach to creating successfully working UI without hurdles. In terms of representation of data on the admin analytics dashboard charting library Recharts v3 is employed while the iconography for the application is represented through icon set Lucide React v0.561.

```
  },
  "dependencies": {
    "@tailwindcss/postcss": "^4.1.18",
    "@tanstack/react-query": "^5.90.12",
    "autoprefixer": "^10.4.23",
    "axios": "^1.13.2",
    "lucide-react": "^0.561.0",
    "postcss": "^8.5.6",
    "react": "^19.2.0",
    "react-dom": "^19.2.0",
    "react-router-dom": "^7.10.1",
    "recharts": "^3.10.1",
    "tailwindcss": "^4.1.18"
  },
  "devDependencies": {
    "@eslint/js": "^9.39.1",
    "@types/react": "^19.2.5",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^5.1.1",
    "eslint": "^9.39.1",
    "eslint-plugin-react-hooks": "^7.0.1",
    "eslint-plugin-react-refresh": "^0.4.24",
    "globals": "^16.5.0",
    "vite": "^7.2.4"
  }
}
```

Figure 4.1 front-end dependency declarations

4.1.1.2. Structure and Organization

The front-end source code adheres to a feature-based directory organization within FE/src/, structured as follows:

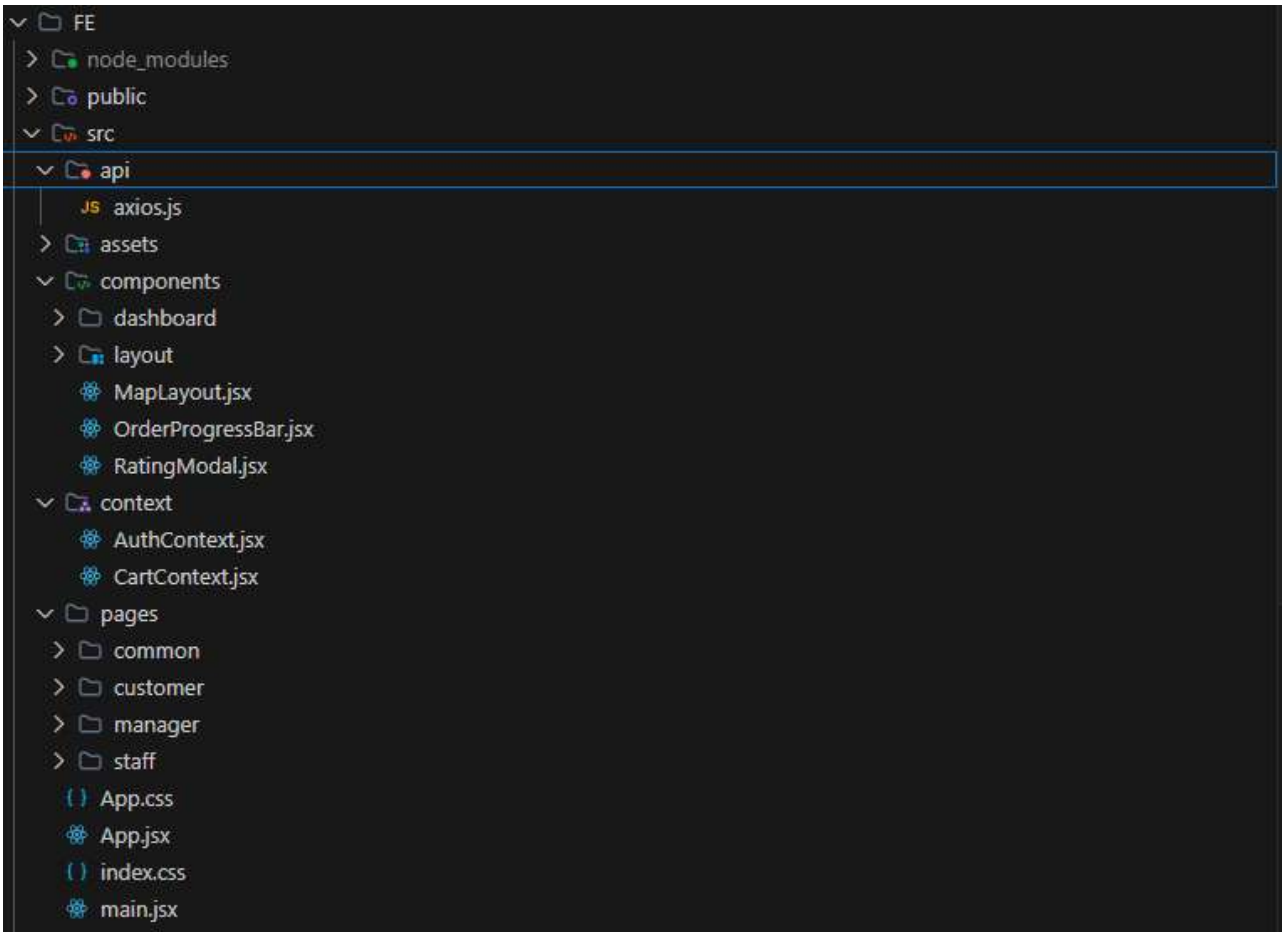


Figure 4.2: Front-end structure

The pages/ directory is divided according to actor roles (Customers, Delivery personnel, Waiter, Chef, and Manager). Each page is constructed to implement the respective logic for the given role by containing all relevant queries and event handlers. This approach promotes maintainability because it separates the logic for each user role from other parts of the application.

4.1.1.3. State Management Architecture

The global state application is controlled with the use of two React Context providers which are set up in the root component.

- AuthContext is responsible for the whole process of authentication such as submitting credentials, keeping the JWT in local storage, hydrating the user's session, propagating various states depending on the user's role, and finishing with logout and deactivation on the server's side. The user information, the login/logout feature, and loading state are available for every child component through the useAuth custom hook.
- The CartContext is responsible for customer's shopping cart represented as an array of items. The CartContext will manage the whole shopping cart process from adding and removing items, changing their number in the cart, and getting totals to the customer. In addition, the context will persist between the navigation.

The information derived from the server (menus, orders, staff, reservations) is obtained solely with the help of TanStack React Query useQuery and useMutation hooks that perform processing of caching, background process synchronization, and optimistic updates on their own regardless of component state.

```
export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        const initializeAuth = () => {
            const token = localStorage.getItem('token');
            const role = localStorage.getItem('role');
            const userId = localStorage.getItem('userId');
            const fullName = localStorage.getItem('full_name');

            if (token) {
                setUser({ role, id: userId, full_name: fullName });
            }
            setLoading(false);
        };

        initializeAuth();
    }, []);

    const login = async (username, password) => {
        try {
            const response = await api.post('/auth/login', { username, password });

            console.log("Login response:", response.data);
            const { access_token, role, user_id, full_name } = response.data;

            localStorage.setItem('token', access_token);
            localStorage.setItem('role', role);
            localStorage.setItem('userId', user_id);
            localStorage.setItem('full_name', full_name);

            setUser({ role, id: user_id, full_name: full_name });
            return { success: true, role };
        } catch (error) {
            console.error("Login failed:", error);
            const msg = error.response?.data?.detail || "Login failed. Please check your credentials.";
            return { success: false, message: msg };
        }
    };
```

Figure 4.3: Login function and token management

4.1.1.4. Routing and Navigation Architecture

The application uses React Router v7 with nested route layouts to enforce role-based page access. Three layout components serve as route wrappers:

Layout	Route Prefix	Pages	Target Role
CustomerLayout	/	Menu, Cart, Tables, Orders, Shipping, QR	Customer

		Payment	
StaffLayout	/staff/*	Dashboard, Table Selection, Order Entry, Active Orders, Schedule, Delivery	Waiter, Chef, Delivery Man
ManagerLayout	/Manager/*	Dashboard, Orders, Menu, Staff, Tables, Payments, Customers, Schedule, Reservation, Coupon, Reviews	Manager

Table 4.1.1.4 Front-end routes

4.1.1.5. Authenticated Data Fetching

The application manages two categories of state using different strategies:

- Server State is managed by TanStack React Query. Every API call is wrapped by using useQuery or useMutation hook, which provides automatic caching, background refetching, loading/error states, and cache invalidation after mutations:

```
// ChefDashboard.jsx – Auto-refreshing server state
const { data: orders = [], isLoading } = useQuery({
    queryKey: ['orders'],
    queryFn: async () => {
        const res = await api.get('/orders/');
        return res.data;
    },
    refetchInterval: 10000  // Auto-refresh every 10 seconds
});

// Mutation with cache invalidation
const updateStatusMutation = useMutation({
    mutationFn: async ({ orderId, status }) => {
        return api.put(`/orders/${orderId}`, { status });
    },
    onSuccess: () => {
        queryClient.invalidateQueries(['orders']);  // Refetch after update
    }
});
```

Figure 4.4 Auto-refreshing server state

All requests made to APIs use one Axios client instance defined in the api/axios.js file. The Axios request interceptors add JWT token stored in local storage to the authorization header of all outgoing requests. This way, all components simply call functions like api.get() and api.post() without managing authentication header.

```
const api = axios.create({
    baseURL: 'http://localhost:8000',
});

api.interceptors.request.use((config) => {
    const token = localStorage.getItem('token');
    if (token) {
        config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
});

export default api;
```

Figure 4.5 Authentication Interceptor implementation

4.1.2. Back-end implementation

4.1.2.1. Dependencies

The server-side part of the application is developed using FastAPI, which is a high-speed asynchronous web framework for Python. The instance of the application is hosted on the ASGI server Uvicorn, which operates on port 8000. The middleware Cross-Origin Resource Sharing (CORS) is set to allow requests from the origins of the Vite development server (localhost:5174, localhost:5173, localhost:3000), allowing every HTTP method and header used for the smoothest API working along with the SPA client.

```
# Configure CORS
origins = [
    "http://localhost:5173",
    "http://localhost:5174",
    "http://localhost:3000",

]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.mount("/images", StaticFiles(directory=UPLOAD_DIR), name="images")

# include routers
app.include_router(customer.router,prefix="/customers",tags=["Customers"])
app.include_router(staff.router,prefix="/staff",tags=["Staff"])
app.include_router(menu_item.router,prefix="/menu-items",tags=["Menu Items"])
app.include_router(order_item.router,prefix="/order-items",tags=["Order Items"])
app.include_router(order.router,prefix="/orders",tags=["Orders"])
app.include_router(table.router,prefix="/tables",tags=["Tables"])
app.include_router(payment.router,prefix="/payments",tags=["Payments"])
app.include_router(staff_schedule.router,prefix="/staff-schedules",tags=["Staff Schedules"])
app.include_router(auth.router,prefix="/auth",tags=["Authentication"])
app.include_router(upload.router,prefix="/upload",tags=["Upload"])
app.include_router(item_rating.router,prefix="/item-ratings",tags=["Item Ratings"])
app.include_router(coupon.router,prefix="/coupons",tags=["Coupons"])
app.include_router(reservation.router,prefix="/reservations",tags=["Reservations"])
```

```
# startup event
@app.on_event("startup")
def test_db_connection():
    # Create tables if they don't exist
    Base.metadata.create_all(bind=engine)

    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
        print(" Database connection successful")
    except Exception as e:
        print(" Database connection failed:", e)
```

Figure 4.6 FastAPI initialization, CORS, and router

The back-end uses the following key Python packages (managed via requirements.txt):

Package	**Purpose**
Fastapi	Web framework with automatic OpenAPI

	documentation
Uvicorn	ASGI server for running the FastAPI application
SQLAlchemy	ORM for database modeling and query building
psycopg2	PostgreSQL database driver (C-based adapter)
pydantic	Data validation and serialization for DTOs
python-jose	JWT token encoding and decoding (HS256)
bcrypt	Password hashing with configurable salt rounds
python-dotenv	Environment variable loading from .env files
email-validator	Email format validation for registration DTOs
starlette	Email format validation for registration DTOs

Table 4.1.2.1 Back-end dependencies

4.1.2.2. Layer Architecture and Modules Organization

The back-end codebase is structured according to a strict Controller > Service > Repository layering design, located at BE/app/ and comprises:

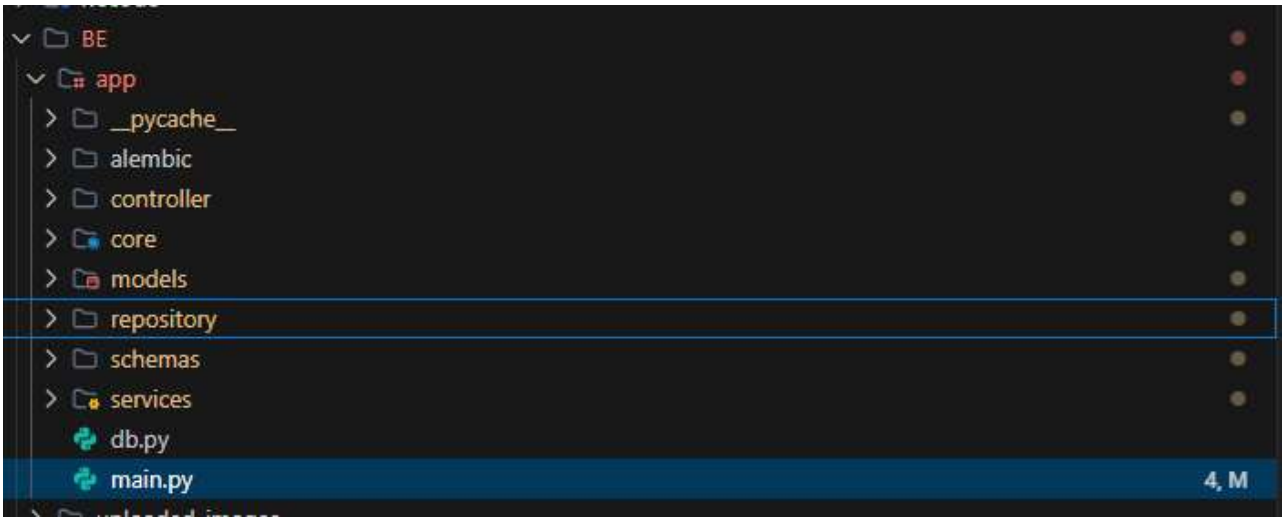


Figure 4.7 Back-end structure

All components of a functional area (for instance, orders, payments, reservations) are realized in all four layers, which means that there is a Controller responsible for HTTP endpoint definitions and request/response schemas; Service is responsible for implementing business logic, validation, and cross-entity coordination; Repository is responsible for making raw SQLAlchemy database session queries. Thanks to this segregation of duties it is possible to change interface agreements, business logic or query improvements independently.

Since the back-end follows a strict three-layer architecture therefore each layer has a single responsibility:

- Controller Layer (controller/) - Includes FastAPI route functions that get HTTP requests, pass them to the service layer, and send back HTTP responses. The controllers are responsible for request validation (Pydantic DTOs), HTTP status codes, and errors:
- Service Layer (services/) - Implements business logic like order subtotal calculation, managing table capacity, authenticating passwords, and computing payment amounts. The services work with more than one repository if necessary:
- Repository Layer (repository/) - Represents all the raw SQL queries using the SQLAlchemy ORM. Each repository class has CRUD functions and custom query functions:

All incoming request data is validated by using Pydantic Data Transfer Objects (DTOs) defined in schemas layer. Each DTO specifies field types, constraints, and optional defaults. Response DTOs use from_attributes = True to enable automatic ORM-to-JSON serialization:

```python
class PaymentCreateDTO(BaseModel):
    order_id: int = Field(..., gt=0)          # Required, must be > 0
    payment_method: PaymentMethod             # Must be a valid enum value

class PaymentResponse(BaseModel):
    id: int
    order_id: int
    payment_date: datetime
    payment_method: PaymentMethod
    amount_paid: float
    payment_status: PaymentStatus
    transaction_id: Optional[str]
    created_at: datetime

    class Config:
        from_attributes = True  # Enables reading from ORM objects
```

Figure 4.8 Request/Response DTOs

Each business module (e.g., Order, Payment, Customer) is represented by one file in each layer, following a consistent naming convention. The following table illustrates how the Order module is mapped across all layers:

Layer	**File**	**Class / Function**	**Responsibility**
Controller	controller/order.py	router (APIRouter)	HTTP endpoints: GET /orders/, POST /orders/, PUT /orders/{id}, etc.
Schema	schemas/ orderDTO.py	OrderCreateDTO, OrderResponseDTO	Request validation + response serialization
Service	services/order.py	OrderService	Business logic: table occupancy, total

			recalculation
Repository	repository/ orderRepository.py	OrderRepository	SQLAlchemy queries: CRUD, status filtering, total aggregation
Model	models/orderly	Order, OrderStatus, OrderType	ORM class + ENUM definitions

Table 4.1.2.2 Back-end layers for module Order

4.1.2.3. Business Logic

4.1.2.3.1. Authentication and Authorization Implementation

Authentication is implemented using stateless JWT tokens with bcrypt password hashing: Password Hashing : The core/security.py module implies hashing and verifying password functions using the bcrypt library with default 12 salt rounds:

```
def hash_password(password: str) -> str:
    pwd_bytes = password.encode("utf-8")
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(pwd_bytes, salt)
    return hashed.decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    plain_bytes = plain_password.encode("utf-8")
    hashed_bytes = hashed_password.encode("utf-8")
    return bcrypt.checkpw(plain_bytes, hashed_bytes)
```

Figure 4.9 Password hashing service

JWT Token Generation : The create_access_token() function encodes user identity (sub), user_type (staff/customer), and role into a JWT with a 60-minute expiration using the HS256 algorithm via python-jose:

```
SECRET_KEY = os.getenv("SECRET_KEY")  # Load from environment
ALGORITHM = "HS256"

def create_access_token(data: dict, expires_minutes: int = 60):
    to_encode = data.copy()
    to_encode["exp"] = datetime.utcnow() + timedelta(minutes=expires_minutes)
    ▷ Generate Mermaid Diagram | ● Open Chat @mermaid-chart
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

Figure 4.10 JWT token generation

Login Flow: The AuthService.login() method first checks the Staff table, then the Customer table. If a match is found and the password is verified, a JWT token is returned along with the user's ID and role:



```
class AuthService:

    def login(self, db, username: str, password: str):
        # — Staff login ——————————————————————————————
        staff = StaffRepository().find_by_username(db, username)
        if staff and verify_password(password, staff.password_hashed):
            if staff.status != staff_status.active:
                staff.status = staff_status.active
                db.add(staff)
                db.commit()
                db.refresh(staff)
            self.assignment_service.dispatch_queue(db)
            token = create_access_token({
                "sub": staff.id,
                "user_type": "staff",
                "role": staff.role,
                "full_name": staff.full_name
            })
            return {
                "access_token": token,
                "user_id": staff.id,
                "role": staff.role,
                "full_name": staff.full_name
            }

        # — Customer login ———————————————————————————
        customer = CustomerRepository().find_by_username(db, username)
        if customer and verify_password(password, customer.password_hashed):
            if customer.status != customer_status.active:
                customer.status = customer_status.active
                db.add(customer)
                db.commit()
                db.refresh(customer)
            token = create_access_token({
                "sub": customer.id,
                "user_type": "customer",
                "role": customer.role,
                "full_name": customer.full_name
            })
            return {
                "access_token": token,
                "user_id": customer.id,
                "role": customer.role,
                "full_name": customer.full_name
            }
```

Figure 4.11 Authorization service implement

4.1.2.3.2. FIFO Staff Dispatching Algorithm implementation

To optimize kitchen throughput and floor service, the backend features an automated, event-driven staff assignment engine backed by concurrency row locks (lock=True) and a persistent order queue (OrderQueue). Upon payment confirmation or manager approval, the system searches for an available chef; if one is found, the chef is assigned and flagged as unavailable (staff_availability.unavailable), advancing the order to preparing. If all chefs are occupied, the order is enqueued in a FIFO queue. When a chef flags an order as ready, the system automatically shifts context: dine-in orders are dispatched to an available waiter, whereas delivery orders are handed off to an available delivery person. Crucially, whenever any staff member completes a task and is freed, the system immediately invokes a queue dispatch scan (dispatch_queue) to automatically allocate waiting orders to the newly available worker.

```python
def try_assign_chef(self, db: Session, order: Order) -> Order:
    """
    Try to assign an available active chef to the order and change availability to unavailable.
    If none is free, put the order in the queue with required_role='chef'.
    """
    chef = self.staff_repo.find_available_by_role(db, StaffRole.chef, lock=True)
    if chef:
        order.staff_id = chef.id
        order.status = OrderStatus.preparing
        chef.availability = staff_availability.unavailable
        # Clear queue entry if the order was previously queued
        self.queue_repo.delete_by_order_id(db, order.id, commit=False)
        db.add(order)
        db.add(chef)
        db.commit()
        db.refresh(order)
    else:
        # Keep current status — order_queue tracks the waiting state
        db.add(order)
        db.commit()
        db.refresh(order)
        self.queue_repo.enqueue(db, order.id, required_role="chef")      You, 4 months ago • optimize
    return order
```

Figure 4.12 FIFO Staff Dispatching implementation

4.1.2.3.3. Finite State Machine Algorithm implementation

The system enforces a deterministic Finite State Machine (FSM) to govern order progression while distinguishing between dine-in and online shipping workflows. When a customer initiates a dine-in order, the system validates table availability and immediately flags the designated table as occupied (is_occupied = True). For shipping orders, the system records the delivery address and routes the order into a pending_confirmation holding state until a manager reviews and validates the order. State transitions are strictly validated against an adjacency matrix—preventing invalid state jumps (e.g., skipping from pending straight to completed)—and terminal transitions systematically release system resources, such as resetting table occupancy to unoccupied once the dine-in order status resolves to completed.

```python
        return created_order

    VALID_TRANSITIONS: dict[OrderStatus, set[OrderStatus]] = {
    OrderStatus.pending: {OrderStatus.paid, OrderStatus.pending_confirmation, OrderStatus.cancelled},
    OrderStatus.paid: {OrderStatus.queued, OrderStatus.preparing, OrderStatus.cancelled},
    OrderStatus.pending_confirmation: {OrderStatus.confirmed, OrderStatus.cancelled},
    OrderStatus.confirmed: {OrderStatus.queued, OrderStatus.preparing, OrderStatus.cancelled},
    OrderStatus.queued: {OrderStatus.preparing, OrderStatus.cancelled},
    OrderStatus.preparing: {OrderStatus.ready, OrderStatus.cancelled},
    OrderStatus.ready: {OrderStatus.served, OrderStatus.delivering, OrderStatus.cancelled},
    OrderStatus.served: {OrderStatus.completed},
    OrderStatus.delivering: {OrderStatus.delivered, OrderStatus.cancelled},
    OrderStatus.delivered: {OrderStatus.completed},
    OrderStatus.completed: set(),  # Terminal state
    OrderStatus.cancelled: set(),  # Terminal state
}

def update(self, db: Session, order_id: int, order_data: OrderUpdateDTO) -> Optional[Order]:
```

Figure 4.13 Finite State Machine Algorithm implementation

4.1.2.3.4. Payment Processing and VietQR Generation

Financial transactions enforce strict consistency by computing order subtotals from live line-item prices, applying validated promotional discounts, and compounding an 8% tax calculation. For digital transactions, the system dynamically constructs a VietQR image payload containing the restaurant's bank BIN, account credentials, the exact payable amount, and an order-specific memo identifier (DH <order_id>).

The automatic completion trigger upon a successful scan and bank transfer is powered by an asynchronous SePay webhook integration (POST /payments/sepay/webhook):

```python
async def receive_sepay_webhook(
    payload: dict,
    authorization: str | None = Header(default=None),
    db: Session = Depends(get_db),
):
    """Validate an incoming SePay transfer and complete its matching payment."""
    if not _sepay_authorized(authorization):
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid webhook credentials")

    transaction_id = str(_sepay_value(payload, "id", "referenceCode") or "").strip()
    transfer_type = str(_sepay_value(payload, "transferType", "type") or "").lower()
    account_number = str(_sepay_value(payload, "accountNumber", "account") or "").strip()
    transfer_amount = _sepay_value(payload, "transferAmount", "amount")
    content = str(_sepay_value(payload, "content", "transactionContent", "description") or "")
    if not transaction_id or transfer_type not in {"in", "credit"}:
        raise HTTPException(status_code=400, detail="Invalid incoming transaction payload")

    config = get_vietqr_config(db)
    if account_number != config["account_no"]:
        raise HTTPException(status_code=400, detail="Transaction account does not match configured account")
    try:
        incoming_amount = Decimal(str(transfer_amount))
    except (InvalidOperation, TypeError):
        raise HTTPException(status_code=400, detail="Invalid transfer amount")

    existing = payment_service.get_by_transaction_id(db, transaction_id)
    if existing:
        if any(payment.payment_status == PaymentStatus.completed for payment in existing):
            return {"accepted": True, "duplicate": True, "payment_status": "completed"}
        raise HTTPException(status_code=409, detail="Transaction ID is already assigned")

    order_match = re.search(r"\bDH\s*(\d+)\b", content, flags=re.IGNORECASE)
    if not order_match:
        raise HTTPException(status_code=400, detail="Transfer reference must contain DH {order_id}")

    payment = db.query(Payment).filter(
        Payment.payment_method == PaymentMethod.QR,
        Payment.payment_status == PaymentStatus.pending,
        Payment.order_id == int(order_match.group(1)),
    ).with_for_update(skip_locked=True).first()
    if not payment:
        raise HTTPException(status_code=404, detail="No matching pending QR payment found")
    if incoming_amount != Decimal(str(payment.amount_paid)):
        raise HTTPException(status_code=400, detail="Transfer amount does not match payment amount")

    payment_service.complete_sepay_payment(db, payment, transaction_id)
    return {"accepted": True, "duplicate": False, "payment_id": payment.id, "payment_status": "completed"}
```

Figure 4.14 Payment Processing implementation

4.1.2.3.5. Table and Reservation

The reservation module coordinates dining space allocation with advance customer commitments. When booking, the system evaluates table capacity and prevents conflicting overlap by cross-checking existing bookings within a designated duration The system tracks

mandatory deposit payments and optional meal pre-orders, linking pre-order line items directly to active menu records.

Administrators can approve or reject reservation requests, assign dedicated waiting staff to specific tables ahead of arrival, and smoothly transition reservations into active dining orders when guests check in.

4.1.2.3.6. Coupon Thresholds

The marketing and retention sub-system features an automated threshold-based reward engine and strict redemption safeguards. Upon every completed order, the system evaluates the customer's total spend against configurable, active promotional tiers; if a spending threshold is met, the system generates a unique, single-use promotional code with an automated expiration date based on configured validity periods (days, weeks, or months). During checkout, coupon application is restricted solely to pending orders, verifying that the coupon belongs to the authenticated customer, has not expired, and has not been previously redeemed (is_used = False). Once applied, the percentage discount is deducted from the order's final amount, and the coupon is bound to the order ID to prevent duplicate redemption.

4.1.3. Database implementation

The physical implementation of the database is performed by using a code-first approach owing to SQLAlchemy serving as the ORM in the FastAPI framework. Instead of writing raw SQL scripts to build tables, enforce constraints, and set up foreign keys, we effectively define each database entity as a Python class, or model, where table columns, data types, and relationships are defined in code via SQLAlchemy's declarative syntax. In this connection, the conceptual and logical designs that were created during the ERD modeling stage in Chapter 3 have their direct equivalent in respective Python model classes since each table, column, and relationship is defined in a declarative way.

When these models are defined, SQLAlchemy automatically creates the necessary SQL commands for building an actual database schema. These commands include statements for creating tables, constraints for both primary and foreign keys, indices, and definitions for data types. Subsequently, it has become unnecessary to manually maintain the SQL commands and it has mitigated the risks of inconsistent application code and database structure. The code-first approach has many positive aspects: the database schema is always aligned with the data models developed in the application since any adjustments applied to the model are immediately embedded in the generated schema; it gives developers an opportunity to work mainly in Python without having to use different programming languages and/or tools; as well as it fits perfectly within the FastAPI framework allowing the developers to query/create/update/validate model instances employing the same object structures without switching between multiple language constructions.

```
metadata = MetaData(schema="restaurant")

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL, echo=False)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

Base = declarative_base(metadata=metadata)
# #create all tables in schema if not exist
```

Figure 4.15 Database schema automated generation

4.2. Results

The Restaurant management system is a web-based application designed to streamline restaurant's core operation in digital platform which enhance convenience for customers and for staff members.

4.2.1. Authentication and Access Control

The system's authentication interface serves as a single point of entry for the users of the system and applies role-based access control to the entire software. The interface includes input fields for the username and password of the user, which are subject to client-side validation to maintain correct format.

After the submission of the form, a request is sent to the back-end authentication controller through the front-end asynchronously. After the server verifies the credentials based on a comparison of the salted bcrypt hashes, it creates the user JWT that contains the identity and the role of the user. The client program keeps the token secure and checks the authorized role parameter and automatically redirects users to the appropriate resource. Customers get redirected to the customers ordering portal, service employees to the Kitchen Display and order processing workstations, and managers to the Manager dashboard. Any attempts of unauthorized access are detected and navigated back to the login screen.

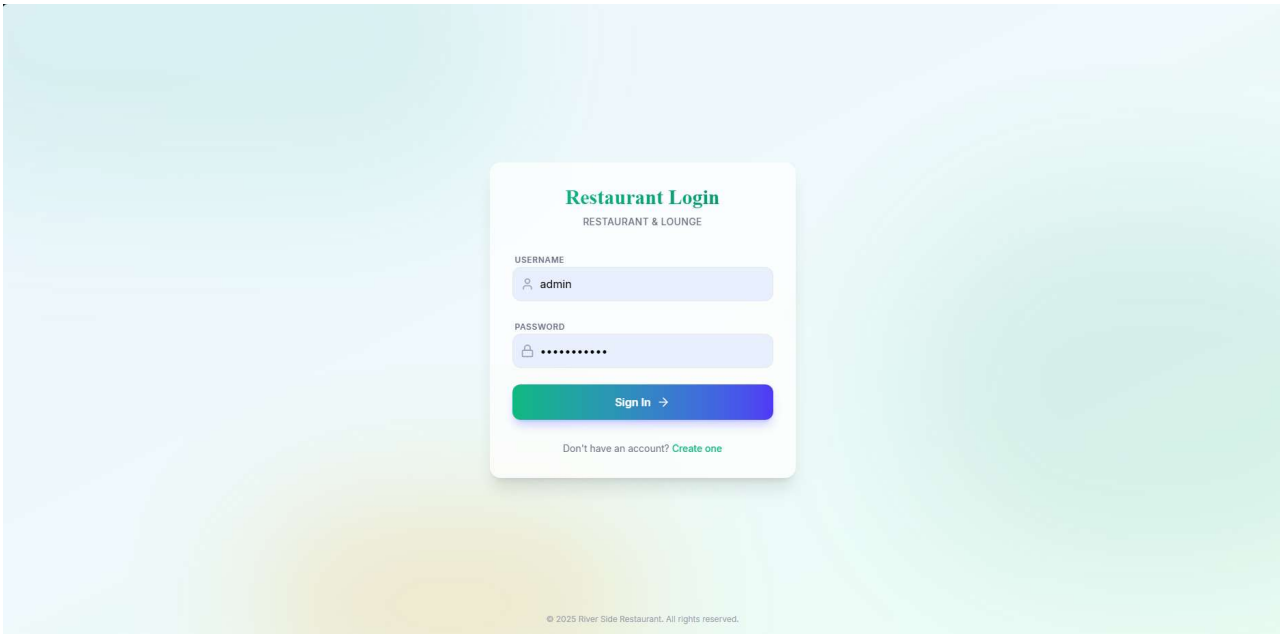


Figure 4.16 Login and Authentication

4.2.2. Customer Ordering and Service Experience Interfaces

The customer interface allows users to conduct their entire dining process from catalog searching, ordering, paying, tracking their order, to feedback review.

- **Menu Exploration and Dish Selection**

The first interface presents a dynamic menu grouped by type of meals; users will see a menu card for each item including images of the dish, prices, descriptions, and availability as well as historical averages of ratings. Users will find the chose item with the help of advanced search and filtering feature. After the item is selected, it will be displayed with complete description regarding its nutritional value, and customers may enter the quantities and cooking notes.

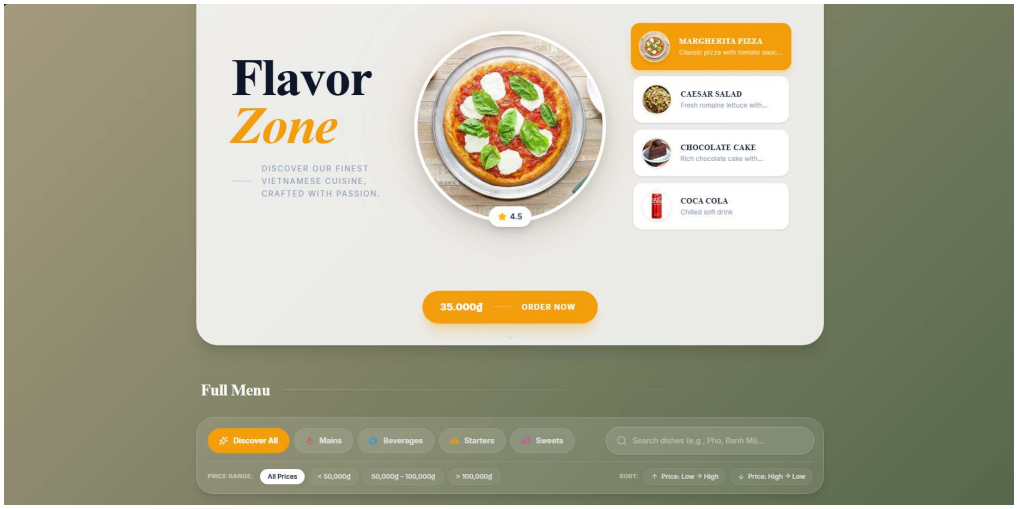


Figure 4.17 Menu Page

- **Cart Management, Discount Application, and Checkout**

The comprehensive ordering procedure contains a slide-over cart drawer that collects the chosen line items, allowing the customer to adjust the quantities and receive instant reconfirmations of their

subtotal. When making a purchase, the client indicates how they would like to receive their order: dine-in (with the option to choose a table) or delivery (with entering the delivery address). Additionally, the customer is capable of using valid promo codes, resulting in the recalculation of the total amount due in real-time. The payment method allows the customers to pay by cash or Quick Response (QR) code.

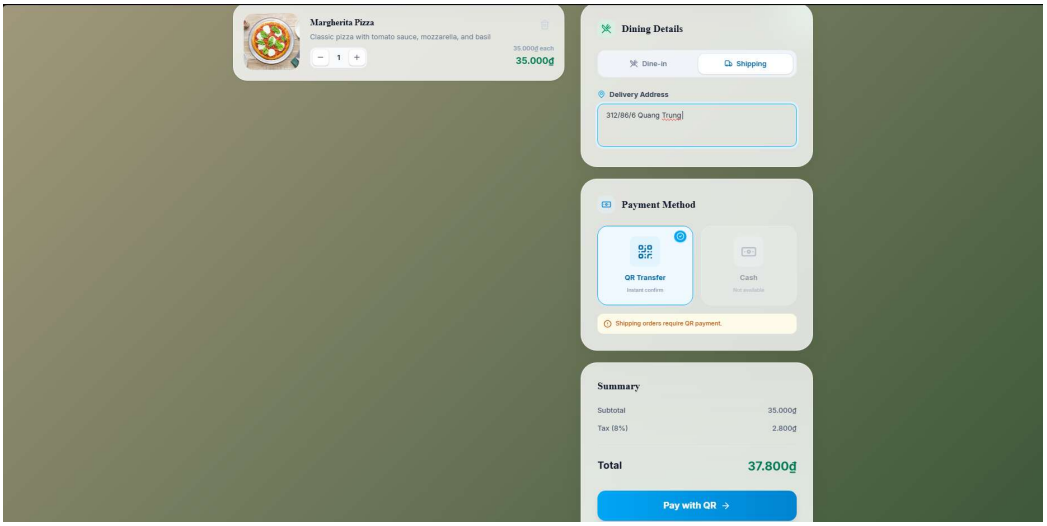


Figure 4.18 Cart Drawer and Checkout Interface

- **Order Tracking and Feedback Submission**

After placing an order, customers can proceed to the specialized Order Tracking screen. This screen shows the current state of the order through different phases (pending, preparing, ready, served/delivering, completed). The moment the order status changes to completed, the system immediately opens a form for rating and reviewing the item. Customers will be able to enter fractional numbers from (1.0 to 5.0) along with qualitative description, and the data will aggregate automatically to update the overall public ratings of menu items.

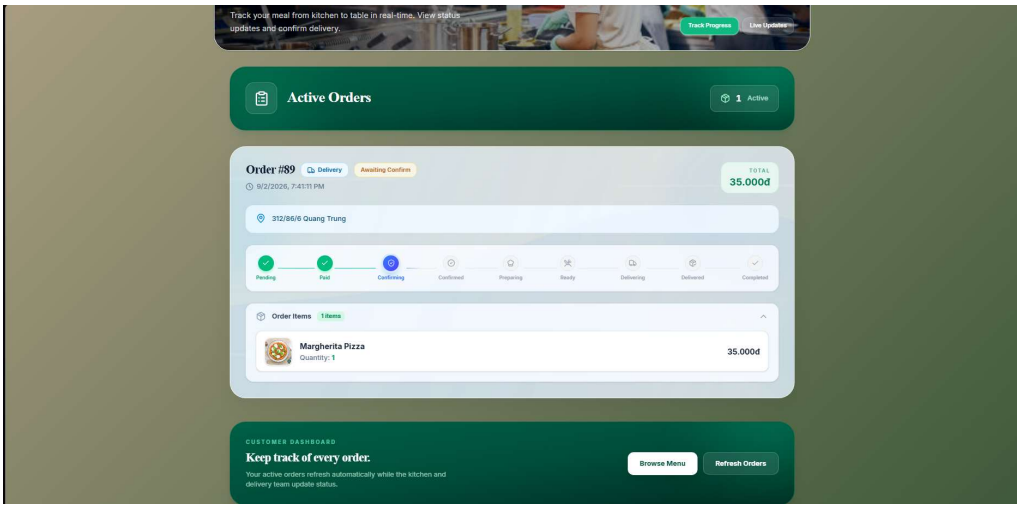


Figure 4.19 Order Tracking and Review Submission Interface

4.2.3. Staff Workstations and Kitchen Display Systems (KDS)

This part of the employee interface component introduces a new design for kitchen ticketing, replacing the traditional paper-based tickets with dynamic operational dashboards based on the board.

- **Kitchen Display System (Chef View)**

The chef's workstation organizes active culinary orders into progressive horizontal lines according to their readiness. The kitchen tickets make it clear which order each order corresponds to, have timers showing how long the order has been getting prepared, indicate tables, contain line items to be prepared, as well as provide specific instructions for cooking them. Chefs make use of single click buttons to change the status of the order from “Preparing” to “Ready.”

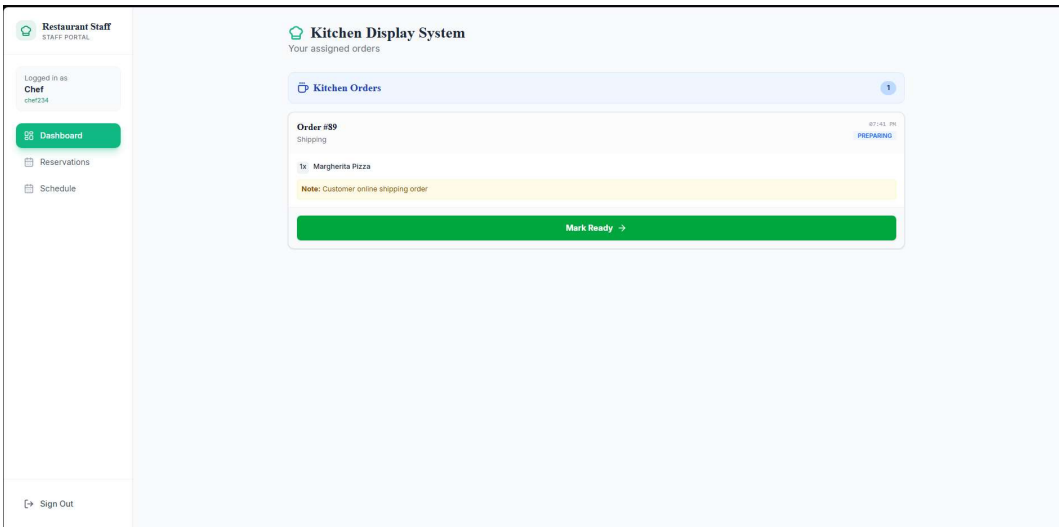


Figure 4.20 Kitchen Display System

- **Service Workers Interface**

The service personnel can utilize their workstation through their respective role-specific interfaces:

- For Waiters Interface: The waiter’s interface consists of all the dine-in orders that are “Ready” in the kitchen. It implies that the dish is prepared and available for delivery to thetable. The waiter has the option to mark the order as “Served” and complete the bill request table.
- For Delivery Processors Interface: The delivery process interface will show the outbound shipment orders. There is information displayed about the shipping mode, customer’s address, contact number, and the status of payment. The courier has the option to change order status from “Delivering” to “Delivered.” This enables the system to share the status of the order with the central database and update the staff availability flags in the system.

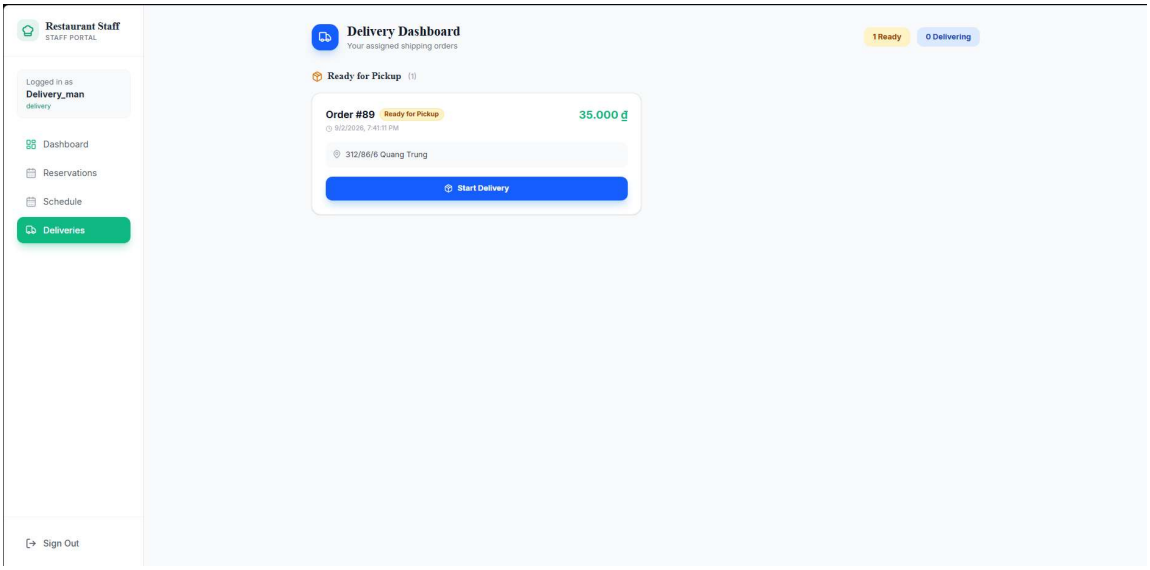


Figure 4.21 Waiter and Delivery Workstation Interface

4.2.4. Manager Dashboard

The operation administration, regulation of personnel, and analytical achievements are combined in one administration dashboard.

- **Executive Analytics and Operational Overview**

The dashboard has a hierarchy of sidebars and a stylish grid of metrics at the entry point. Data on statistics is provided in the cards for daily revenues, number of orders and occupancy rates. There is a feature to analyze historical performance data on a weekly, monthly, and yearly basis.

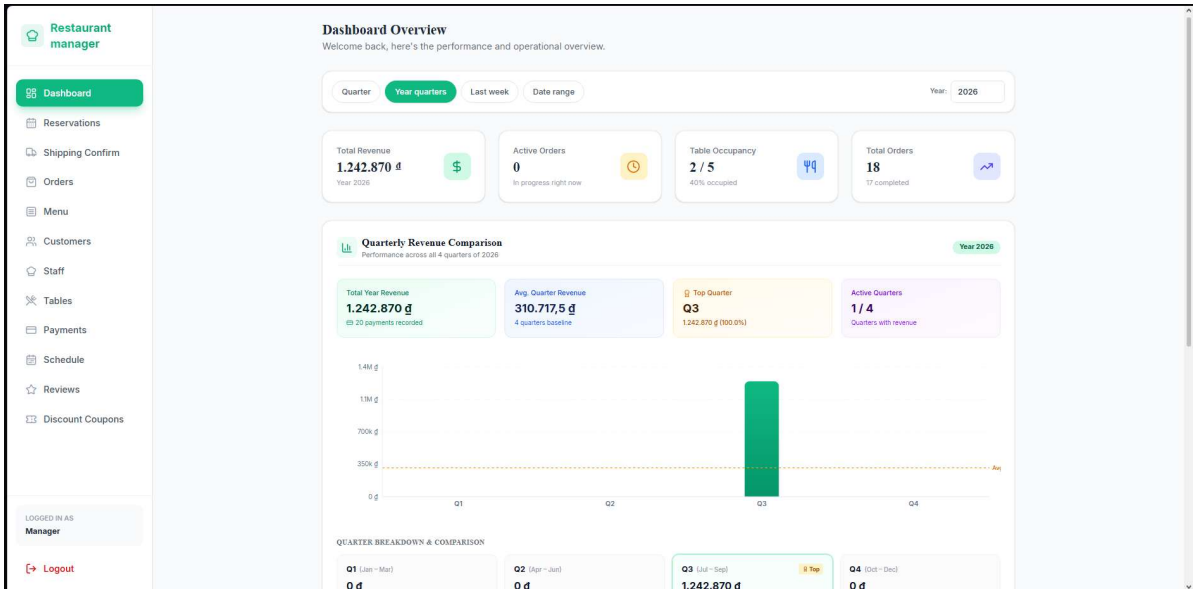


Figure 4.22 Manager Overview and Metric Analytics Interface

- **Data Tables and Entity Management Modals**

The dashboard has efficient layout of tables for data, where every table is equipped with advanced search and filtering options:

- Management of Menu and Pricing: It allows managers to create and change menu items, prices, and availability.

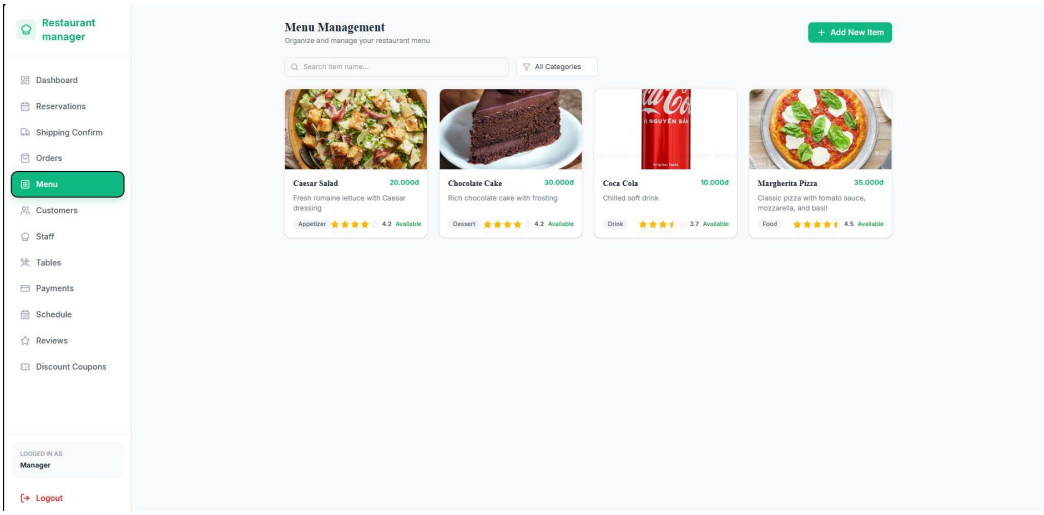


Figure 4.23: Menu Management Interface

- Management of Layouts of Tables: It assists managers in changes of seats, location of tables and occupancy status.

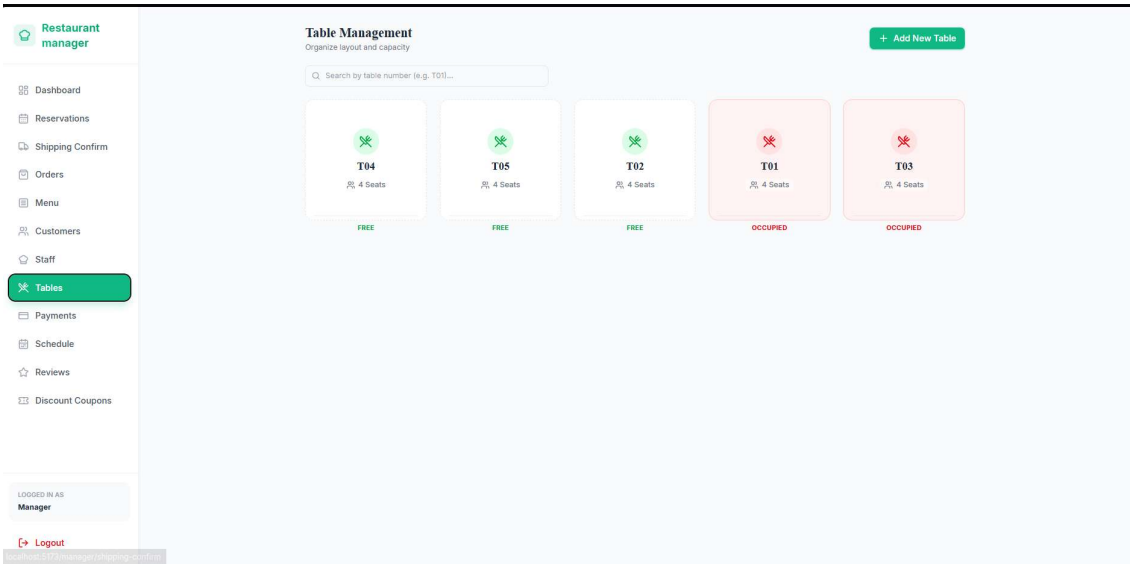


Figure 4.24: Table Management Interface

- Staff Scheduling and Roster Management : assignment of staff shifts across predetermined weekly schedules, ensuring appropriate staffing coverage across kitchen and service roles.

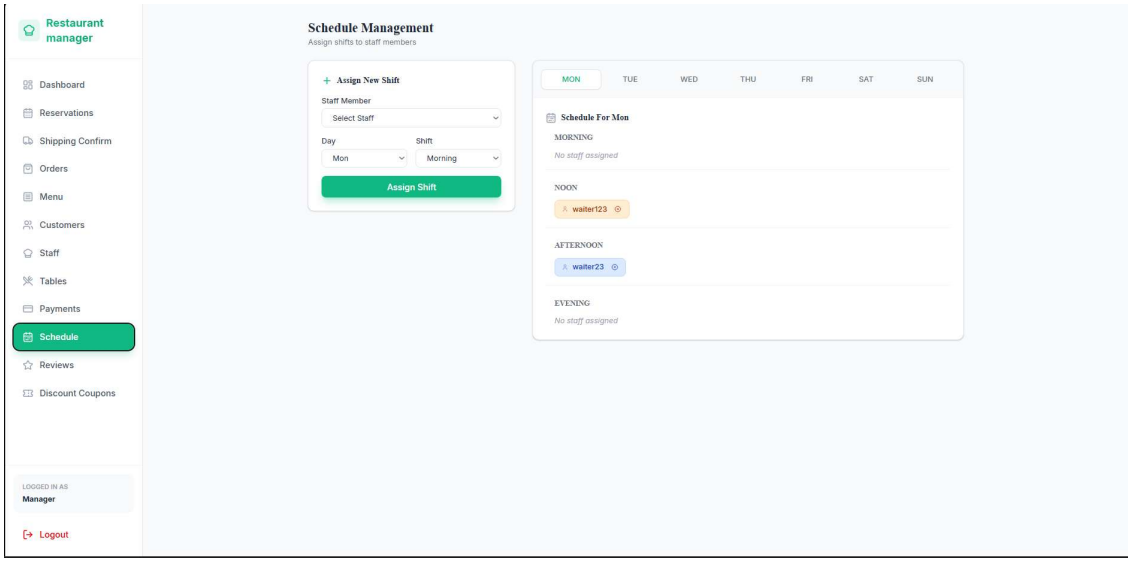


Figure 4.25: Scheduling Interface

4.2.5. Table Reservation and Pre-order Management Interfaces

The reservation subsystem assists with making bookings for visits to restaurants, providing for table assignments, and for pre-ordering food items.

- **Customer Reservation and Table Booking Portal**

The customer reservation module lets users specify the date of visit, time slot, expected duration, and number of guests. There is a detailed view of tables that shows their locations and seating capacity. The customers can enter their special dietary needs, and request advance orders for dishes, and can view the deposit sum determined by the system. After users fill in their reservation details and confirm their deposit payments through a digital reference, the reservation is registered with a "Pending" status in the customer's reservation history.

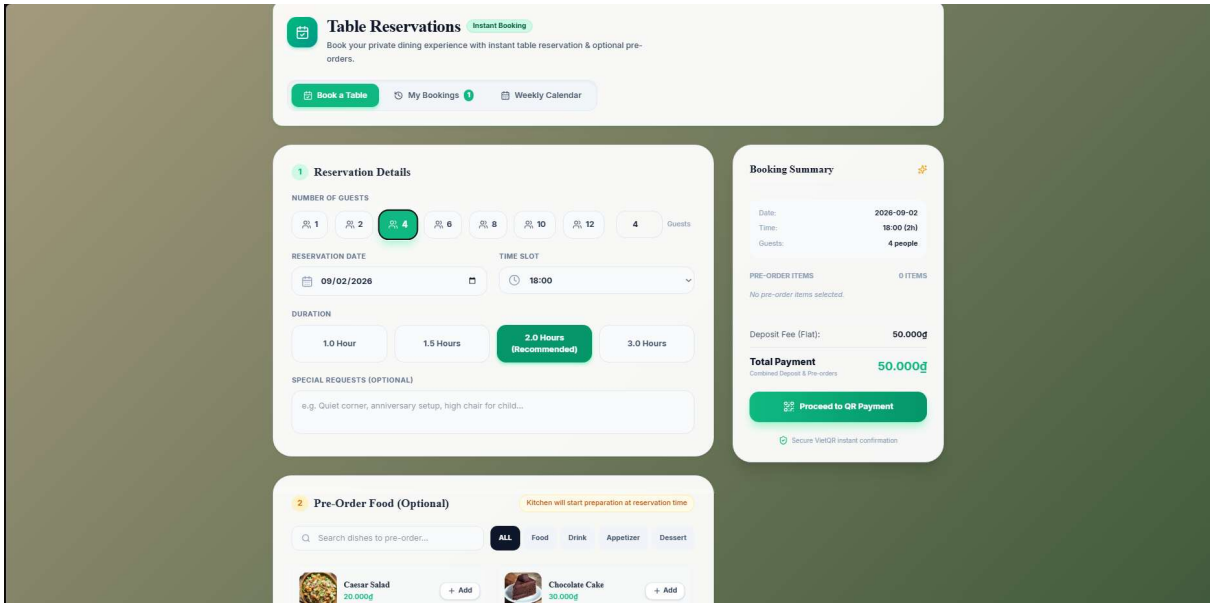


Figure 4.26: Customer Table Reservation and Pre-order Interface

- **Administrative Reservation Management and Staff Assignment**

Incoming reservation requests are overseen by restaurant managers and receptionists through a specific reservations coverage interface, which provides a table-style and calendar-style list of reservations filtered by status. Administrative staff can view customer information, confirm deposits, and allocate the staff members previously associated with certain reservations by means of the staff assignments. When reservations are approved, the given table is assigned to the corresponding time without creating any conflicts in the dining room layout.

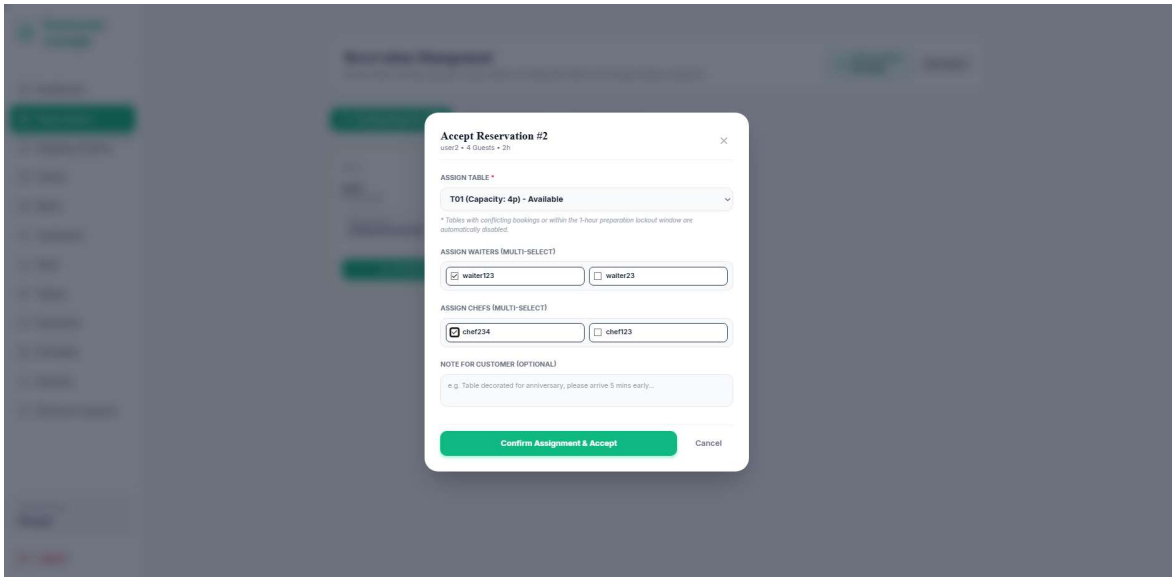


Figure 4.27: Reservation Management and Staff Assignment Interface

4.2.6. Promotional Discount and Coupon Management Interfaces

The subsystem for coupons and discounts features an automated approach for rewarding customers in compliance with specified thresholds and also provides a real-time mechanism for redeeming coupons at the time of purchase.

- **Customer Coupon Wallet**

A customer can navigate through the promotional wallet available in their profile and view all coupons eligible, including discount percentages, expiry date of every coupon, and its use status. Following that, during checkout as explained in Section 4.2.2, there will be a special field provided where a client has to enter a promo code. The program will check the validity of the promo code and get feedback on the code without waiting for the program to finish running. In case the code is valid, the last cost will be calculated instantly and the results will show the total cost broken down by subtotal, how much reduction due to promo codes, and final tax amount.

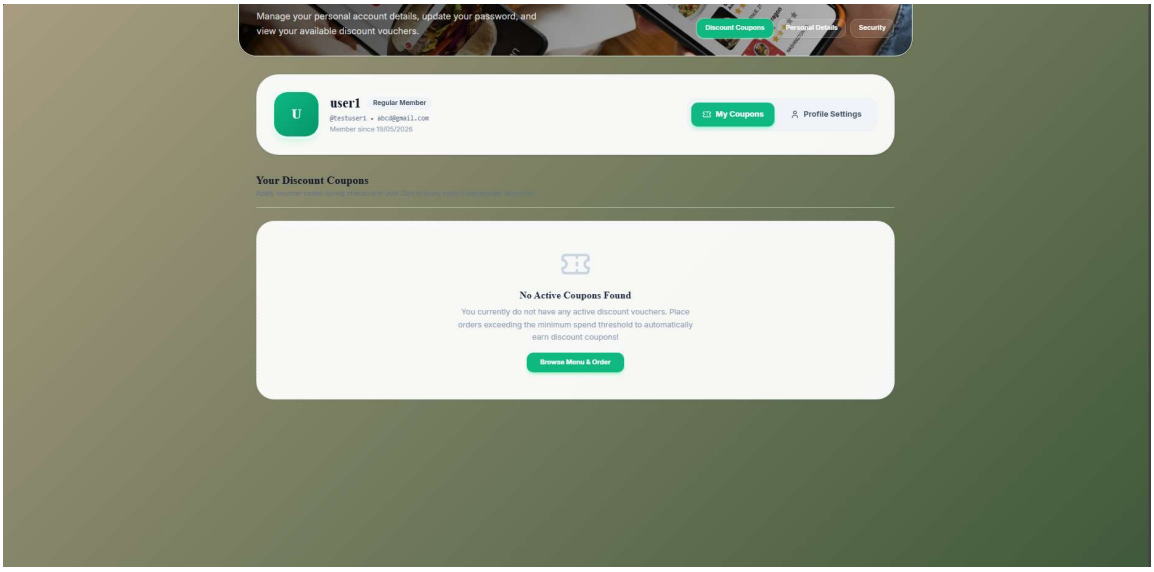


Figure 4.28: Customer Coupon Redemption

- **Coupon Rule and Threshold Configuration**

Managers can adjust promotional management tools which will enable them to create automated customer reward systems. The structured modal forms allow managers to set spend thresholds (for example the minimum amount of spending needed for receiving a reward), establish discount percentages, define time units of validity (days, weeks, or months) and activate or deactivate certain rules.

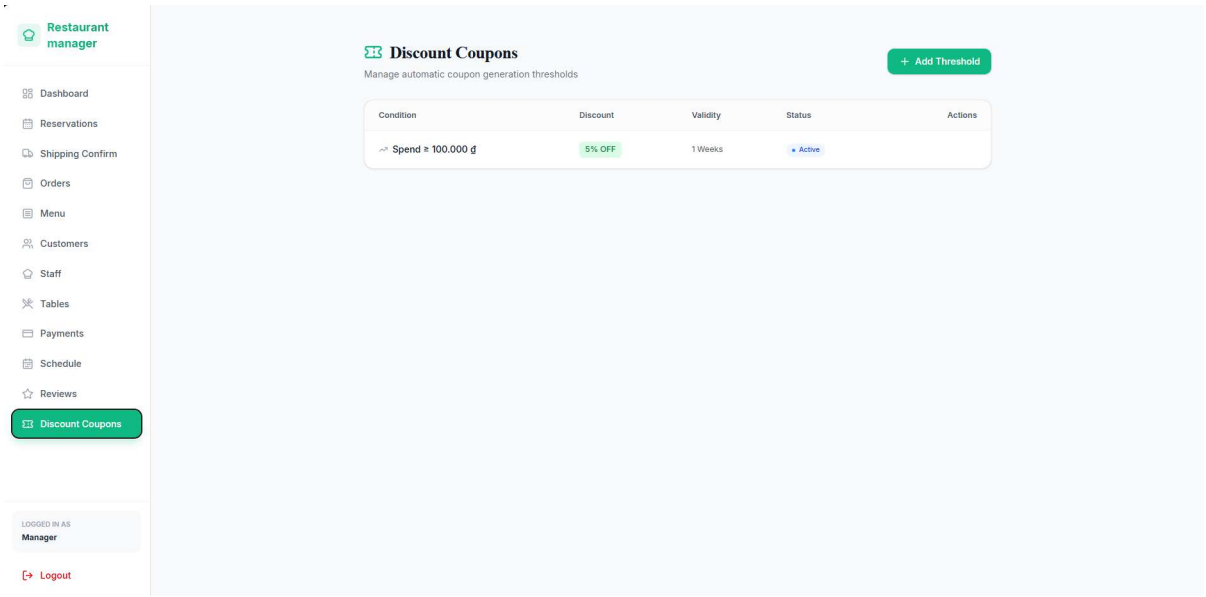


Figure 4.29: Discount Threshold and Coupon Configuration Interface

CHAPTER 5

DISCUSSION AND EVALUATION

5.1. Discussion

Design and implementation of the "River Side" Restaurant Management System has been motivated by a desire to close the gap between a high-class dining experience and efficient operations. The following key aspects implemented with regard to the thesis need to be discussed:

5.1.1. "Pay-First" Operational Model

The choice of one of the most interesting architectures has been a decision to use a "pay-first" approach to the processing of electronic orders. The "pay-first" operation involves collecting money at the moment when the order is created (VietQR code or money confirmation). Thus, there is no possibility of skipping out; this allows waiters to stop managing transactions and start providing services as soon as the payment is made.

5.1.2. Technical Architecture and Maintainability

The decision to utilize FastAPI as a back-end framework and React as the front end was a wise one, along with a rigid structure based on Layered Architecture (Controller-Service-Repository). This resulted in the isolation of business logic (e.g. duplicate field, table occupation logic) from database queries and routing via API. It will be much easier to add extensions (e.g, an app for delivery drivers) in the future without changing business logic.

5.2. Comparison

For analyzing the competitive position of the "River Side" system, it is compared against two leaders on the market, namely Haidilao and KFC.

Features	**Restaurant Management System**	**Haidilao [30]**	**KFC[29]**
Service Model	Dine-in/ Shipping/ Reservation	Dine-in/Shipping /Reservation	Dine-in/Shipping/ Reservation
Ordering Flow	Order → Pay → Prepare	Order → Prepare → Pay	Order → Pay → Prepare
Real-time Tracking	Order Status	Internal Dashboard for Haidilao location	Order Ready Boards

Table 5.2 Features of 3 systems

5.2.1. Compare with Haidilao

	Similarities	Differences
Restaurant Management System	-Provides support for Dine-in and Shipping orders. -Utilizing digital interfaces for table ordering and status updates	-Free individual item choices -Account login for quick preferences -Various food categories
Haidilao [30]		-Highly customized for massive

		scale and physical management -Offers restaurant tracking location -Focus on combo menus -Q&A feature -Review feature

Table 5.2.1 Comparison with Haidilao

5.2.2. Compare with KFC

	Similarities	Differences
Restaurant Management System	-Provides for Dine-in and Shipping orders. - Implement "Pay-First" models, ensuring speed and minimizing transaction errors -Implement account system for quick preferences -Free individual item choices -Various food categories	-Detailed table management required for sit-down dining.
KFC [29]		-Optimized for extreme speed and turnover -Promote for combo -Offers restaurant tracking location - E-gift card system -Shipping tracking

Table 5.2.2 Comparison with KFC

5.3. Limitations

5.3.1. Performance

The system uses synchronous SQLAlchemy sessions with the psycopg2 driver. Database queries and transaction commits therefore block the request-processing thread while they are being executed. The payment workflow also requests QR images from an external VietQR service, so response time depends partly on the availability and latency of that service. In addition, the customer payment page periodically checks the payment status, which generates repeated requests while a payment remains pending. These factors may affect response time under heavy concurrent usage.

5.3.2. Scalability

The current system uses a single relational database and performs order-queue dispatching within the application process. Queue entries are checked sequentially, and staff assignment may require several database operations for each order. Menu images are stored on the backend server rather than in dedicated object storage. This design is suitable for a small restaurant deployment, but larger workloads would require database connection-pool tuning, caching, background job processing, centralized file storage, and possibly horizontal application scaling.

5.3.3. Security

The system provides password hashing and JWT-based authentication, but access control is not yet applied consistently to all backend routes. Some staff, manager, payment, and configuration endpoints do not explicitly require the corresponding role dependency. In particular, payment status changes and some administrative operations require stronger authorization and

ownership checks before production deployment. The SePay webhook is protected by an API key, but the key must be stored securely and rotated when necessary. The system should also use HTTPS, validate webhook

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1. Conclusion

Designing and implementing the Restaurant Management System proved to be an outstanding success, resulting in the completion of a complete restaurant management system. This is because the system is a multi-role application that provides services to the customer, kitchen staff, waiters, delivery staff, and administrators in one integrated system.

Until now, this system has managed to achieve:

- Efficiency of Operations: The adoption of the "Pay-First" approach and the integration of the Kitchen Display System (KDS) have increased efficiency from receiving an order to its fulfillment.
- Improved Customer Engagement: With the help of a map of table selection and the VIP membership program, the app has offered a unique customer experience at par with the service offered by the best restaurants in the city.
- Strong Technical Framework: By applying a modular architecture with FastAPI and React, the software can ensure optimal performance while remaining flexible to possible future developments in technology.
- Versatility: The platform's ability to handle both dine-in (with table allocation) and delivery orders (with shipping tracking) can provide the company with a more diverse portfolio of services.

In conclusion, the current work shows that modern web development tools can be used to create a complex restaurant management system that combines the fast pace of quick-service restaurants with the high quality of service offered by full-service restaurants.

6.2. Future work

While the current version of the system provides a solid foundation, yet, several areas have been mark as potential for future development to further optimize restaurant operations:

6.2.1. Mobile applications

Develop a native iOS and Android application for enhancing user experience for customers and staffs. Some features that would allow for:

- GPS Tracking: real-time tracking location of delivery man in shipping
- Notifications: for customer will alert for order readiness, promotion deals; for staffs allow quick communication between Manager and staff members

6.2.2. AI-driven Analytics

Integrating an AI to system which provides feature such as:

- Menu Recommendations: recommend module for customers based on their past orders, popular pairings, and preferences
- Market monitor: alert adjusting and recommending menu price based on market fluctuations.

6.2.3. Automated Inventory Management

System will be granted with tracking raw ingredient stock by monitoring quantities of a menu item sold and automatically deducting required ingredient in stock. Alert Manager when ingredient stock below low lever and demand for refill.

REFERENCES

[1] MDN Web Docs, "Single-page application (SPA)," Mozilla Developer Network, 2024. [Online]. Available: https://developer.mozilla.org/en-US/docs/Glossary/SPA

[2] Meta Platforms, Inc., "React – A JavaScript library for building user interfaces," React Documentation, 2024. [Online]. Available: https://react.dev/

[3] C. Chedeau, "React's diff algorithm," Meta Engineering Blog, 2013. [Online]. Available: https://legacy.reactjs.org/docs/reconciliation.html

[4] React Team, "React Compiler," React Documentation, 2024. [Online]. Available: https://react.dev/learn/react-compiler

[5] E. You, "Vue.js – The Progressive JavaScript Framework," Vue.js Documentation, 2024. [Online]. Available: https://vuejs.org/

[6] T. Linsley, "TanStack Query Documentation," 2024. [Online]. Available: https://tanstack.com/query/latest

[7] Google LLC, "Angular – The modern web developer's platform," Angular Documentation, 2024. [Online]. Available: https://angular.dev/

[8] A. Wathan, "Tailwind CSS Documentation," Tailwind Labs, 2024. [Online]. Available: https://tailwindcss.com/docs

[9] Tailwind Labs, "Tailwind CSS v4.0," Tailwind CSS Blog, 2025. [Online]. Available: https://tailwindcss.com/blog/tailwindcss-v4

[10] S. Ramírez, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[11] S. Colvin, "Pydantic Documentation," 2024. [Online]. Available: https://docs.pydantic.dev/

[12] Django Software Foundation, "ASGI Specification," 2024. [Online]. Available: https://asgi.readthedocs.io/

[13] S. Colvin, "Pydantic V2 Plan," Pydantic Blog, 2023. [Online]. Available: https://docs.pydantic.dev/latest/concepts/performance/

[14] OpenJS Foundation, "Express.js – Fast, unopinionated, minimalist web framework for Node.js," 2024. [Online]. Available: https://expressjs.com/

[15] Django Software Foundation, "Django Documentation," 2024. [Online]. Available: https://docs.djangoproject.com/

[16] Django Software Foundation, "Asynchronous support," Django Documentation, 2024. [Online]. Available: https://docs.djangoproject.com/en/5.0/topics/async/

[17] TechEmpower, "TechEmpower Framework Benchmarks," 2024. [Online]. Available: https://www.techempower.com/benchmarks/

[18] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[19] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015.

[20] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference*, 1999, pp. 81–91.

[21] The PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," 2024. [Online]. Available: https://www.postgresql.org/

[22] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[23] T. Lane, "MVCC in PostgreSQL," PostgreSQL Wiki, 2024. [Online]. Available: https://wiki.postgresql.org/wiki/MVCC

[24] Oracle Corporation, "MySQL Documentation," 2024. [Online]. Available: https://dev.mysql.com/doc/
[25] MongoDB, Inc., "MongoDB Documentation," 2024. [Online]. Available: https://www.mongodb.com/docs/
[26] MongoDB, Inc., "Transactions," MongoDB Documentation, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/
[27] M. Bayer, "SQLAlchemy – The Database Toolkit for Python," 2024. [Online]. Available: https://www.sqlalchemy.org/
[28] OWASP Foundation, "SQL Injection Prevention Cheat Sheet," 2024. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
[29] KFC website reference: https://kfcvietnam.com.vn
[30] Haidilao website reference: https://haidilaohotpot.vn/vi
[31] F. D. Davis, “Perceived Usefulness, Perceived Ease of Use, and User Acceptance of Information Technology,” MIS Quarterly, vol. 13, no. 3, pp. 319–340, 1989, doi: 10.2307/249008.
[32] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Algorithms (JWA),” RFC 7518, Internet Engineering Task Force, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7518.
[33] K. H. Mandabach, G. F. Blanch, D. Vanleeuwen, D. A. Revelas, and R. P. Cole, “Who's in Charge Here: Restaurant Managers or Point of Sale Systems?” Journal of Foodservice Business Research, vol. 6, no. 3, pp. 105–117, 2003. doi: 10.1300/J369v06n03_07.
[34] “Unpacking the adoption behavior of restaurant management systems: A unified model,” Telematics and Informatics Reports, vol. 20, article 100278, Dec. 2025, doi: 10.1016/j.teler.2025.100278.